

MTH8304 - Apprentissage non-supervisé et séries chronologiques

Un premier classifieur: les k plus proches voisins

Julie Carreau

julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

Classifieur : fonction discriminante

$$\mathbf{x} \in \mathbb{R}^d \mapsto t \in \{1, \dots, K\}$$

$\mathbf{x}$: descripteurs/attributs d'un "objet" et t sa classe ou son type

Espèces d'iris

Un iris est décrit par la longueur et la largeur des pétales et des sépales.

Il y a plusieurs espèces de iris : setosa, versicolor, virginica

Classifier : déduire automatiquement l'espèce à partir de mesures de la fleur

MNIST

Un chiffre est décrit par $d = 28 \times 28 = 784$ intensités de gris avec $x_i \in \{0, 1, \dots, 255\}$

Il y a 10 chiffres : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Classifier : déduire automatiquement le chiffre à partir de $\mathbf{x} \in \{0, 1, \dots, 255\}^{784}$

Nearest neighbor NN - le plus proche voisin PPV

Jeu de données d'entraînement :

$$\{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)\}$$

Soit un nouvel objet avec $\mathbf{x}^*$ le vecteur de descripteurs associé

Soit

$$j^* = \arg \min_j d(\mathbf{x}_j, \mathbf{x}^*)$$

Où $d(\cdot, \cdot)$ est une distance, e.g.,

$$d(\mathbf{x}_j, \mathbf{x}_k) = \sqrt{\sum_{i=1} (x_{j,i} - x_{k,i})^2}$$

→ j^* est le plus proche voisin du nouvel objet

La classe assignée au nouvel objet est t_{j^*} , la classe de son plus proche voisin

Qu'est-ce qu'une distance ?

K nearest neighbors KNN - les K plus proches voisins KPPV

Jeu de données d'entraînement :

$$\{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)\}$$

Soit un nouvel objet avec $\mathbf{x}^*$ le vecteur de descripteurs associé

Soient $\{j_1, \dots, j_K\}$ les K plus proches voisins du nouvel objet

→ $\mathbf{x}_{j_1}, \dots, \mathbf{x}_{j_K}$ sont les K vecteurs de descripteurs les plus proches de $\mathbf{x}^*$

La classe assignée au nouvel objet est la classe majoritaire parmi les classes des K plus proches voisins $t_{j_1}, \dots, t_{j_K}$

Données iris dans R

```
dim(iris)
class(iris)
summary(iris)
str(iris)
head(iris)
tail(iris)
```

Données iris dans R (suite)

```
library("RColorBrewer")
library(ggplot2)
theme_set(theme_bw(18))

ggplot(iris,aes(x=Sepal.Length,y=Sepal.Width,col=Species)) +
  geom_point(size=2.2, col ="black") + geom_point(size=2)+
  scale_colour_manual(values = brewer.pal(n =3, name = "Accent"))

ggplot(iris,aes(x=Petal.Length,y=Petal.Width,col=Species)) +
  geom_point(size=2.2, col ="black") + geom_point(size=2)+
  scale_colour_manual(values = brewer.pal(n =3, name = "Accent"))
```

Question

Comment changer les titres des axes et de la légende en français ?
Comment sauvegarder les graphiques ?

En utilisant les données `iris` et les mesures des sépales seulement :

- ❶ Classer le 1^{er} élément à partir des 149 éléments restants
- ❷ Généraliser le code pour un élément quelconque
- ❸ Appliquer le code sur chaque ligne d'une grille de valeurs (ci-dessous) et faire le graphique des classes résultantes pour représenter les frontières entre les classes :

```
ngrid <- 50

iris.grid <- expand.grid(
  Sepal.Length = seq(min(iris$Sepal.Length), max(iris$Sepal.Length),
    length = ngrid),
  Sepal.Width = seq(min(iris$Sepal.Width), max(iris$Sepal.Width),
    length = ngrid))

iris.grid$Species <- factor(rep(NA, nrow(iris.grid)),
  levels = levels(iris$Species))
```

→ Modifier le code développé ci-dessus pour prendre en compte K voisins

Appliquer à nouveau sur la grille de valeurs en prenant soin de ré-initialiser la variable contenant les classes :

```
iris.grid$Species <- factor(rep(NA, nrow(iris.grid)),  
                             levels = levels(iris$Species))
```

Tracer les frontières entre les classes pour $K = 3, 9, 27$ et 81 et comparer en incluant dans la comparaison le cas $K = 1$.

MTH8304 - Apprentissage non-supervisé et séries chronologiques

3. Régression logistique et méthodes de ré-échantillonnage

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

- 1 Introduction
- 2 Régression logistique
- 3 Méthodes de ré-échantillonnage
 - Validation croisée
 - Bootstrap

- 1 Introduction
- 2 Régression logistique
- 3 Méthodes de ré-échantillonnage



Crédits figure : PRML C. Bishop

Reconnaissance de chiffres écrits à la main : chaque image contient 28×28 pixels.

- **Tâche** : étant donné $\mathbf{x} \in \mathbb{R}^{784}$, identifier le chiffre associé à l'image
- $\mathbf{x} \in C_k, k = 1, \dots, K$
- la plupart du temps : $C_k \cap C_j = \emptyset, j \neq k$ (partition)



Crédits figure : PRML C. Bishop

Reconnaissance de chiffres écrits à la main : chaque image contient 28×28 pixels.

→ **Tâche** : étant donné $\mathbf{x} \in \mathbb{R}^{784}$, identifier le chiffre associé à l'image

$\mathbf{x} \in C_k, k = 1, \dots, K$

la plupart du temps : $C_k \cap C_j = \emptyset, j \neq k$ (partition)

→ **Classifieur linéaire** : surfaces de décision sont linéaires

surfaces de décision : fonctions de $\mathbf{x}$ séparant $\mathbb{R}^d$ selon C_k

données linéairement séparables : classes exactement séparées par des surfaces linéaires

Classification supervisée (suite)

→ **Comment** encoder une réponse catégorique (qualitative)

problèmes si on utilise directement $\{0, 1, \dots, 9\}$

deux classes : $t \in \{0, 1\}$, $t = 1 \leftrightarrow \mathbf{x} \in C_1$ et $t = 0 \leftrightarrow \mathbf{x} \in C_2$

$$P(\mathbf{x} \in C_k) = t, \quad k = 1, 2$$

K classes : $\mathbf{t} \in \{0, 1\}^K$ tel que $t_j = 1 \leftrightarrow \mathbf{x} \in C_j$

$$P(\mathbf{x} \in C_j) = t_j, \quad j = 1, \dots, K$$

Classification supervisée (suite)

→ **Comment** encoder une réponse catégorique (qualitative)

problèmes si on utilise directement $\{0, 1, \dots, 9\}$

deux classes : $t \in \{0, 1\}$, $t = 1 \leftrightarrow \mathbf{x} \in C_1$ et $t = 0 \leftrightarrow \mathbf{x} \in C_2$

$$P(\mathbf{x} \in C_k) = t, \quad k = 1, 2$$

K classes : $\mathbf{t} \in \{0, 1\}^K$ tel que $t_j = 1 \leftrightarrow \mathbf{x} \in C_j$

$$P(\mathbf{x} \in C_j) = t_j, \quad j = 1, \dots, K$$

→ **Fonctions d'erreur** pour la classification

- erreur quadratique
- log-vraisemblance
- erreur de classification

Trois types d'approches pour classifier

① Fonction discriminante

une classe est attribuée directement à chaque vecteur x

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

Trois types d'approches pour classifier

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

Trois types d'approches pour classifier

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

③ Approche probabiliste générative

on cherche à estimer $P(\mathbf{x}|C_1)$ et par le théorème de Bayes

$$P(C_1|\mathbf{x}) = \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x})}$$

- loi normale multivariée - analyse discriminante (linéaire et quadratique)

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

③ Approche probabiliste générative

on cherche à estimer $P(\mathbf{x}|C_1)$ et par le théorème de Bayes

$$P(C_1|\mathbf{x}) = \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x})}$$

- loi normale multivariée - analyse discriminante (linéaire et quadratique)

- 1 Introduction
- 2 Régression logistique
- 3 Méthodes de ré-échantillonnage

Approche probabiliste discriminante

On modélise $P(C_k|\mathbf{x})$, $1 \leq k \leq K$

Règle de classification :

$$\mathbf{x} \in C_k \Leftrightarrow P(C_k|\mathbf{x}) > P(C_j|\mathbf{x}) \quad \forall j \neq k$$

On considère les modèles linéaires généralisés :

$$y(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x} + w_0)$$

où $f(\cdot)$ est appelée fonction d'activation (son inverse est la fonction de lien)

→ $f(\cdot)$ permet d'interpréter $y(\mathbf{x})$ comme des probabilités

→ Les surfaces de décisions sont linéaires :

$$y(\mathbf{x}) = \text{constante} \Leftrightarrow \mathbf{w}^T \mathbf{x} + w_0 = \text{constante}$$

→ Ces modèles ne sont pas linéaires en $\mathbf{w}$ (contrairement en régression), ce qui entraîne des propriétés plus complexes

Modèles linéaires avec fonctions de base fixes

$$y(\mathbf{x}, \mathbf{w}) = f(w_0 + w_1\phi_1(\mathbf{x}) + \cdots + w_{M-1}\phi_{M-1}(\mathbf{x})) = f(\mathbf{w}^T \phi(\mathbf{x}))$$

où $\mathbf{w} = (w_0, w_1, \dots, w_{M-1})^T$ et $\phi(\mathbf{x}) = (\phi_0(\mathbf{x}) = 1, \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))$

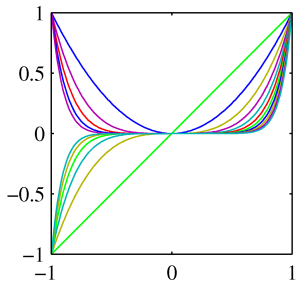
→ Fonctions de bases fixes $\phi_j(\cdot)$: pré-traitement ou extraction de caractéristiques

→ Si $\phi_j(\cdot)$ non-linéaires : surfaces de décision non-linéaires

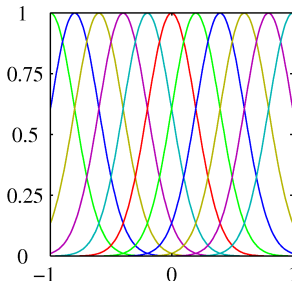
→ Modèles linéaires généralisés : même type de relation entre $y(\mathbf{x}, \mathbf{w})$ et $\mathbf{w}$

Choix possibles de fonctions de base

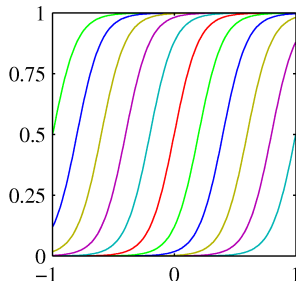
Polynômes (globales)



Gaussiennes (locales)



Sigmoïdes (locales)



Crédits figure : PRML C. Bishop

$$\phi_j(x) = x^j$$

$$\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\}$$

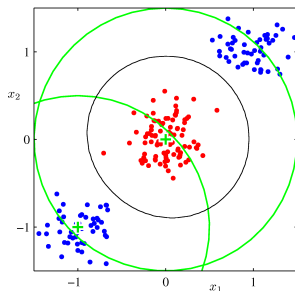
$$\begin{aligned}\phi_j(x) &= \sigma\left(\frac{x-\mu_j}{s}\right) \\ \sigma(a) &= \frac{1}{1+\exp(-a)}\end{aligned}$$

Autres fonctions de bases : splines (locales), Fourier (globales), ondelettes (locales)

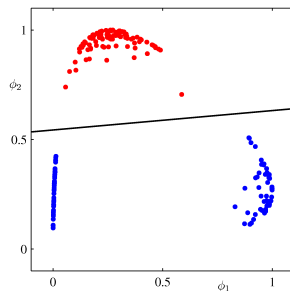
Illustration et limitations

Deux classes (rouge et bleue)

(x_1, x_2) l'espace d'origine



(ϕ_1, ϕ_2) , l'espace transformé



Crédits figure : PRML C. Bishop

- ϕ_j fonctions de base gaussiennes dont les centres et contours sont donnés par croix et ellipses vertes
- **Limitation** : les fonctions de base sont fixées *a priori* (fléau de la dimensionalité)
- **Alternatives** : réseaux à fonctions de base radiales et machines à vecteurs de support (réseaux de neurones artificiels)

Régression logistique : deux classes

$$P(C_1|\mathbf{x}) = y(\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^T \phi(\mathbf{x})) \quad P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$$

Fonction d'activation : fonction logistique ou sigmoïde ou "squashing function"

$$\sigma : \mathbb{R} \mapsto [0, 1]$$

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

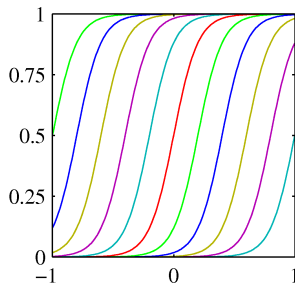
$$a = w_0 + w_1 x$$

Propriétés :

$$\sigma'(a) = \sigma(a)(1 - \sigma(a))$$

$$\sigma(-a) = 1 - \sigma(a)$$

$$a = \ln(\sigma/(1-\sigma))$$



Crédits figure : PRML C. Bishop

Règle de classification : $\mathbf{x} \in C_1 \Leftrightarrow y(\mathbf{x}, \mathbf{w}) \geq 0.5$

Fonction d'erreur : entropie croisée

Vraisemblance de $\{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_N, t_N)\}$, avec $y(\mathbf{x}_n) = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))$:

$$P(t_1, \dots, t_N | \mathbf{w}) = \prod_{n=1}^N y(\mathbf{x}_n)^{t_n} (1 - y(\mathbf{x}_n))^{1-t_n}$$

Loi de Bernoulli conditionnelle :

- $y(\mathbf{x}_n)$ ou $1 - y(\mathbf{x}_n)$: la probabilité du modèle pour succès ou échec
- t_n : le succès ou l'échec observé à l'exemple n

Log-vraisemblance négative ou la fonction d'erreur *d'entropie croisée* :

$$E(\mathbf{w}) = -\ln(P(t_1, \dots, t_N | \mathbf{w})) = -\sum_{n=1}^N \{t_n \ln(y(\mathbf{x}_n)) + (1-t_n) \ln(1-y(\mathbf{x}_n))\}$$

Fonction d'erreur : entropie croisée (suite)

Entraînement : minimiser l'entropie croisée

Gradient :

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y(\mathbf{x}_n) - t_n) \phi(\mathbf{x}_n)$$

- **Pas de solution analytique** contrairement à la régression
- **Optimisation numérique** : méthodes de descente du gradient
- **Limitation** : maximiser l'entropie croisée directement peut générer des solutions de sur-apprentissage en cas de données linéairement séparables
- **Alternative** : optimisation itérative de type *Newton-Raphson* appelée "les moindres carrés re-pondérés itératifs" ou *Iterative Reweighted Least Squares (IRLS)*

Régression logistique : K classes

Fonction d'activation : softmax

$$P(C_k|\mathbf{x}) = y_k(\mathbf{x}, \mathbf{w}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)} \quad \text{où } a_k = \mathbf{w}_k^T \phi(\mathbf{x})$$

Règle de classification : $\mathbf{x} \in C_k \Leftrightarrow y_k(\mathbf{x}, \mathbf{w}) \geq y_j(\mathbf{x}, \mathbf{w}) \quad \forall j \neq k$

Soit $\{(\mathbf{x}_n, \mathbf{t}_n)\}_{n=1}^N$ où $\mathbf{t}_n$ est un encodage un sur K .

Vraisemblance est donnée par

$$P(\mathbf{t}_1, \dots, \mathbf{t}_N | \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K y_k(\mathbf{x}_n)^{t_{nk}}$$

Log-vraisemblance négative ou la fonction d'erreur d'entropie croisée :

$$\begin{aligned} E(\mathbf{w}_1, \dots, \mathbf{w}_K) &= -\ln(P(\mathbf{t}_1, \dots, \mathbf{t}_N | \mathbf{w}_1, \dots, \mathbf{w}_K)) \\ &= -\sum_{n=1}^N \sum_{k=1}^K \{t_{nk} \ln(y_k(\mathbf{x}_n))\} \end{aligned}$$

- 1 Introduction
- 2 Régression logistique
- 3 Méthodes de ré-échantillonnage
 - Validation croisée
 - Bootstrap

Méthodes de ré-échantillonnage indispensables en modélisation statistique

Ces méthodes consistent

- à tirer au hasard des échantillons à partir de l'ensemble d'entraînement
- à entraîner le modèle d'intérêt sur chaque échantillon

But : obtenir de l'information additionnelle

- **Validation croisée** : estimer l'erreur de généralisation
 - évaluer la performance du modèle
 - sélectionner le niveau de complexité/flexibilité
- **Bootstrap** : estimer l'incertitude (la variabilité)
 - l'estimation d'un paramètre ou sur des prévisions du modèle

Le plus souvent : sélection du niveau de complexité de l'algorithme

- Le nombre de voisins k dans l'approche des K-plus-proches-voisins
- Le nombre de fonctions de base (l'ordre du polynôme) pour la régression logistique

→ **Principe** : sélectionner le niveau de complexité qui maximise la capacité à généraliser

Erreur de généralisation : erreur sur des données qui n'ont pas été vues auparavant

Ensemble de validation

Stratégie : séparer aléatoirement les données en un ensemble d'entraînement et un ensemble de validation

Soit $\mathcal{D}$ les données disponibles

Ens. d'entraînement : $\mathcal{D}^{\text{train}}$

Ens. de validation : $\mathcal{D}^{\text{valid}}$

$$\mathcal{D}^{\text{train}} \cup \mathcal{D}^{\text{valid}} = \mathcal{D} \quad \mathcal{D}^{\text{train}} \cap \mathcal{D}^{\text{valid}} = \emptyset$$

Pour chaque valeur de M , on calcule $E(\mathbf{w}; \mathcal{D}^{\text{train}})$ et $E(\mathbf{w}; \mathcal{D}^{\text{valid}})$

Erreur de généralisation $\approx E(\mathbf{w}; \mathcal{D}^{\text{valid}})$

Lorsque la complexité du modèle est trop faible :

→ pas assez de flexibilité (biais ou sous-apprentissage)

Lorsque la complexité du modèle est trop élevée :

→ trop de flexibilité (variance ou sur-apprentissage)

Deux difficultés avec cette approche :

- plusieurs façons de choisir $\mathcal{D}^{\text{valid}}$
 - l'estimation de l'erreur de généralisation peut être très variable
- un sous-ensemble des données est utilisé pour l'entraînement
 - si le jeu de données est petit, le modèle sera moins bien entraîné et l'erreur de validation pourra être sur-estimée

Validation croisée Leave-One-Out (LOO)

Ens. de validation :

$$\mathcal{D}_i^{\text{valid}} = \{(\mathbf{x}_i, t_i)\}$$

Ens. d'entraînement :

$$\mathcal{D}_i^{\text{train}} = \mathcal{D} \setminus \mathcal{D}_i^{\text{valid}}$$

→ tous sauf $i^{\text{ème}}$ exemple

$E(\mathbf{w}; \mathcal{D}_i^{\text{valid}})$: estimateur sans biais de l'erreur de généralisation mais très variable

On estime l'erreur de généralisation par :

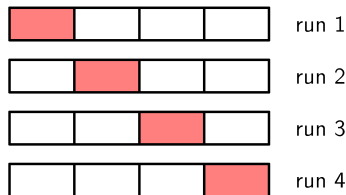
$$\frac{1}{N} \sum_{n=1}^N E(\mathbf{w}; \mathcal{D}_i^{\text{valid}})$$

→ $n - 1$ observations sont utilisées pour l'entraînement : moins de risques de sur-estimer l'erreur de généralisation

→ si on effectue plusieurs fois la validation LOO, on doit obtenir (en principe) les mêmes résultats

Validation croisée K-folds

Stratégie : séparer aléatoirement les données en K groupes (folds)



Crédits figure : PRML C. Bishop

Ens. de validation :

$\mathcal{D}_{(k)}^{\text{valid}}$ un des groupe

Ens. d'entraînement :

$\mathcal{D}_{(k)}^{\text{train}} = \mathcal{D} \setminus \mathcal{D}_{(k)}^{\text{valid}}$

→ tous sauf $k^{\text{ème}}$ groupe

On estime l'erreur de généralisation par :

$$\frac{1}{K} \sum_{k=1}^K E(\mathbf{w}; \mathcal{D}_{(k)}^{\text{valid}})$$

→ K-folds est une forme intermédiaire entre LOO ($K = n$) et la validation simple

→ K-folds est moins intense en calculs que LOO

→ K-folds est moins variable que la validation simple

Validation croisée K-folds (suite)

Biais

Validation simple : sur-estimation de l'erreur de généralisation si ens. d'entraînement petit

LOO : approximativement sans biais (moins de biais possible)

K-folds : intermédiaire entre validation simple et LOO

Validation croisée K-folds (suite)

Biais

Validation simple : sur-estimation de l'erreur de généralisation si ens. d'entraînement petit

LOO : approximativement sans biais (moins de biais possible)

K-folds : intermédiaire entre validation simple et LOO

Variance

LOO : entraînement de n modèles très semblables

Les prévisions des n modèles sont fortement positivement corrélées

K-folds : moins de corrélation car moins de recoupement entre les ens. d'entraînement

→ LOO a en général une variance plus élevée que K-folds

Validation croisée K-folds (suite)

Biais

Validation simple : sur-estimation de l'erreur de généralisation si ens. d'entraînement petit

LOO : approximativement sans biais (moins de biais possible)

K-folds : intermédiaire entre validation simple et LOO

Variance

LOO : entraînement de n modèles très semblables

Les prévisions des n modèles sont fortement positivement corrélées

K-folds : moins de corrélation car moins de recoupement entre les ens. d'entraînement

→ LOO a en général une variance plus élevée que K-folds

→ **Compromis bias-variance** pour le choix de K dans K-folds

→ Typiquement, $K = 5, 10$

Sélection de modèles ou évaluation de modèles via **estimation de l'erreur de généralisation**

Les techniques de validation simple, LOO et K-folds fournissent des estimations de l'erreur de généralisation

Ces techniques sont très générales, il suffit de définir la forme de la fonction d'erreur

Pour la classification, on peut utiliser le taux d'erreurs de classification :

$$E(\mathbf{w}; \mathcal{D}) = \frac{1}{N} \sum_{n=1}^N I(t_n \neq y_n)$$

Moyenne échantillonnale

Échantillon aléatoire : $X_1, X_2, \dots, X_N$ i.i.d.

$$\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i$$

Quelle est la distribution de $\bar{X}$?

→ Approximer la distribution échantillonnale par simulation

① $\mathcal{D}_1 = \{x_{1,1}, \dots, x_{1,N}\}$ un tirage aléatoire de N valeurs de la loi de X_1

② $\bar{x}_1 = \frac{1}{N} \sum_{n=1}^N x_{1,n}$

On répète P fois : $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_P$

→ $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_P$ est un tirage aléatoire de la distribution de $\bar{X}$

Soit $\mathcal{D} = \{x_1, \dots, x_N\}$ un tirage aléatoire de N valeurs de la loi de X_1

- ① $\mathcal{D}_1^* = \{x_{1,1}^*, \dots, x_{1,N}^*\}$ un tirage aléatoire avec remplacement de N valeurs parmi $\mathcal{D}$
- ② $\bar{x}_1^* = \frac{1}{N} \sum_{n=1}^N x_{1,n}^*$

On répète P fois : $\bar{x}_1^*, \bar{x}_2^*, \dots, \bar{x}_P^*$

→ $\bar{x}_1^*, \bar{x}_2^*, \dots, \bar{x}_P^*$ est approximativement un tirage aléatoire de la distribution de $\bar{X}$

→ Permet d'estimer la distribution échantillonnale dans des cas très généraux

→ Remplacer $\bar{x}_p^*$ par la statistique de votre choix : la prévision d'un modèle statistique, l'estimation d'un paramètre, etc. ...

MTH8304 - Apprentissage non-supervisé et séries chronologiques

5. Loi normale multivariée et modèles générateurs probabilistes

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

Plan

- 1 Loi normale multivariée
- 2 Modèles générateurs probabilistes

- 1 Loi normale multivariée
- 2 Modèles générateurs probabilistes

Fonctions de densité et moments

Une seule variable aléatoire $X \in \mathbb{R}$:

$$f_X(x; \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp \left\{ -\frac{1}{2\sigma^2} (x - \mu)^2 \right\}$$

L'espérance est $E[X] = \mu$ et la variance est $Var[X] = \sigma^2$

Fonctions de densité et moments

Une seule variable aléatoire $X \in \mathbb{R}$:

$$f_X(x; \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp \left\{ -\frac{1}{2\sigma^2}(x - \mu)^2 \right\}$$

L'espérance est $E[X] = \mu$ et la variance est $Var[X] = \sigma^2$

Un vecteur aléatoire $\mathbf{X} = (X_1, \dots, X_D) \in \mathbb{R}^D$:

$$f_{\mathbf{X}}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \right\}$$

où $\mathbf{x}, \boldsymbol{\mu} \in \mathbb{R}^D$, $\boldsymbol{\Sigma}$ est une matrice $D \times D$ et $|\boldsymbol{\Sigma}|$ est son déterminant

L'espérance est $E[\mathbf{X}] = \boldsymbol{\mu}$ et la covariance est

$$\begin{aligned} cov[\mathbf{X}] &= E[(\mathbf{X} - E[\mathbf{X}])(\mathbf{X} - E[\mathbf{X}])^T] = E[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^T] \\ &= \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \sigma_{12}^2 & \cdots & \sigma_{1D}^2 \\ \sigma_{21}^2 & \sigma_2^2 & \cdots & \sigma_{2D}^2 \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{D1}^2 & \sigma_{D2}^2 & \cdots & \sigma_D^2 \end{pmatrix} \Rightarrow \text{matrice symétrique} \end{aligned}$$

Théorème central de la limite

Soit $X_1, X_2, \dots, X_N$ des variables aléatoires i.i.d. avec $E[X_i] = \mu$ et $Var[X_i] = \sigma^2 > 0$ $1 \leq i \leq N$

Soit $\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i$ (aussi aléatoire), alors lorsque $N \rightarrow \infty$

$$\bar{X} \xrightarrow{\text{distribution}} N\left(\mu, \frac{\sigma^2}{N}\right)$$

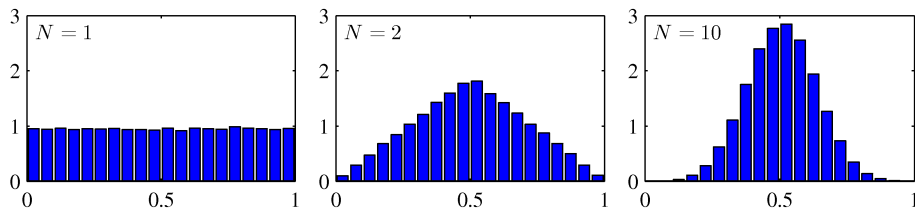
Théorème central de la limite

Soit $X_1, X_2, \dots, X_N$ des variables aléatoires i.i.d. avec $E[X_i] = \mu$ et $Var[X_i] = \sigma^2 > 0 \quad 1 \leq i \leq N$

Soit $\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i$ (aussi aléatoire), alors lorsque $N \rightarrow \infty$

$$\bar{X} \xrightarrow{\text{distribution}} N\left(\mu, \frac{\sigma^2}{N}\right)$$

Illustration TCL : $X_i \sim U[0, 1]$



Crédits figure : PRML C. Bishop

Propriétés géométriques

Distance de Mahalanobis :

$$\Delta = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

- Si $\boldsymbol{\Sigma}$ est l'identité, Δ est la distance euclidéenne
- Contours de densité $f_{\mathbf{X}}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \text{constante}$ correspondent à $\Delta = \text{constante}$

Propriétés géométriques

Distance de Mahalanobis :

$$\Delta = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

- Si $\boldsymbol{\Sigma}$ est l'identité, Δ est la distance euclidéenne
- Contours de densité $f_{\mathbf{X}}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \text{constante}$ correspondent à $\Delta = \text{constante}$

Illustration distance de Mahalanobis : cas bivarié

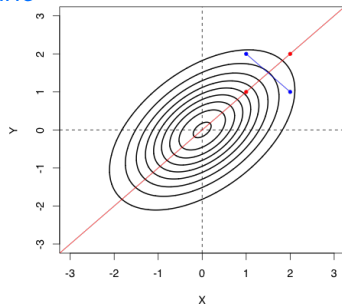
$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$$

où $\rho = \text{Cov}(X_1, X_2)/(\sigma_1\sigma_2)$ est la corrélation

Exemple : $\sigma_1 = \sigma_2 = 1$ $\rho = 0.5$

→ $\Delta \approx 1.15$ $\Delta = 2$

→ distance euclidéenne : $\sqrt{2} \approx 1.41$ dans les deux cas



Propriétés géométriques (suite)

Décomposition valeurs propres $\lambda_i \in \mathbb{R}$ - vecteurs propres $\mathbf{u}_i$, $1 \leq i \leq D$

$$\mathbf{\Sigma} \mathbf{u}_i = \lambda_i \mathbf{u}_i \quad \text{avec} \quad \mathbf{u}_i^T \mathbf{u}_j = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{sinon} \end{cases} \Rightarrow \text{orthonormés}$$

Alors on peut écrire : $\mathbf{\Sigma} = \sum_{i=1}^D \lambda_i \mathbf{u}_i \mathbf{u}_i^T$ et $|\mathbf{\Sigma}| = \prod_{j=1}^D \lambda_j$

$\mathbf{\Sigma}$ est définie positive ($\mathbf{w}^T \mathbf{\Sigma} \mathbf{w} > 0$, $\forall \mathbf{w} \neq 0$) : $\lambda_i > 0$, $1 \leq i \leq D$

Propriétés géométriques (suite)

Décomposition valeurs propres $\lambda_i \in \mathbb{R}$ - vecteurs propres $\mathbf{u}_i$, $1 \leq i \leq D$

$$\Sigma \mathbf{u}_i = \lambda_i \mathbf{u}_i \quad \text{avec} \quad \mathbf{u}_i^T \mathbf{u}_j = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{sinon} \end{cases} \Rightarrow \text{orthonormés}$$

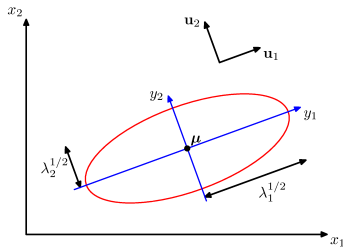
Alors on peut écrire : $\Sigma = \sum_{i=1}^D \lambda_i \mathbf{u}_i \mathbf{u}_i^T$ et $|\Sigma| = \prod_{j=1}^D \lambda_j$

Σ est définie positive ($\mathbf{w}^T \Sigma \mathbf{w} > 0$, $\forall \mathbf{w} \neq 0$) : $\lambda_i > 0$, $1 \leq i \leq D$

Posons $y_i = \mathbf{u}_i^T (\mathbf{x} - \mu)$

→ y_i : nouvelles coordonnées après translation et rotation

→ Ellipse rouge : $\Delta = 1$



Crédits figure : PRML C. Bishop

Paramétrisation de la matrice de variance-covariance

Il faut $D(D+1)/2$ paramètres pour définir Σ en raison de sa symétrie

→ **Attention** : pas évident de garantir que Σ soit définie positive

→ **Croissance quadratique** : difficulté lorsque D est grand, en particulier pour calculer Σ^{-1}

Paramétrisation de la matrice de variance-covariance

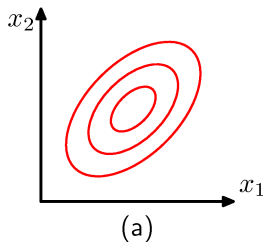
Il faut $D(D+1)/2$ paramètres pour définir Σ en raison de sa symétrie

→ **Attention** : pas évident de garantir que Σ soit définie positive

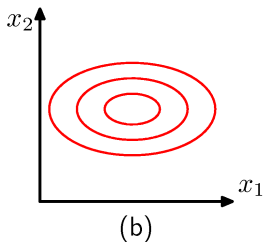
→ **Croissance quadratique** : difficulté lorsque D est grand, en particulier pour calculer Σ^{-1}

Formes restreintes de matrices de variance-covariance

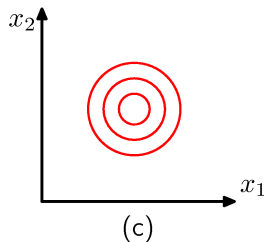
générale



diagonale



isotropique



Crédits figure : PRML C. Bishop

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12}^2 \\ \sigma_{21}^2 & \sigma_2^2 \end{pmatrix}$$

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{pmatrix}$$

$$\Sigma = \begin{pmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{pmatrix}$$

Lois marginales

Soit un vecteur aléatoire

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{pmatrix}$$

Alors la distribution marginale de $\mathbf{x}_a$ est

$$f(\mathbf{x}_a) = \int f(\mathbf{x}_a, \mathbf{x}_b) d\mathbf{x}_b$$

Si $\mathbf{x}$ suit une loi normale multivariée, posons

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix} \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}$$

Alors $\mathbf{x}_a$ suit aussi une loi normale multivariée avec

$$E[\mathbf{x}_a] = \boldsymbol{\mu}_a \quad \text{cov}[\mathbf{x}_a] = \boldsymbol{\Sigma}_a$$

→ Loi conditionnelle $\mathbf{x}_a | \mathbf{x}_b$ aussi gaussienne

Maximum de vraisemblance

Données : $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$

Vraisemblance :

$$L(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = \prod_{n=1}^N f(\mathbf{x}_n; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

Log-vraisemblance négative :

$$-\ln(L(\boldsymbol{\mu}, \boldsymbol{\Sigma} | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_n - \boldsymbol{\mu})$$

Estimateurs :

$$\boldsymbol{\mu}_{ML} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$$

$$\boldsymbol{\Sigma}_{ML} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu}_{ML})(\mathbf{x}_n - \boldsymbol{\mu}_{ML})^T$$

- 1 Loi normale multivariée
- 2 Modèles générateurs probabilistes

Trois types d'approches pour classifier

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

③ Approche probabiliste générative

on cherche à estimer $P(\mathbf{x}|C_1)$ et par le théorème de Bayes

$$P(C_1|\mathbf{x}) = \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x})}$$

- loi normale multivariée - analyse discriminante (linéaire et quadratique)

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

③ Approche probabiliste générative

on cherche à estimer $P(\mathbf{x}|C_1)$ et par le théorème de Bayes

$$P(C_1|\mathbf{x}) = \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x})}$$

- loi normale multivariée - analyse discriminante (linéaire et quadratique)

Modèles générateurs probabilistes

On modélise les probabilités *a priori* $P(C_k)$ et les probabilités *conditionnelles des classes* $P(\mathbf{x}|C_k)$, $1 \leq k \leq K$

Par le théorème de Bayes

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k)P(C_k)}{\sum_{j=1}^K P(\mathbf{x}|C_j)P(C_j)}$$

Deux classes

$$P(C_1|\mathbf{x}) = \frac{1}{1 + \exp(-a)} = \sigma(a)$$

Avec le logarithme du ratio des probabilité ou “log odds”

$$a = \ln \left(\frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x}|C_2)P(C_2)} \right)$$

K classes

$$P(C_k|\mathbf{x}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$$

Avec $a_k = \ln (P(\mathbf{x}|C_k)P(C_k))$

Densités conditionnelles des classes : loi gaussienne

$$\mathbf{x}|C_k \sim N(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad \forall k$$

Posons $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_k, \quad \forall k$

Densités conditionnelles des classes : loi gaussienne

$$\mathbf{x}|C_k \sim N(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \forall k$$

Posons $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_k, \forall k$

Deux classes

$$P(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + w_0)$$

Surface de décision

→ $P(C_1|\mathbf{x}) = 0.5 \Leftrightarrow \mathbf{w}^T \mathbf{x} + w_0 = 0$

→ Linéaires en $\mathbf{x}$ avec $\mathbf{w}, w_0$ définis en termes de $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ et $P(C_k)$

Densités conditionnelles des classes : loi gaussienne

$$\mathbf{x}|C_k \sim N(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \forall k$$

Posons $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_k, \forall k$

Deux classes

$$P(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + w_0)$$

Surface de décision

→ $P(C_1|\mathbf{x}) = 0.5 \Leftrightarrow \mathbf{w}^T \mathbf{x} + w_0 = 0$

→ Linéaires en $\mathbf{x}$ avec $\mathbf{w}, w_0$ définis en termes de $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ et $P(C_k)$

K classes

$$a_k(\mathbf{x}) = \sigma(\mathbf{w}_k^T \mathbf{x} + w_{k0})$$

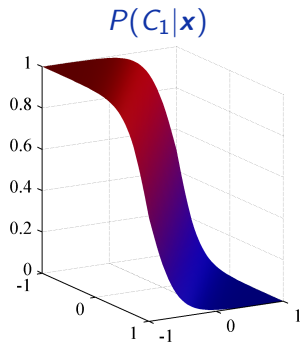
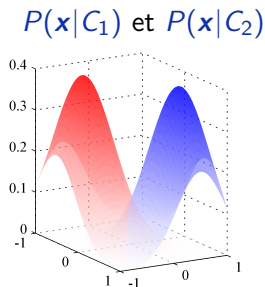
Surfaces de décision

→ $P(C_j|\mathbf{x}) = P(C_k|\mathbf{x})$ et $P(C_i|\mathbf{x}) \leq P(C_k|\mathbf{x}), \forall i \neq k, j$

→ Linéaires en $\mathbf{x}$ avec $\mathbf{w}_k, w_{k0}$ définis en termes de $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ et $P(C_k)$

Densités conditionnelles des classes : loi gaussienne (suite)

Illustration : deux classes (bleue et rouge)



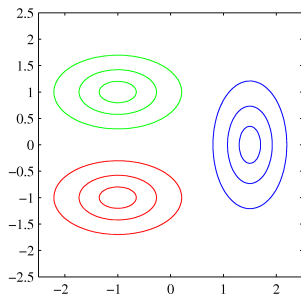
Crédits figure : PRML C. Bishop

Densités conditionnelles des classes : loi gaussienne (suite)

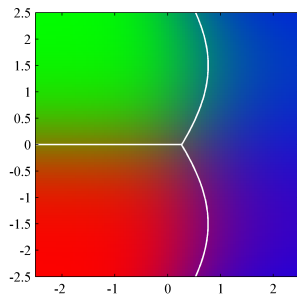
Surfaces de décision quadratique : Σ_k distinctes $\forall k$

Illustration : trois classes (bleue, rouge et verte)

$P(\mathbf{x}|C_1)$, $P(\mathbf{x}|C_2)$ et $P(\mathbf{x}|C_3)$



$P(C_1|\mathbf{x})$, $P(C_2|\mathbf{x})$ et $P(C_3|\mathbf{x})$



Crédits figure : PRML C. Bishop

Maximum de vraisemblance

Deux classes avec $\Sigma = \Sigma_k, k = 1, 2$

Ensemble d'entraînement :

$\{(\mathbf{x}_n, t_n)\}_{n=1}^N$, N_1 exemples sont dans C_1 et N_2 exemples sont dans C_2

Estimateurs :

$$P(C_1) = \frac{N_1}{N_1 + N_2}$$

$$\mu_1 = \frac{1}{N_1} \sum_{n=1}^N t_n \mathbf{x}_n$$

$$\mu_2 = \frac{1}{N_2} \sum_{n=1}^N (1 - t_n) \mathbf{x}_n$$

$$\Sigma = \frac{N_1}{N} \left(\frac{1}{N_1} \sum_{t_n=1} (\mathbf{x}_n - \mu_1)(\mathbf{x}_n - \mu_1)^T \right) + \frac{N_2}{N} \left(\frac{1}{N_2} \sum_{t_n=0} (\mathbf{x}_n - \mu_2)(\mathbf{x}_n - \mu_2)^T \right)$$

MTH8304 - Apprentissage non-supervisé et séries chronologiques

7. Arbres (bagging/boosting) et machines à vecteurs de support

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

Plan

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble
- 4 Fonctions discriminantes linéaires
- 5 Machines à vecteurs de support

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble
- 4 Fonctions discriminantes linéaires
- 5 Machines à vecteurs de support

① Fonction discriminante

une classe est attribuée directement à chaque vecteur $\mathbf{x}$

- KNN
- arbres de décision
- fonctions linéaires
- machines à vecteurs de support

② Approche probabiliste discriminante

on cherche à estimer $P(C_1|\mathbf{x})$ et s'en suit $P(C_2|\mathbf{x}) = 1 - P(C_1|\mathbf{x})$

- régression logistique

③ Approche probabiliste générative

on cherche à estimer $P(\mathbf{x}|C_1)$ et par le théorème de Bayes

$$P(C_1|\mathbf{x}) = \frac{P(\mathbf{x}|C_1)P(C_1)}{P(\mathbf{x})}$$

- loi normale multivariée - analyse discriminante (linéaire et quadratique)

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble
- 4 Fonctions discriminantes linéaires
- 5 Machines à vecteurs de support

Introduction

Les arbres de décision s'appliquent dans des contextes de régression ou de **classification**.

Ils reposent sur un partitionnement (ou segmentation) de l'espace des prédicteurs x en régions de décision homogènes.

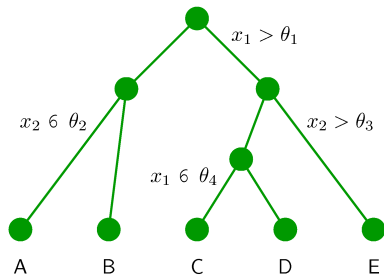
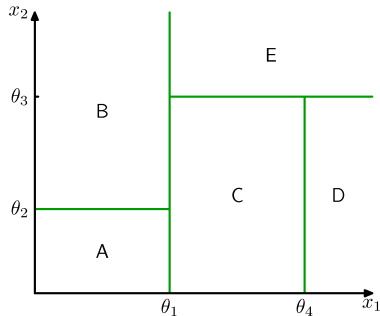
La prédiction consiste en la moyenne (régression) ou le mode (classification) des réponses dans la région à laquelle appartient le prédicteur.

L'ensemble des règles permettant de partitionner l'espace des prédicteurs peut se résumer en un arbre.

Les arbres de décision sont appréciés en raison de la facilité à les **interpréter**.

Tels quels, ils ne sont pas compétitifs. Leurs performances est fortement améliorées lorsqu'ils sont utilisés dans des **méthodes d'ensemble**.

Partionnement et arbre de décision



Crédits figure : PRML C. Bishop

→ Partition en 5 régions :

$$A = \{(x_1, x_2) | x_1 < \theta_1 \ \& \ x_2 < \theta_2\}$$

$$B = \{(x_1, x_2) | x_1 < \theta_1 \ \& \ x_2 > \theta_2\}$$

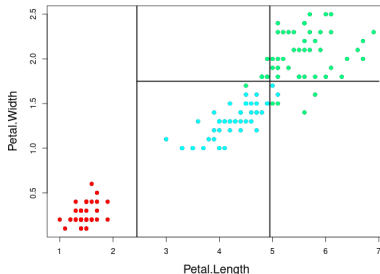
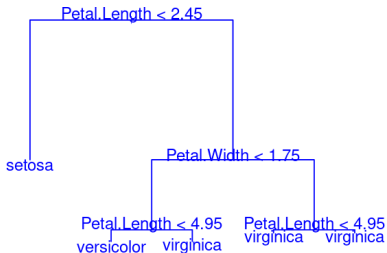
$$C = \{(x_1, x_2) | \theta_1 < x_1 < \theta_4 \ \& \ x_2 < \theta_3\}$$

$$D = \{(x_1, x_2) | x_1 > \theta_4 \ \& \ x_2 < \theta_3\}$$

$$E = \{(x_1, x_2) | x_1 > \theta_1 \ \& \ x_2 > \theta_3\}$$

Données iris

```
> library(tree)
> t.iris <- tree(Species ~ Petal.Length + Petal.Width, iris)
> summary(t.iris)
> plot(t.iris, col = "blue", lwd = 2)
> text(t.iris, col = "blue", cex = 2)
```



➔ Interprétation ?

- Feuilles (noeuds terminaux) : régions de décision
- L'arbre est inversé : la racine en premier et les feuilles en dernier
- Noeuds internes : séparation dans l'espace des prédicteurs

Prévision pour un nouvel exemple

Soit x , un nouveau point dans l'espace des prédicteurs :

- 1 Identifier dans quelle région de décision il appartient ;
- 2 La classe prédite par l'arbre est la classe majoritaire pour la région de décision identifiée (ou proportion de chaque classe).

```
> xtest <- data.frame(Petal.Length = 2.5, Petal.Width = 1)
> predict(t.iris, xtest)
  setosa versicolor  virginica
1      0  0.9791667  0.0208333
```

“Cultiver” un arbre

Comment trouver la partition optimale ?

“Cultiver” un arbre

Comment trouver la partition optimale ?

→ On cherche des régions de décision $R_1, R_2, \dots, R_J$ de forme rectangulaire de façon à minimiser :

$$\sum_{j=1}^J \sum_{i \in R_j} E_i,$$

où E_i résulte de l'évaluation de la fonction d'erreur sur l'exemple i .

→ Du point de vue calculatoire, impossible de considérer toutes les partitions en J régions.

→ Approche “top-down greedy” : séparation binaire récursive

- Démarrer à la racine de l'arbre (toutes les observations sont dans une seule région) puis procéder branche par branche en descendant dans l'arbre ;
- À chaque étape, déterminer la meilleure séparation possible sans se préoccuper des étapes successives.

“Cultiver” un arbre (suite)

Pour effectuer la séparation binaire récursive :

→ sélectionner un prédicteur x_j , $1 \leq j \leq p$ et le seuil de séparation s afin de déterminer deux régions

$$\{\mathbf{x} : x_j < s\} \quad \{\mathbf{x} : x_j > s\}$$

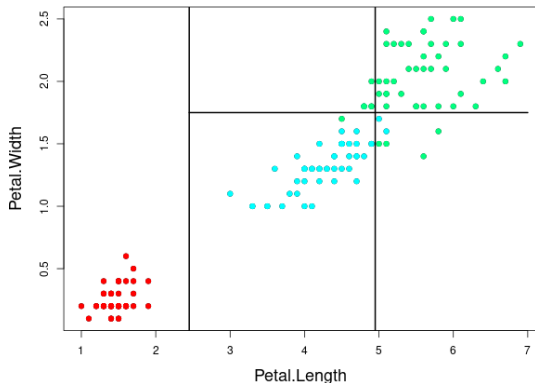
de sorte à générer la plus grande réduction dans la fonction d'erreur.

→ Recherche exhaustive : on parcourt tous les x_j , on optimise s et on retient la combinaison x_j et s donnant la réduction d'erreur la plus élevée.

→ Quand arrêter les itérations ?

On fixe un nombre maximal d'observations par feuille.

“Cultiver” un arbre (données iris)



- Nombre minimal d'observations dans un noeud enfant : 5
- Nombre minimal d'observations dans un noeud : 10
- Critère sur la réduction en erreur créée par la séparation

“Élaguer” un arbre

La procédure de séparation binaire réursive tend à créer des arbres trop complexes (sur-apprentissage).

Un arbre moins profond : diminuer la variance avec faible augmentation du biais.

Stratégie envisageable : aggrandir l'arbre tant que la réduction d'erreur est suffisamment importante.

Stratégie myope : une plus grande réduction d'erreur pourrait venir d'une branche plus profonde.

Stratégie habituelle :

- Créer un arbre très grand T_0
- **Élaguer l'arbre**

“Élaguer” un arbre (suite)

Comment procéder pour l'élagage ? On cherche le sous-arbre qui génère la plus faible erreur de généralisation.

Plutôt que d'estimer l'erreur de généralisation de tous les sous-arbres possibles, on s'appuie sur le résultat suivant :

→ l'ensemble des sous-arbres indexé par α correspondant à la minimisation de

$$T_\alpha = \arg \min \underbrace{\sum_{m=1}^{|T|} \sum_{x_i \in R_m} E_i}_{\text{adéquation}} + \alpha \underbrace{|T|}_{\text{complexité}},$$

où $|T|$ indique le nombre de feuilles, est une séquence imbriquée.

Autrement dit : $T_{0.1} \subseteq T_{0.01} \subseteq T_0 = T$

α : niveau de pénalité pour la complexité.

Plus $\alpha \uparrow$, plus l'arbre tend à être élagué.

→ α est choisi en minimisant l'erreur de généralisation

Fonctions d'erreur pour la classification

→ Réponse qualitative : prévision donnée par la classe dominante dans une feuille

→ On peut aussi s'intéresser aux proportions de chaque classe dans une feuille : $\hat{p}_{mk}$

Quelles fonctions d'erreur utiliser ?

Soit la région de décision m et soit $\hat{\mathbf{p}}_m = (\hat{p}_{m1}, \dots, \hat{p}_{mK})$ la proportion de chaque classe dans cette région.

Taux d'erreur de classification pour la région m est donné par :

$$E_m = 1 - \max_k \hat{p}_{mk}$$

→ Fraction des observations qui n'appartiennent pas à la classe dominante

Fonctions d'erreur pour la classification (suite)

→ L'erreur de classification n'est pas suffisamment sensible pour créer des arbres

→ On utilise plutôt deux autres mesures :

Indice de Gini

$$\sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

Mesure la variance totale entre les K classes

Entropie

$$-\sum_{k=1}^K \hat{p}_{mk} \ln(\hat{p}_{mk})$$

Semblable numériquement à l'indice de Gini

→ Ces deux quantités mesurent la pureté d'un noeud :

elles diminuent si $\hat{p}_{mk} \approx 0$ ou $\hat{p}_{mk} \approx 1, \forall k$

Même si l'erreur de classification ne réduit pas, il peut s'avérer important d'ajouter un noeud afin d'augmenter la pureté des noeuds et donc la certitude de la prévision.

- Partition avec séparations alignées avec les axes : sous-optimal si par exemple la surface de décision optimal est linéaire avec un angle de 45° ;
- La décision est “dure” : chaque région des prédicteurs est associée avec une décision unique (versus probabiliste, notamment près des frontières entre les régions) ;
- La structure d'un arbre est en général très sensible aux détails des données d'entraînement (grande variance). On verra des méthodes d'ensemble pour y remédier.

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble**
- 4 Fonctions discriminantes linéaires
- 5 Machines à vecteurs de support

Les méthodes d'ensemble combinent plusieurs modèles plus simples.

Ces modèles plus simples sont appelés “weak learners” : avec une faible/sans biais et un niveau de variance indésirable.

Bagging, forêt aléatoire et boosting sont des méthodes d'ensemble qui reposent sur des arbres.

Les arbres de décisions sont affectés d'un fort niveau de variance :

- si l'ensemble d'entraînement est modifié, l'arbre résultant peut être très différent ;
- les règles apprises par l'arbre reflètent de trop près les données plutôt que des mécanismes de la structure sous-jacente.

Bootstrap aggregation - bagging : procédure pour réduire la variance d'une méthode de modélisation statistique.

Rappel :

$$Z_1, Z_2, \dots, Z_n \quad \text{i.i.d. t.q.} \quad E[Z_i] = \mu \quad \text{et} \quad \text{Var}[Z_i] = \sigma^2 < \infty$$

Alors

$$\bar{Z} = \frac{1}{n} \sum_{i=1}^n Z_i \quad \text{avec} \quad E[\bar{Z}] = \mu \quad \text{et} \quad \text{Var}[\bar{Z}] = \frac{\sigma^2}{n}$$

→ Prendre la moyenne permet de réduire la variance

Bagging : fonctionnement

Soit $\mathcal{D}_1^*, \mathcal{D}_2^*, \dots, \mathcal{D}_B^*$, B échantillons obtenus par bootstrap (tirage aléatoire avec remplacement).

Soit $\hat{f}^{*b}(x)$, la méthode choisie entraînée sur $\mathcal{D}_b^*$, avec $1 \leq b \leq B$.

Le bagging consiste à prendre la moyenne des B méthodes :

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

- Applicable pour la régression et la classification avec une petite modification.
- Applicable à de nombreux modèles mais particulièrement utiles pour les arbres de décision : amélioration remarquable de l'erreur de généralisation.
- Performance peu sensible à la valeur de B choisie : on prend une valeur assez grande sans risque de sur-apprentissage.

Bagging : arbres de classification

- ① Créer B arbres de classification, chacun sur un échantillon bootstrap $\mathcal{D}_b^*$, avec $1 \leq b \leq B$;
- ② Ces arbres sont profonds et non élagués : biais faible et variance élevée ;
- ③ Chacun des B arbres est utilisé pour faire une prévision et retenir la classe prédite le plus souvent (vote de majorité).

Erreur “out-of-bag”

Estimer l'erreur de généralisation sans validation croisée :

pour un échantillon bootstrap $\mathcal{D}_b^*$, environ $1/3$ des observations sont laissées de côté.

Pour une observation i , on utilise les prévisions des arbres $\hat{f}^{*b}(x)$ pour lesquels $i \notin \mathcal{D}_b^*$;

→ on aura environ $B/3$ prévisions : on prend un vote de majorité.

On répète pour tout $1 \leq i \leq N$ et on pourra calculer une fonction d'erreur hors-échantillon (test) qui estime l'erreur de généralisation.

→ Si B est suffisamment grand, l'erreur “out-of-bag” est pratiquement équivalente à celle obtenue par validation croisée.

Le bagging améliore la performance mais au coût de la perte de la facilité d'interprétation de l'arbre.

Il est possible de produire un résumé de l'influence des prédicteurs à partir de la fonction d'erreur de notre choix :

- on cumule les diminutions de la fonction d'erreur à chaque noeud dues à un prédicteur particulier ;
- on fait la moyenne sur les B arbres.

Semblable au bagging avec une astuce supplémentaire : décorréliser les arbres.

→ Moins les arbres sont corrélés, plus le gain en variance est important en combinant les prévisions.

Fonctionnement :

- On construit B arbres sur des échantillons bootstrap ;
- Chaque fois qu'un noeud (règle de décision) est considéré, un **sous-ensemble aléatoire de prédicteurs** est utilisé parmi les p candidats.

→ Un choix fréquent : $m \approx \sqrt{p}$.

Principe de décorrélation

Supposons qu'il existe un prédicteur avec une très forte capacité à prédire et que les autres variables sont plus modérées ;

→ la plupart des arbres utilisés dans le bagging vont utiliser ce prédicteur au premier noeud (forte réduction de l'erreur)

→ tous les arbres vont donc se ressembler fortement et fournir des prévisions très corrélées.

Les forêts aléatoires corrigent ce problème en ne considérant qu'un sous-ensemble aléatoire de prédicteurs à chaque noeud ;

→ cette procédure permet de décorréler les arbres et de réduire la variance du bagging.

→ En pratique, on utilise souvent $m = \sqrt{p}$.

Combine plusieurs classifieurs de base (“weak learners”) de façon séquentielle (et non pas indépendante comme pour le bagging et les forêts aléatoires) ;

→ il suffit que leur performance soit meilleure qu’une réponse aléatoire (pile ou face).

Plusieurs variantes de boosting existent, on s’intéresse à une variante connue appelée *AdaBoost* pour “Adaptive boosting”.

Principe de base :

Chaque classifieur est entraîné sur des données pondérées avec des poids qui dépendent de la performance des classifieurs précédents ;

→ plus d’importance sera accordée aux points mal classifiés

→ les prévisions sont combinées au travers d’un vote de majorité pondéré (les classifieurs n’ont pas tous le même poids dans le vote).

Boosting : deux classes

$\mathbf{x}_1, \dots, \mathbf{x}_N$ des vecteurs d'entrée ou prédicteurs

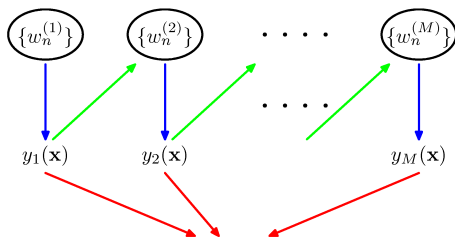
$t_1, \dots, t_N$ les réponses (sorties) binaires avec $t_n \in \{-1, 1\}$

Chaque classifieur est entraîné sur des données pondérées

Poids initiaux : $w_n = 1/N$

→ poids sont modifiés selon la performance du classifieur précédent

Lorsque le nombre de classifieurs désiré est atteint, leurs prévisions sont combinées.



$$Y_M(\mathbf{x}) = \text{sign} \left(\sum_m^M \alpha_m y_m(\mathbf{x}) \right)$$

Crédits figure : PRML C. Bishop

Boosting : algorithme

① Initialisation des poids : $w_n^{(1)} = 1/N$ avec $1 \leq n \leq N$

② Pour $1 \leq m \leq M$:

① Entraîner un classifieur $y_m(\mathbf{x})$ en minimisant l'erreur pondérée suivante :

$$J_m = \sum_{n=1}^N w_n^{(m)} I(y_m(\mathbf{x}) \neq t_n),$$

où $I(\cdot)$ est la fonction indicatrice.

② Calculer

$$\epsilon_m = \frac{\sum_{n=1}^N w_n^{(m)} I(y_m(\mathbf{x}) \neq t_n)}{\sum_{n=1}^N w_n^{(m)}}$$

et

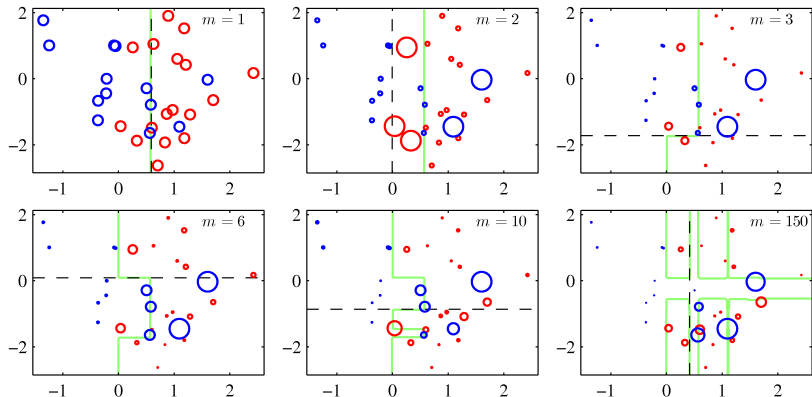
$$\alpha_m = \ln \left\{ \frac{1 - \epsilon_m}{\epsilon_m} \right\}$$

③ Mettre à jour les poids $w_n^{(m+1)} = w_n^m \exp\{\alpha_m I(y_m(\mathbf{x}) \neq t_n)\}$

③ Faire les prévisions avec le modèle final

$$Y_M(\mathbf{x}) = \text{sign} \left(\sum_{m=1}^M \alpha_m y_m(\mathbf{x}) \right).$$

Boosting : illustration



Crédits figure : PRML C. Bishop

Classifieurs de base : arbres avec un seul noeud “stumps”

Noir : surface de décision du dernier classifieur

Vert : surface de décision du boosting

Boosting : fonction d'erreur

On peut montrer que l'algorithme du boosting permet de minimiser de façon itérative la fonction d'erreur suivante :

$$\sum_{n=1}^N \exp\{-t_n f_m(\mathbf{x}_n)\}$$

où $f_m(\mathbf{x}_n)$ est un classifieur défini en termes de classifieurs de base $y_l(\mathbf{x})$:

$$f_m(\mathbf{x}) = \frac{1}{2} \sum_{l=1}^m \alpha_l y_l(\mathbf{x})$$

→ Cette interprétation du boosting a permis le développement de nombreuses extensions : multi-classes, régression, etc.

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble
- 4 Fonctions discriminantes linéaires**
- 5 Machines à vecteurs de support

Fonction discriminante linéaire : deux classes

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$$

$\mathbf{w} = (w_1, \dots, w_d)$: vecteur de poids

w_0 : biais (pas au sens statistique)

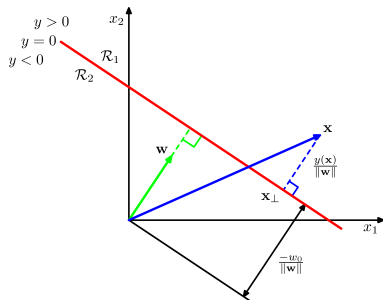
Règle de classification :

$$\mathbf{x} \in C_1 \Leftrightarrow y(\mathbf{x}) \geq 0 \quad \& \quad \mathbf{x} \in C_2 \text{ sinon}$$

Surface de décision : hyperplan

$$y(\mathbf{x}) = 0$$

Fonction discriminante linéaire : deux classes (suite)



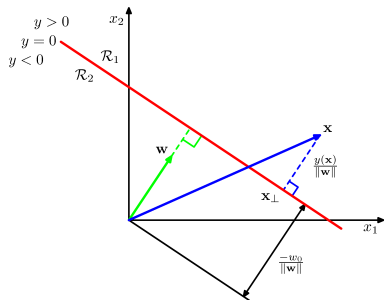
Crédits figure : PRML C. Bishop

Rôle de $\mathbf{w}$

$\mathbf{x}_A$ et $\mathbf{x}_B$ t.q. $y(\mathbf{x}_A) = y(\mathbf{x}_B) = 0$, alors
 $\mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) = 0$

→ $\mathbf{w} \perp$ tout vecteur sur la surface de décision, i.e., $\mathbf{w}$: orientation de la surface de décision

Fonction discriminante linéaire : deux classes (suite)



Crédits figure : PRML C. Bishop

Rôle de $\mathbf{w}$

$\mathbf{x}_A$ et $\mathbf{x}_B$ t.q. $y(\mathbf{x}_A) = y(\mathbf{x}_B) = 0$, alors
 $\mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) = 0$

→ $\mathbf{w} \perp$ tout vecteur sur la surface de décision, i.e., $\mathbf{w}$: orientation de la surface de décision

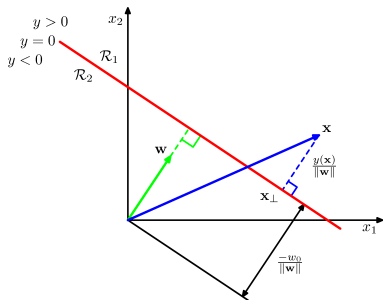
Rôle de w_0

$\mathbf{x}$ t.q. $y(\mathbf{x}) = 0$, la distance de l'origine :

$$\frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\|} = -\frac{w_0}{\|\mathbf{w}\|}$$

→ w_0 : position de la surface de décision

Fonction discriminante linéaire : deux classes (suite)



Crédits figure : PRML C. Bishop

Rôle de w

x_A et x_B t.q. $y(x_A) = y(x_B) = 0$, alors $w^T(x_A - x_B) = 0$

→ $w \perp$ tout vecteur sur la surface de décision, i.e., w : orientation de la surface de décision

Rôle de w_0

x t.q. $y(x) = 0$, la distance de l'origine :

$$\frac{w^T x}{\|w\|} = -\frac{w_0}{\|w\|}$$

→ w_0 : position de la surface de décision

Distance à la surface de décision

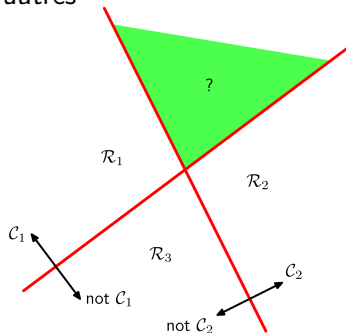
$$x = x_{\perp} + r \frac{w}{\|w\|} \quad (\times w^T \& + w_0) \Rightarrow r = \frac{y(x)}{\|w\|}$$

Fonction discriminante linéaire : K classes

Deux approches générant des régions ambiguës :

one-versus-the-rest :

$K-1$ classifieurs binaires séparant C_k
des autres



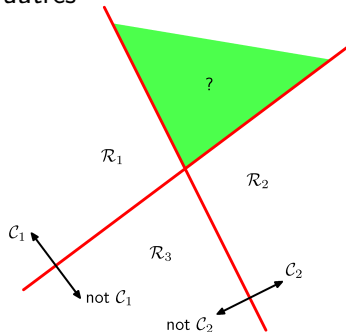
Crédits figure : PRML C. Bishop

Fonction discriminante linéaire : K classes

Deux approches générant des régions ambiguës :

one-versus-the-rest :

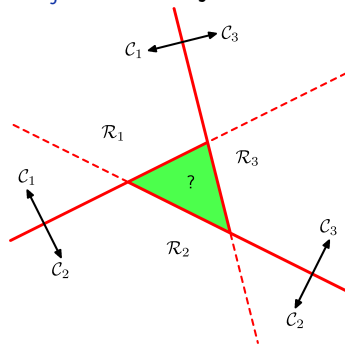
$K-1$ classifieurs binaires séparant C_k des autres



Crédits figure : PRML C. Bishop

one-versus-one :

$K(K-1)/2$ classifieurs binaires séparant C_k de C_j + vote majoritaire



Crédits figure : PRML C. Bishop

Fonction discriminante linéaire : K classes (suite)

Solution : un seul classifieur comprenant K fonctions linéaires :

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0} \quad 1 \leq k \leq K$$

Règle de décision :

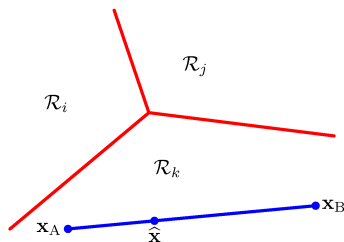
$$\mathbf{x} \in C_k \Leftrightarrow y_k(\mathbf{x}) > y_j(\mathbf{x}) \quad \forall j \neq k$$

Surface de décision entre C_k et C_j :

$$\begin{aligned} y_k(\mathbf{x}) &= y_j(\mathbf{x}) \Leftrightarrow \\ (\mathbf{w}_k - \mathbf{w}_j)^T \mathbf{x} + (w_{k0} - w_{j0}) &= 0 \end{aligned}$$

→ Hyper-plan : mêmes propriétés géométriques que pour 2 classes

→ Régions de décision : connectées et convexes



Crédits figure : PRML C. Bishop

Moindres carrés

Soit $\widetilde{\mathbf{W}}$ la matrice $(D+1) \times K$ t.q. la $k^{\text{ème}}$ colonne est $\tilde{\mathbf{w}}_k = (w_{k0}, \mathbf{w}_k^T)$

Soit $\mathbf{T}$ la matrice $N \times K$ t.q. la $n^{\text{ème}}$ ligne est $\mathbf{t}_n^T$ (encodage pour K classes)

La fonction d'erreur de la somme des carrés :

$$E_D(\widetilde{\mathbf{W}}) = \frac{1}{2} \sum_{n=1}^N \sum_{k=1}^K (y_k(\mathbf{x}_n) - \mathbf{T}_{n,k})^2$$

Estimateur des moindres carrés s'obtient :

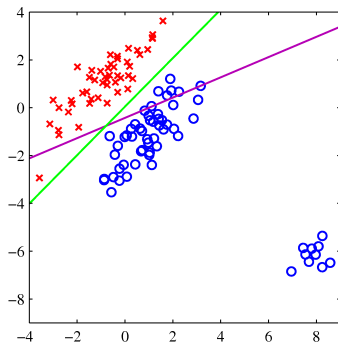
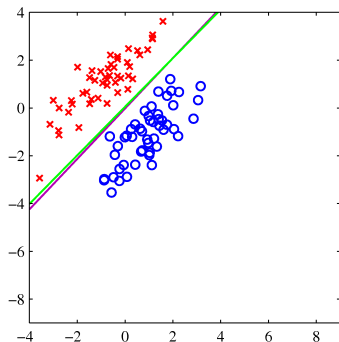
$$\widetilde{\mathbf{W}} = \arg \min_{\mathbf{V}} E_D(\mathbf{V})$$

→ Solution analytique pour $\widetilde{\mathbf{W}}$, i.e., comme une fonction de $\mathbf{x}_n$ et $\mathbf{t}_n$

Problèmes avec les moindres carrés

$$E_D(\widetilde{\mathbf{W}}) = \frac{1}{2} \sum_{n=1}^N \sum_{k=1}^K (y_k(\mathbf{x}_n) - \mathbf{T}_{n,k})^2$$

Exemple : les solutions “trop correctes” sont pénalisées



Crédits figure : PRML C. Bishop

→ Autres fonctions d'erreur : par exemple, les machines à vecteurs de support

- 1 Introduction
- 2 Arbres de décisions pour la classification
- 3 Méthodes d'ensemble
- 4 Fonctions discriminantes linéaires
- 5 Machines à vecteurs de support

Machines à vecteurs de support : introduction

Intermédiaire entre les méthodes paramétrisées et les méthodes basées sur la mémoire

Algorithmes développés dans les années 1990 : un des meilleurs classifieurs disponibles (deux classes, extension multi-classes pas évidente)

Trois étapes :

- **Classifieur à marge maximale** : surface discriminante est un hyperplan à marge maximale
→ applicable quand les classes sont linéairement séparables
- **Classifieur à vecteurs de support** : surface discriminante est un hyperplan mais des violations de la marge sont permises
- **Machines à vecteurs de support** : surface discriminante non-linéaire grâce à l'approche par noyaux ("kernel trick")

Classifieur à marge maximale

Fonction discriminante :

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

Région 1 (classe 1) :

$$y(\mathbf{x}) > 0 \iff \mathbf{w}^T \mathbf{x} + b > 0$$

Surface de décision (hyperplan) :

$$y(\mathbf{x}) = 0 \iff \mathbf{w}^T \mathbf{x} + b = 0$$

Région 2 (classe 2) :

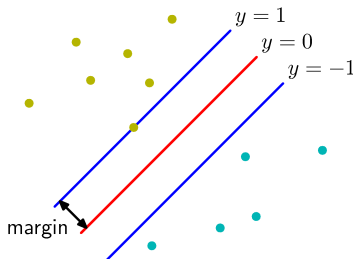
$$y(\mathbf{x}) < 0 \iff \mathbf{w}^T \mathbf{x} + b < 0$$

→ Une infinité d'hyperplans séparent parfaitement les classes

Distance à la surface de décision :

$$\frac{|y(\mathbf{x})|}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$$

→ Marge : plus petite distance à la surface de décision



Crédits figure : PRML C. Bishop

Classifieur à marge maximale

Soient $\mathbf{x}_1 = (x_{11}, \dots, x_{1p})^T, \dots, \mathbf{x}_N = (x_{N1}, \dots, x_{Np})^T$ dans $\mathbb{R}^p$

Soient $t_1, \dots, t_N$ dans $\{-1, 1\}$ (attention, souvent notés $y_1, \dots, y_N$)

Soient $\mathbf{w}$ et b t.q. $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ est séparé parfaitement bien les données, alors :

$$\begin{cases} y(\mathbf{x}_i) > 0 & \text{si } t_i = 1 \\ y(\mathbf{x}_i) < 0 & \text{si } t_i = -1 \end{cases} \iff t_i y(\mathbf{x}_i) > 0 \quad \forall i$$

- Le signe de $y(\mathbf{x})$ indique la classe ou région
- L'amplitude $|y(\mathbf{x})|$ proportionnelle à la distance à la surface de décision
- Plus $|y(\mathbf{x})|$ est grand, plus $\mathbf{x}$ est loin de la surface de décision et plus on est sûr de la classification

Classifieur à marge maximale

Pour un hyperplan donné, $\mathbf{w}$, b fixés t.q. $\|\mathbf{w}\|^2 = 1$

La marge est définie par : $M = \min_i |y(\mathbf{x}_i)|$

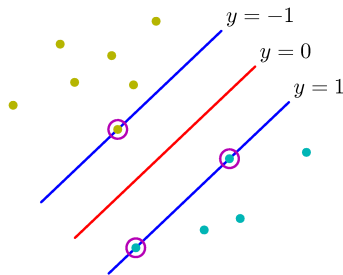
→ Plus M est grand, plus on s'attend à une bonne capacité à généraliser

On cherche l'hyperplan avec la marge maximale M^*

$\mathbf{x}^*$ est un vecteur de support

$$\Leftrightarrow M^* = |y(\mathbf{x}^*)|$$

L'hyperplan optimal dépend directement des vecteurs de support mais pas des autres observations



Crédits figure : PRML C. Bishop

Classifieur à marge maximale

Le classifieur est la solution au problème d'optimisation :

$$\max_{\mathbf{w}, b, M} M = \max_{\mathbf{w}, b} \min_i |y(\mathbf{x}_i)|$$

Sous contraintes :

$$\begin{aligned} \|\mathbf{w}\|^2 &= 1 \\ t_i y(\mathbf{x}_i) &= t_i (\mathbf{w}^T \mathbf{x}_i + b) > M \quad \forall i \end{aligned}$$

→ On cherche un hyperplan t.q. toutes les observations sont bien classifiées, i.e., du bon côté de l'hyperplan avec une distance d'au moins M

→ Il existe des solutions efficaces à ce problème d'optimisation

Soient $\mathbf{w}^{\text{opt}}$ et b^{opt} , les paramètres de l'hyperplan optimal

Pour prédire la classe d'un nouveau point $\mathbf{x}$, on se base sur le signe de $y(\mathbf{x}; \mathbf{w}^{\text{opt}}, b^{\text{opt}})$

Classifieur à vecteurs de support

Dans de nombreux cas, il n'existe pas de d'hyperplan qui sépare parfaitement bien les classes

Extension du classifieur à marge maximale au cas où les données ne sont pas linéairement séparables

→ On cherche un hyperplan qui sépare presque parfaitement les données

→ **Marge souple** : on permet des violations de la marge

→ Même dans le cas de données linéairement séparables, il peut être utile d'utiliser une marge souple pour éviter le sur-apprentissage.

Autrement dit, on accepte un hyperplan qui ne classe pas parfaitement bien toutes les observations :

- Plus grande robustesse vis-à-vis des observations (moins de variance)
- Meilleure classification de la majorité des observations

Classifieur à vecteurs de support

Une observation $(\mathbf{x}_i, t_i)$ peut

→ être bien classifiée :

- du bon côté de la marge et $t_i y(\mathbf{x}_i) > M$
- du mauvais côté de la marge et $0 < t_i y(\mathbf{x})_i < M$

→ être mal classifiée : du mauvais côté de l'hyperplan et $t_i y(\mathbf{x}_i) < 0$

Classifieur à vecteurs de support

Une observation $(\mathbf{x}_i, t_i)$ peut

→ être bien classifiée :

- du bon côté de la marge et $t_i y(\mathbf{x}_i) > M$
- du mauvais côté de la marge et $0 < t_i y(\mathbf{x}_i) < M$

→ être mal classifiée : du mauvais côté de l'hyperplan et $t_i y(\mathbf{x}_i) < 0$

Soit ϵ_i , une variable de mou
("slack variable") t.q.

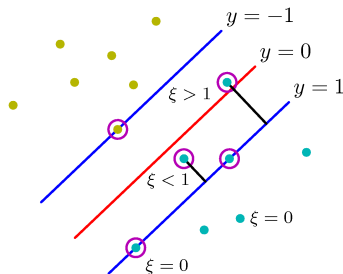
$$t_i y(\mathbf{x}_i) \geq M(1 - \epsilon_i)$$

Alors : $(\mathbf{x}_i, t_i)$ est

→ du bon côté de la marge et $\epsilon_i = 0$

→ du mauvais côté de la marge et
 $0 < \epsilon_i < 1$

→ mal classifiée et $\epsilon_i > 1$



Classifieur à vecteurs de support

Le classifieur est la solution au problème d'optimisation :

$$\max_{\mathbf{w}, b, M, \epsilon} M = \max_{\mathbf{w}, b} \min_i |y(\mathbf{x}_i)|$$

Sous contraintes :

$$\begin{aligned} \|\mathbf{w}\|^2 &= 1 \\ t_i y(\mathbf{x}_i) &= t_i (\mathbf{w}^T \mathbf{x}_i + b) > M(1 - \epsilon_i), \quad \epsilon_i > 0 \quad \forall i \\ \sum_{i=1}^N \epsilon_i &\leq C \end{aligned}$$

Où $C \geq 0$ est un paramètre contrôlant la complexité :

- Plus C est grand, plus des violations de marges sont autorisées
- Si $C = 0$, aucune violation n'est acceptée
- Si $C > 0$, au plus C observations peuvent être du mauvais côté de l'hyperplan ($\epsilon_i > 1$)

→ Choix de C : compromis biais-variance

- C petit : marges plus étroites rarement violées et un classifieur très ajusté aux données (biais ↓ - variance ↑)
- C grand : marges plus grandes plus souvent violées et un classifieur moins ajusté aux données (biais ↑ - variance ↓)

→ Choix de C : pour minimiser l'erreur de généralisation (erreur de test)

Vecteurs de support

- Observations se trouvant sur la marge ou violant la marge
- Seules observations à affecter la position de l'hyperplan et donc du classifieur
- Des changements dans les observations bien classifiées n'influencent pas la surface de décision (forme de robustesse)
- Le nombre de vecteurs de support est relié à la valeur du paramètre (ou hyper-paramètre) C
- Plus la marge est grande, plus il y aura de vecteurs de support

Machine à vecteurs de support

Approche par noyaux “kernel trick” : permet de convertir un classifieur linéaire en un classifieur non-linéaire

→ Reliées aux fonctions de base $\phi(\mathbf{x}) = (\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \dots, \phi_M(\mathbf{x}))^T$

→ Le classifieur prend la forme $y(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b$

Le classifieur est la solution au problème d'optimisation :

$$\max_{\mathbf{w}, b, M, \epsilon} M = \max_{\mathbf{w}, b} \min_i |y(\mathbf{x}_i)|$$

Sous contraintes :

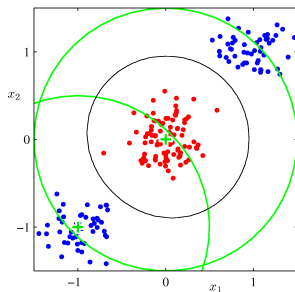
$$\begin{aligned} \|\mathbf{w}\|^2 &= 1 \\ t_i y(\mathbf{x}_i) &= t_i (\mathbf{w}^T \phi(\mathbf{x}_i) + b) > M(1 - \epsilon_i), \quad \epsilon_i > 0 \quad \forall i \\ \sum_{i=1}^N \epsilon_i &\leq C \end{aligned}$$

→ Le classifieur peut avoir des surfaces de décision non-linéaires grâce à ϕ

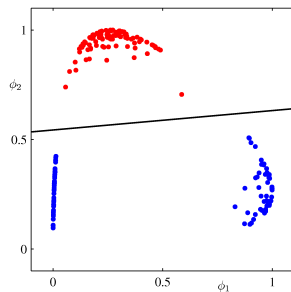
Illustration et limitations

Deux classes (rouge et bleue)

(x_1, x_2) l'espace d'origine



(ϕ_1, ϕ_2) , l'espace transformé



Crédits figure : PRML C. Bishop

- ϕ_j fonctions de base gaussiennes dont les centres et contours sont donnés par croix et ellipses vertes
- **Limitation** : les fonctions de base sont fixées *a priori* (fléau de la dimensionalité)
- **Alternatives** : réseaux à fonctions de base radiales et machines à vecteurs de support (réseaux de neurones artificiels)

Machine à vecteurs de support

Approche par noyaux : mise en oeuvre efficace des fonctions de base

- Un noyau est une mesure de similarité exprimée par un produit scalaire
- Produit scalaire habituel entre deux observations :

$$\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^P x_{ik} x_{jk}$$

Équivalent au coefficient de corrélation de Pearson

Le classifieur à vecteurs de support peut s'écrire :

$$y(\mathbf{x}) = b + \sum_{i=1}^N \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle$$

avec $\alpha_1, \dots, \alpha_N$, N paramètres

- Pour estimer $\alpha_1, \dots, \alpha_N$ et b , il suffit de connaître les $n(n-1)/2$ pairs de produits scalaires

Machine à vecteurs de support

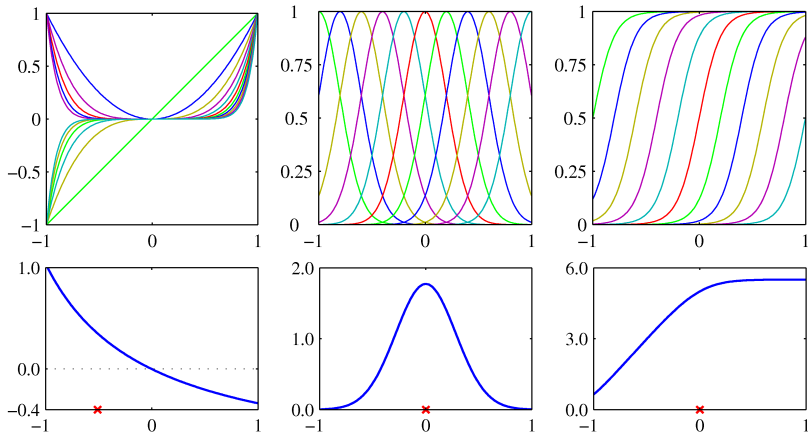
- $\alpha_i \neq 0$ seulement si $\mathbf{x}_i$ est un vecteur de support
- Soit $\mathcal{S}$, l'ensemble des indices des vecteurs de support, alors

$$y(\mathbf{x}) = b + \sum_{i \in \mathcal{S}} \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle$$

- **En résumé** : il suffit de connaître les produits scalaires pour représenter le classifieur linéaire et seulement aux vecteurs de support.

Noyau : généralisation du produit scalaire

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$



Crédits figure : PRML C. Bishop

Noyau : généralisation du produit scalaire

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Noyau linéaire :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \sum_{k=1}^P x_{ik} x_{jk}$$

Noyau polynomial :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \left(1 + \sum_{k=1}^P x_{ik} x_{jk} \right)^P$$

Noyau radial (gaussien) :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp \left(-\gamma \sum_{k=1}^P (x_{ik} - x_{jk})^2 \right), \quad \gamma > 0$$

Machine à vecteurs de support

La machine à vecteurs de support est le classifieur donné par

$$y(\mathbf{x}) = b + \sum_{i \in \mathcal{S}} \alpha_i K(\mathbf{x}, \mathbf{x}_i)$$

Noyau radial (gaussien) :

$$K(\mathbf{x}, \mathbf{x}_j) = \exp \left(-\gamma \sum_{k=1}^P (x_{ik} - x_{jk})^2 \right)$$

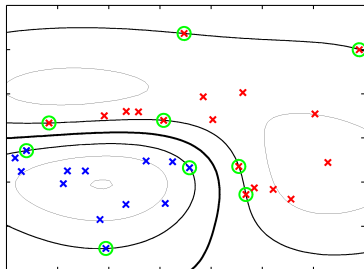
avec $\gamma > 0$

Si $\|\mathbf{x} - \mathbf{x}_j\|$ est grand $\Rightarrow K(\mathbf{x}, \mathbf{x}_j)$ sera petit

→ $\mathbf{x}_j$ ne joue pas un rôle important pour classifier $\mathbf{x}$

→ Caractère local du noyau gaussien

→ **Avantage des noyaux :** ils permettent d'utiliser un grand nombre de fonctions de base (espace de caractéristiques très grand) tout en limitant les calculs



Crédits figure : PRML C. Bishop

MTH8304 - Apprentissage non-supervisé et séries chronologiques

8. Classification non-supervisée: clustering et partitionnement

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

Plan

- 1 Introduction
- 2 K-moyennes
- 3 K-medoïdes
- 4 Clustering hiérarchique

1 Introduction

2 K-moyennes

3 K-medoïdes

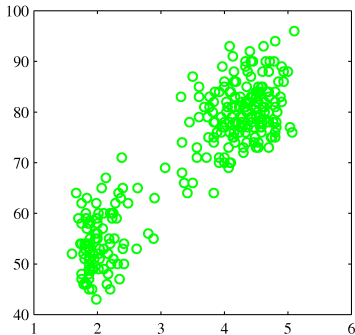
4 Clustering hiérarchique

Classification non-supervisée :

- recherche automatique de sous-groupes
- établissement d'une partition

Exemple : recherche de sous-groupes

Geyser Old faithful, Yellow Stone. 272 observations.



Crédits figure : PRML C. Bishop

Temps entre deux éruptions en min (axe des y) versus la durée d'une éruption en min (axe des x).

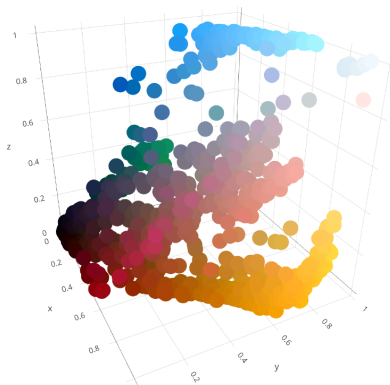
Exemple : établissement d'une partition

Segmentation d'images

Original image



Crédits figure : PRML C. Bishop



Chaque pixel d'une image est une point dans $[0, 1]^3$ décrivant les intensités RGB

- 1 Introduction
- 2 K-moyennes
- 3 K-medoïdes
- 4 Clustering hiérarchique

K-moyennes (K-means)

Problème : identifier des groupes (clusters) dans un espace multidimensionnel

Données $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$: N observations dans $\mathbb{R}^D$

But : établir une partition des données en K clusters ou groupes avec K fixé

En pratique : attribuer une classe à chaque exemple $\mathbf{x}_n$

Stratégie : faire en sorte que les distances intra-groupes soient petites par rapport au distances inter-groupes

K-moyennes (K-means)

Soient $\mu_k \in \mathbb{R}^D$ et $1 \leq k \leq K$ des **prototypes** associés à chaque cluster

But : trouver une attribution des classes et des vecteurs μ_k

Critère : la **somme des distances carrées** de chaque exemple x_n au prototype μ_k de la classe associée est minimale

K-moyennes (K-means)

Soient $\mu_k \in \mathbb{R}^D$ et $1 \leq k \leq K$ des **prototypes** associés à chaque cluster

But : trouver une attribution des classes et des vecteurs μ_k

Critère : la **somme des distances carrées** de chaque exemple x_n au prototype μ_k de la classe associée est minimale

Soit $r_{nk} \in \{0, 1\}$, $1 \leq k \leq K$ une variable indicatrice de classe pour x_n

Soit la fonction de coût :

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2$$

→ J mesure la somme des distances carrées de chaque point à son prototype

→ On cherche des valeurs de r_{nk} et de μ_k qui minimisent J

K-moyennes (K-means)

Pour minimiser J p/r aux r_{nk} et μ_k , on adopte une procédure itérative :

- 1 Choisir des valeurs initiales pour μ_k , $1 \leq k \leq K$
- 2 **E (expectation)** : optimiser J par rapport aux r_{nk}
- 3 **M (maximisation)** : optimiser J par rapport aux μ_k
- 4 Répéter les étapes **E** et **M** jusqu'à convergence.

K-moyennes (K-means)

Pour minimiser J p/r aux r_{nk} et μ_k , on adopte une procédure itérative :

- 1 Choisir des valeurs initiales pour μ_k , $1 \leq k \leq K$
- 2 **E (expectation)** : optimiser J par rapport aux r_{nk}
- 3 **M (maximisation)** : optimiser J par rapport aux μ_k
- 4 Répéter les étapes **E** et **M** jusqu'à convergence.

→ La convergence est assurée lorsqu'il n'y a plus de changement dans les attributions des classes, pourvu que le nombre d'itérations permis soit suffisant.

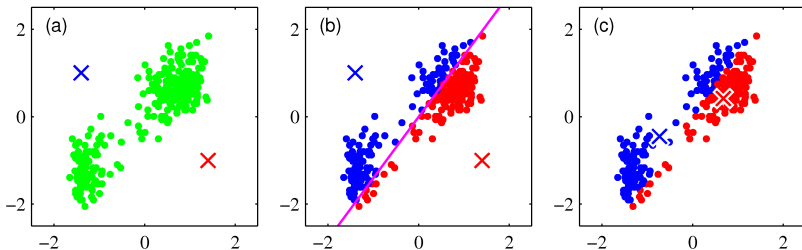
→ Chaque étape **E** et **M** réduit la valeur de J ce qui garantit la convergence.

→ Cependant, l'algorithme peut converger vers un minimum local.

K-moyennes (K-means) : données Old faithful

Données centrées et réduites

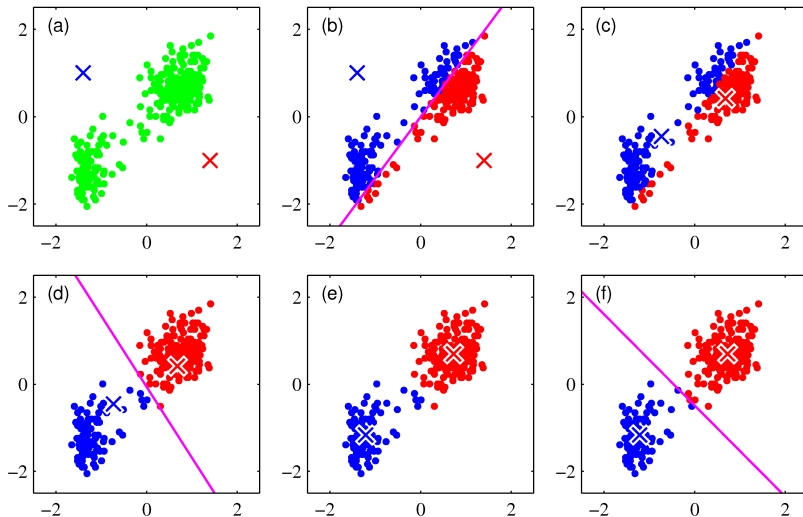
Prototypes initialisés d'habitude en tirant au hasard K exemples



K-moyennes (K-means) : données Old faithful

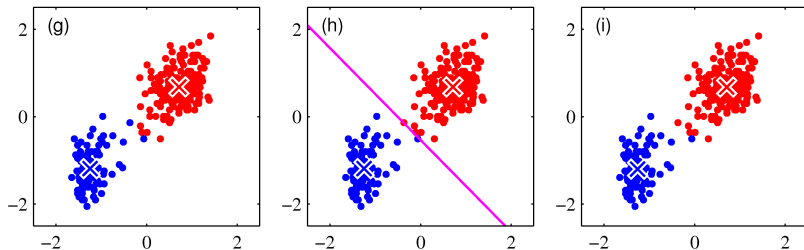
Données centrées et réduites

Prototypes initialisés d'habitude en tirant au hasard K exemples

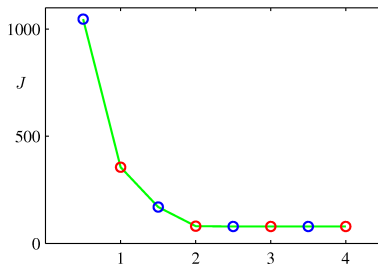
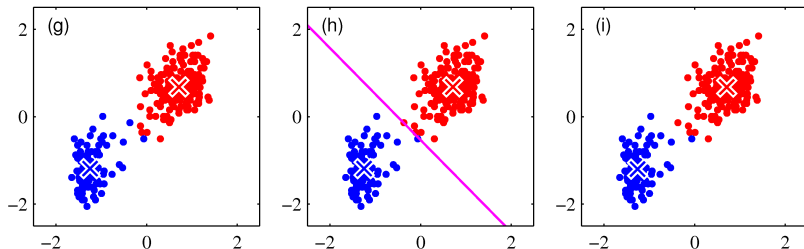


Crédits figure : PRML C. Bishop

K-moyennes (K-means) : données Old faithful



K-moyennes (K-means) : données Old faithful



Crédits figure : PRML C. Bishop

E (expectation)

- J est une fonction linéaire des r_{nk}
- On peut optimiser pour chaque n séparément en choisissant $r_{nk} = 1$ pour la valeur de k qui minimise $\|\mathbf{x}_n - \mu_k\|^2$
- Autrement dit : $\mathbf{x}_n$ est attribué à la classe k telle que μ_k est le centre le plus proche

K-moyennes (K-means)

M (maximisation)

→ J est une fonction quadratique des μ_k

→ Il faut résoudre

$$\frac{\partial J}{\partial \mu_k} == 0$$

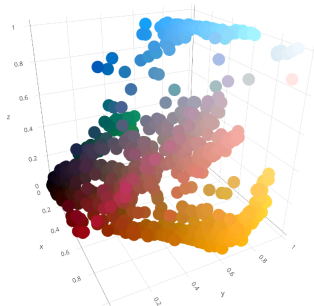
On obtient

$$\mu_k = \frac{\sum_n r_{nk} \mathbf{x}_n}{\sum_n r_{nk}}$$

Le dénominateur est égal au nombre d'exemples dans la classe k

μ_k est donc égal à la moyenne des exemples de la classe k , d'où le nom des K-moyennes.

K-moyennes (K-means) : segmentation d'image



$K = 2$



$K = 3$



$K = 10$



Original image



Crédits figure : PRML C. Bishop

K-moyennes (K-means) : dans R

Librairie stats : `kmeans`

Arguments principaux :

- `x` : matrice de dimension $N \times D$
- `centers` : soit le nombre de clusters K soit un vecteur de prototypes initiaux
- `iter.max = 10` : le nombre d'itérations permis
- `nstart = 1` : si `centers` est un nombre, le nombre de prototypes initiaux choisis aléatoirement
- `trace = FALSE` : si `TRUE`, permet de vérifier la convergence

- 1 Introduction
- 2 K-moyennes
- 3 K-medoïdes**
- 4 Clustering hiérarchique

Distance versus dissimilarité

Distance : $d : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ telle que les axiomes suivants sont satisfaits :

- Identité : $d(x, y) = 0 \iff x = y$
- Symétrie : $d(x, y) = d(y, x)$
- Triangle : $d(x, z) \leq d(x, y) + d(y, z)$

Il s'en suit :

- Non-négativité : $d(x, y) \geq 0$

Distance versus dissimilarité

Distance : $d : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ telle que les axiomes suivants sont satisfaits :

- Identité : $d(x, y) = 0 \iff x = y$
- Symétrie : $d(x, y) = d(y, x)$
- Triangle : $d(x, z) \leq d(x, y) + d(y, z)$

Il s'en suit :

- Non-négativité : $d(x, y) \geq 0$

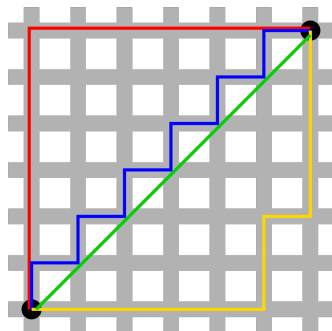
Les K-moyennes sont basées sur la distance euclidéenne $\|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2$

→ Norme L_2 de la différence entre 2 vecteurs

Autres types de distance en changement la norme :

→ Norme L_1 : distance de Manhattan

→ Norme L_∞ : $\|\mathbf{x}_n - \boldsymbol{\mu}_k\|_\infty$



Crédits figure : Wikipedia

Distance versus dissimilarité

Dissimilarité : généralisation de la notion de distance

Similarité : l'inverse de la dissimilarité, e.g., $-||\mathbf{x}_n - \boldsymbol{\mu}_k||$

Noyaux : expriment une notion de similarité, e.g., le noyau gaussien

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{||\mathbf{x} - \mathbf{x}'||}{2\sigma^2}\right)$$

Moins $\mathbf{x}$ et $\mathbf{x}'$ se ressemblent, plus $k(\mathbf{x}, \mathbf{x}')$ devient petit

➔ Une dissimilarité a le comportement inverse : plus $\mathbf{x}$ et $\mathbf{x}'$ sont différents, plus la dissimilarité est grande

K-medoids

Généralisation des K-moyennes basée sur la notion de dissimilarité

Soit $\mathcal{V}(\mathbf{x}, \mathbf{x}')$ une mesure de dissimilarité

Les K-medoids minimisent la fonction de coût suivante :

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(\mathbf{x}_n, \mu_k)$$

K-medoids

Généralisation des K-moyennes basée sur la notion de dissimilarité

Soit $\mathcal{V}(\mathbf{x}, \mathbf{x}')$ une mesure de dissimilarité

Les K-medoids minimisent la fonction de coût suivante :

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(\mathbf{x}_n, \boldsymbol{\mu}_k)$$

Les prototypes $\boldsymbol{\mu}_k$ sont choisis parmi les données

Étape **E** : attribuer les points $\mathbf{x}_n$ à la classe x de sorte que $\mathcal{V}(\mathbf{x}_n, \boldsymbol{\mu}_k)$ soit minimisée

Étape **M** : pour chaque k , on recherche $\mathbf{x}_{n_k} \in C_k$ tel que

$$\sum_{n=1}^N r_{nk} \mathcal{V}(\mathbf{x}_n, \mathbf{x}_{n_k})$$

soit minimisée

Librairie `cluster` : pam “partitioning around medoids”

Arguments principaux :

- `x` : matrice de dimension $N \times D$ ou matrice de dissimilarité de dimension $N \times N$ ou objet de type `dist`
- `k` : le nombre de clusters
- `diss = inherits(x, "dist")` : indique si `x` est de type `dist`
- `metric = c("euclidean", "manhattan")` : la distance (ou métrique) utilisée pour calculer les dissimilarités
- `medoids = NULL` : permet de préciser un vecteur de medoïdes initiaux
- `stand = FALSE` : standardisation des colonnes de `x`

- 1 Introduction
- 2 K-moyennes
- 3 K-medoïdes
- 4 Clustering hiérarchique

Dendrogramme : arbre renversé définissant des partitions emboîtées

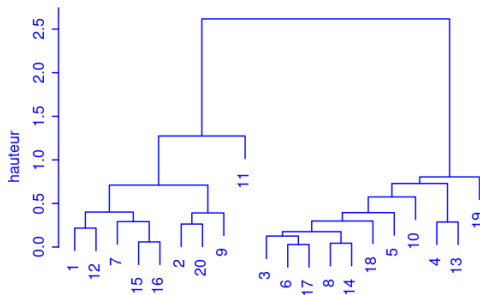
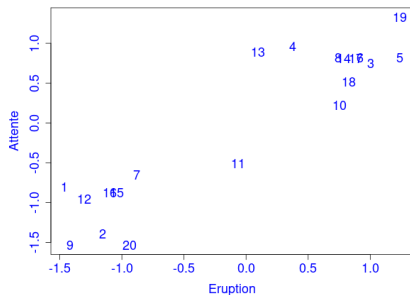
Clustering agglomératif (bottom-up) : commence par les feuilles et combine les clusters en remontant vers la base

- ➔ On va voir d'abord comment interpréter un dendrogramme
- ➔ Puis on verra comment construire un dendrogramme

Dans quel contexte a-t-on déjà parlé d'arbre renversé et de partition ?

Interprétation du dendrogramme

20 observations tirées aléatoirement des données Old Faithful

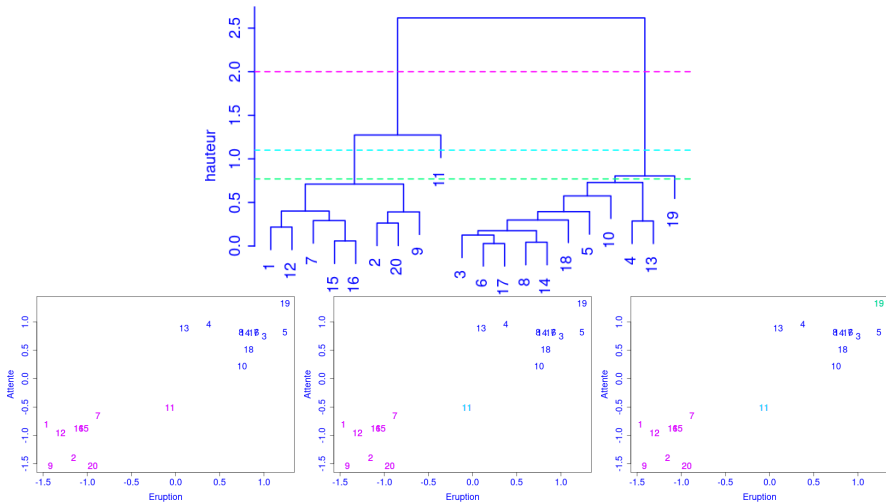


Interprétation du dendrogramme

- Chaque feuille représente une observation
- En montant, les feuilles fusionnent pour former des branches : ce sont des observations semblables
- En montant davantage, les branches fusionnent entre elles ou avec des feuilles
- Le plus bas la fusion se produit, le plus similaire sont les groupes d'observations
- **La hauteur** à laquelle la fusion se produit indique combien semblables (ou dissemblables) les observations sont.
- **Attention** : la proximité de deux observations dans le dendrogramme n'indique pas la similarité
 - Par exemple, les observations 13 et 19.

Coupe du dendrogramme

- Permet de créer des partitions avec un nombre de clusters allant de 1 à N
- Coupe à différentes hauteurs génère différents nombres de clusters

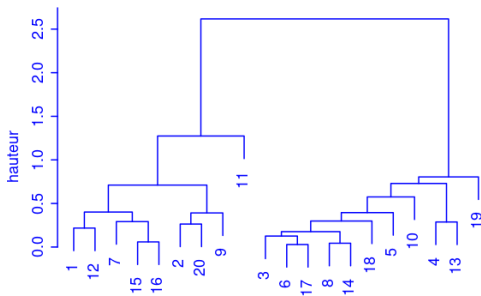
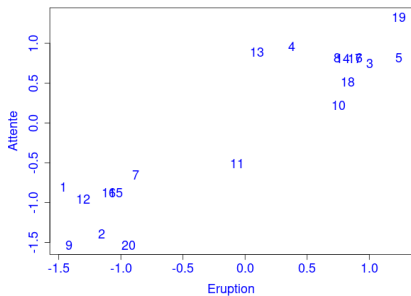


- La hauteur de la coupe joue le même rôle que le nombre de clusters K dans les K-moyennes ;
- À partir d'un seul dendrogramme, on peut extraire des partitions de toutes les tailles possibles ;
- Le niveau où couper se fait souvent à l'oeil mais en général, la même difficulté que pour les K-moyennes se présente ;
- Les partitions sont emboîtées : pas toujours une hypothèse appropriée.

Algorithme du clustering agglomératif

- 1 Soit une mesure de dissimilarité, e.g., la distance euclidéenne ;
Calculer toutes les $n(n-1)/2$ paires de dissimilarités.
- 2 Chaque observation est considérée comme un groupe (cluster) ;
- 3 Identifier la paire de clusters qui est la moins dissemblable (le plus semblable) et fusionner ;
- 4 Mettre à jour les calculs de dissimilarités inter-clusters ;
- 5 Itérer les étapes 3 et 4 jusqu'à ce qu'il ne reste qu'un seul cluster.

Algorithme du clustering agglomératif



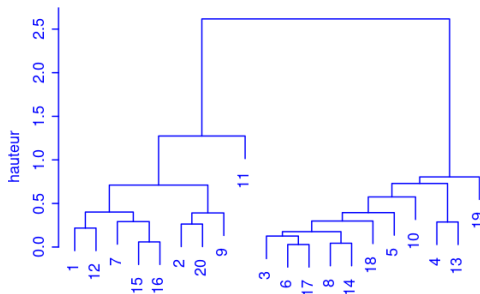
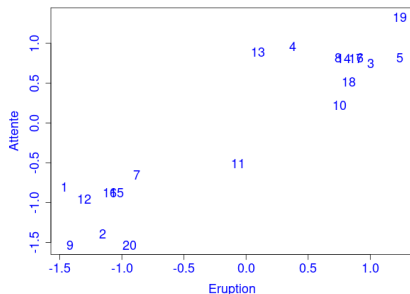
→ 1^{ère} fusion : 6 et 17

→ 2^{ème} fusion : 8 et 14

→ 3^{ème} fusion : 15 et 16

→ 4^{ème} fusion : 3 et {6, 17}

Algorithme du clustering agglomératif



➔ 1^{ère} fusion : 6 et 17

➔ 2^{ème} fusion : 8 et 14

➔ 3^{ème} fusion : 15 et 16

➔ 4^{ème} fusion : 3 et {6, 17}

➔ La hauteur est donnée par la mesure de dissimilarité

➔ Comment calculer la dissimilarité entre deux clusters ?

Notion de “linkage”

Linkage (liaison) : dissimilarité entre deux clusters

Principales liaisons :

- **Liaison complète** : dissimilarité inter-cluster maximale
→ Calcul de toutes les dissimilarités entre observations des deux clusters et la plus **grande** est retenue
 - **Liaison simple** : dissimilarité inter-cluster minimale
→ Calcul de toutes les dissimilarités entre observations des deux clusters et la plus **petite** est retenue
 - **Liaison moyenne** : dissimilarité inter-cluster moyenne
→ Calcul de toutes les dissimilarités entre observations des deux clusters et la dissimilarité **moyenne** est retenue
- Les différents types de liaisons vont générer des différences dans les dendrogrammes.

Clustering agglomératif dans R

Librairie `cluster` : agnes “agglomerative nesting”

Arguments principaux :

- `x` : matrice de dimension $N \times D$ ou matrice de dissimilarité de dimension $N \times N$ ou objet de type `dist`
- `diss = inherits(x, "dist")` : indique si `x` est de type `dist`
- `metric = "euclidean"` : la distance (ou métrique) utilisée pour calculer les dissimilarités
- `stand = FALSE` : standardisation des colonnes de `x`
- `method = "average"` : type de liaison utilisée

Pour couper le dendrogramme, voir la fonction `cutree` du package `stats`.

MTH8304 - Apprentissage non-supervisé et séries chronologiques

9. Estimation de densité non-paramétrique et clustering

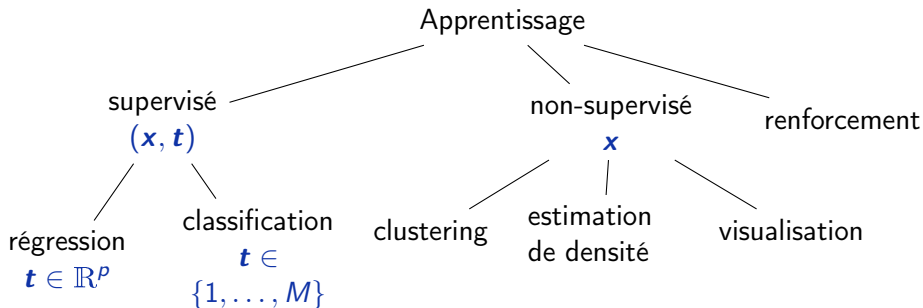
Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

- 1 Estimation de densité non-paramétrique
- 2 Mélange de distributions
- 3 Estimation de densité et clustering
- 4 Entraînement et choix de modèles

- 1 Estimation de densité non-paramétrique
- 2 Mélange de distributions
- 3 Estimation de densité et clustering
- 4 Entraînement et choix de modèles

Estimation de densité : apprentissage non-supervisé



Estimation de densité paramétrique

Données : $x_1, x_2, \dots, x_N, x_i \in \mathbb{R} \forall i$

→ On cherche à estimer la distribution des données.

→ On s'appuie sur **une loi paramétrique** (e.g., normale, exponentielle, ...) de densité $f(x; \theta)$ et on estime les paramètres θ .

Estimation par minimisation de la log-vraisemblance négative :

$$l(\theta; x_1, x_2, \dots, x_N) = - \sum_{n=1}^N \ln f(x_n; \theta)$$

→ **Limitation** : la loi paramétrique choisie peut ne pas représenter très bien la distribution des données, par exemple en cas de multi-modalité.

Estimation de densité non-paramétrique : histogramme

Données (1D) : $x_1, x_2, \dots, x_N, x_i \in \mathbb{R} \forall i$

→ Partition des données en intervalles disjoints de longueur Δ_i , souvent, on choisit $\Delta_i = \Delta \forall i$:

$[\min(x_i), \min(x_i) + \Delta), [\min(x_i) + \Delta, \min(x_i) + 2\Delta), \dots$

→ Soit n_i , le nombre de données appartenant au $i^{\text{ème}}$ intervalle, on en dérive une densité de cette façon :

$$p_i = \frac{n_i}{\Delta N}$$

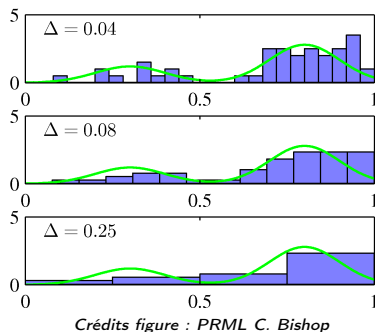
La modèle de densité non-paramétrique est donné par :

$$\hat{p}(x) = p_i \leftrightarrow x \in i^{\text{ème}} \text{ intervalle}$$

→ On peut facilement voir que $\int \hat{p}(x) dx = 1$

→ L'histogramme fournit un modèle où la densité est constante par intervalle.

Estimation de densité non-paramétrique : histogramme



Modèle générateur : densité donnée par la courbe verte (mélange de 2 gaussiennes)

Largeur de l'intervalle Δ agit comme un paramètre de lissage (complexité) :

- $\Delta \uparrow$: biais (pas assez de structure - aspect multimodal)
- $\Delta \downarrow$: variance (structure provenant des données)

Estimation de densité non-paramétrique : histogramme

Propriétés :

- Plus besoin des données une fois que l'histogramme est calculé pour évaluer $\hat{p}(x)$
- Pratique pour une visualisation rapide en 1D ou 2D
- Discontinuités artificielles introduites aux limites des intervalles
- Difficilement applicable en haute dimension car si on utilise M intervalles par dimension, il faudra M^D hypercubes en dimension D

Deux leçons :

- Estimation de densité : considérer des **voisinages locaux** et donc une forme de **distance**
- La **taille du voisinage** est un **paramètre de lissage** contrôlant la complexité de l'estimateur

Estimation de densité non-paramétrique : voisinage

Données (D-dimension) : $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$, $\mathbf{x}_i \in \mathbb{R}^D \forall i$ tirées d'une densité inconnue $p : \mathbb{R}^D \rightarrow [0, 1]$

Soit $\mathcal{R} \subset \mathbb{R}^D$, une petite région telle que $\mathbf{x} \in \mathcal{R}$ et soit

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}$$

→ Le nombre de points K tombant dans $\mathcal{R}$ suit une loi binomiale de paramètre P et donc la statistique K/N est t.q. :

$$E \left[\frac{K}{N} \right] = P \quad \text{et} \quad \text{Var} \left[\frac{K}{N} \right] = \frac{P(1-P)}{N}$$

Alors pour N grand : $K \simeq NP$

→ Supposons que $\mathcal{R}$ soit assez petite pour que $p(\mathbf{x})$ soit constante sur cette région, alors :

$$P \simeq p(\mathbf{x})V \quad \text{où} \quad V = \int_{\mathcal{R}} d\mathbf{x}$$

Estimation de densité non-paramétrique : voisinage

En combinant $K \simeq NP$ et $P \simeq p(\mathbf{x})V$, on obtient :

$$\hat{p}(\mathbf{x}) = \frac{K}{NV}$$

→ Deux hypothèses conflictuelles :

- $\mathcal{R}$ doit être suffisamment assez petite pour que la densité soit constante sur cette région
- en même suffisamment grande pour que la distribution de K ait une faible variance

→ Deux façons d'exploiter ce résultat :

- 1 On fixe V et on détermine K à parti des données : approche à noyaux
- 2 On fixe K et on détermine la valeur de V à partir des données : approche de type K plus proches voisins.

Dans les deux cas : estimateurs convergeant vers la densité du modèle générateur lorsque $N \rightarrow \infty$ et des conditions sur K et V

Estimation de densité non-paramétrique : KDE

Kernel Density Estimator (KDE) - estimateurs de densité à noyaux

Estimateur de "Parzen window" : $\mathcal{R}$ est un hypercube

Soit la fonction à noyaux :

$$k(\mathbf{u}) = \begin{cases} 1, & \text{si } |u_i| \leq 1/2, i = 1, \dots, D \\ 0, & \text{sinon} \end{cases}$$

Alors

$$k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right) = 1 \Leftrightarrow \mathbf{x}_n \in \text{hypercube centré en } \mathbf{x} \text{ de côté } h$$

Le nombre total de point dans cet hypercube est :

$$K = \sum_{n=1}^N k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right)$$

On combine avec le résultat précédent :

$$\hat{p}(\mathbf{x}) = \frac{K}{NV} = \frac{1}{Nh^D} \sum_{n=1}^N k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right) \text{ où } V = h^D$$

Estimation de densité non-paramétrique : KDE

$$\hat{p}(\mathbf{x}) = \frac{1}{Nh^D} \sum_{n=1}^N k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right)$$

Par symétrie de $k(\mathbf{u})$, l'estimateur $\hat{p}(\mathbf{x})$ peut être interprété comme la somme pondérée de N hypercubes centrés sur chaque $\mathbf{x}_n$

→ **Même problème que l'histogramme** : discontinuités artificielles aux frontières des hypercubes

→ **Solution** : choisir une fonction à noyaux plus lisse comme le noyau gaussien

$$\hat{p}(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{\sqrt{2\pi}h} \exp\left\{-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2h^2}\right\}$$

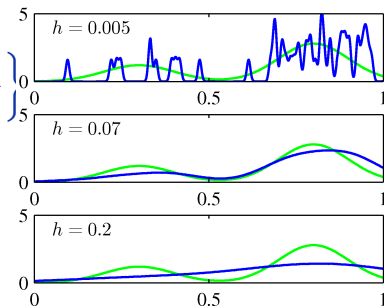
→ Contribution de chaque point $\mathbf{x}$ donnée par le noyau gaussien

Estimation de densité non-paramétrique : KDE

$$\hat{p}(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{\sqrt{2\pi}h} \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2h^2} \right\}$$

h : écart-type du noyau gaussien est un paramètre de lissage

→ Sélection de h :
complexité du modèle



Crédits figure : PRML C. Bishop

Autres fonctions à noyaux possibles :

$$\begin{aligned} k(\mathbf{u}) &\geq 0 \\ \int k(\mathbf{u}) d\mathbf{u} &= 1 \end{aligned}$$

- Pas d'entraînement : garder en mémoire les données
- Temps de calcul élevé pour de grands jeux de données

Estimation de densité non-paramétrique : KPP

Dans KDE, h , la taille du voisinage, est la même partout

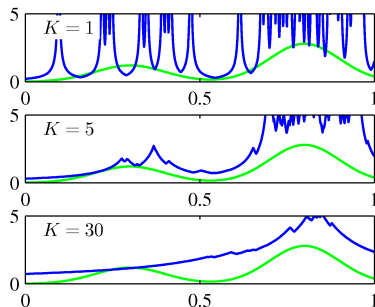
Sous-optimal : régions avec beaucoup de données (h petit) et des régions avec peu de données (h grand)

K plus proches voisins (KPPV) : la taille du voisinage varie dans l'espace

$$\hat{p}(\mathbf{x}) = \frac{K}{NV}$$

K est fixé, V est le volume de la sphère contenant les K points

→ Sélection de K :
complexité du modèle



Crédits figure : PRML C. Bishop

- 1 Estimation de densité non-paramétrique
- 2 Mélange de distributions**
- 3 Estimation de densité et clustering
- 4 Entraînement et choix de modèles

Mélange de distributions

Les modèles de mélange de distributions consistent à prendre une combinaison **linéaire** et **convexe** de distributions. La densité d'un mélange discret peut s'écrire :

$$\hat{p}(x) = \sum_{k=1}^K \pi_k f(x; \theta_k)$$

où $0 \leq \pi_k \leq 1$, $\sum_{k=1}^K \pi_k = 1$ et $f(x; \theta_k)$ est la densité d'une loi de probabilité de paramètre θ_k .

➔ Le plus souvent, on choisit la loi normale/gaussienne.

On a déjà vu les mélanges dans deux cadres :

- L'estimation de densité
 - Le mélange de gaussiennes permet un compromis entre une approche “memory-based”, KDE avec noyau gaussien, et une approche paramétrique, loi gaussienne

On a déjà vu les mélanges dans deux cadres :

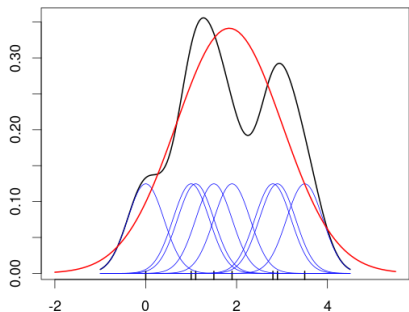
- L'estimation de densité
 - Le mélange de gaussiennes permet un compromis entre une approche “memory-based”, KDE avec noyau gaussien, et une approche paramétrique, loi gaussienne
- La classification supervisée : modèles générateurs probabilistes (LDA/QDA)
 - Le mélange de gaussiennes permet aussi de faire de la classification non-supervisée, une forme de généralisation des K-moyennes

- 1 Estimation de densité non-paramétrique
- 2 Mélange de distributions
- 3 Estimation de densité et clustering**
- 4 Entraînement et choix de modèles

Estimateur à noyaux versus paramétrique

Données :

$$\{x_1, x_2, \dots, x_N\}$$



Estimateur à noyaux gaussiens (noir) :

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{\sqrt{2\pi}h} \exp \left\{ -\frac{(x - x_n)^2}{2h^2} \right\}$$

Loi gaussienne (rouge) :

$$\hat{p}(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{(x - \mu)^2}{2\sigma^2} \right\}$$

Mélange de gaussiennes

La densité du mélange de gaussiennes est donnée par :

$$\hat{p}(x) = \sum_{k=1}^K \pi_k \underbrace{\frac{1}{\sqrt{2\pi}\sigma_k} \exp \left\{ -\frac{(x - \mu_k)^2}{2\sigma_k^2} \right\}}_{f(x; \mu_k, \sigma_k)}$$

- Composante k : la densité gaussienne $f(x; \mu_k, \sigma_k)$ centrée en μ_k et d'écart-type σ_k
- Coefficients/poids du mélanges : π_k indique l'importance de chaque gaussienne dans le mélange

Mélange de gaussiennes

La densité du mélange de gaussiennes est donnée par :

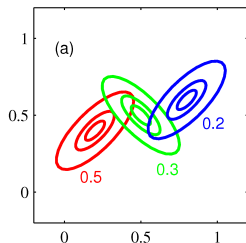
$$\hat{p}(x) = \sum_{k=1}^K \pi_k \underbrace{\frac{1}{\sqrt{2\pi}\sigma_k} \exp \left\{ -\frac{(x - \mu_k)^2}{2\sigma_k^2} \right\}}_{f(x; \mu_k, \sigma_k)}$$

- Composante k : la densité gaussienne $f(x; \mu_k, \sigma_k)$ centrée en μ_k et d'écart-type σ_k
- Coefficients/poids du mélanges : π_k indique l'importance de chaque gaussienne dans le mélange
- Si $K = 1$: une seule loi gaussienne (modèle paramétrique)
- Si $K = N$, $\mu_k = x_n$ et $\sigma_k = h$: estimateur à noyaux

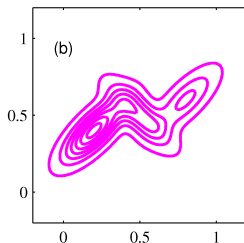
Mélange de gaussiennes : illustration

Mélange de trois gaussiennes en 2D : densité

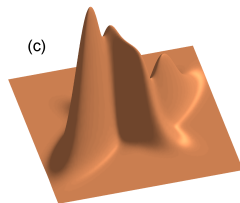
Chaque composante et
poids associé



Loi marginale $\hat{p}(\mathbf{x})$



Loi marginale $\hat{p}(\mathbf{x})$



Crédits figure : PRML C. Bishop

Mélange de gaussiennes : estimation

Données : $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, $\mathbf{x}_n \in \mathbb{R}^D$

Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

avec $\boldsymbol{\pi} = (\pi_1, \dots, \pi_K)$, $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K)$ et $\boldsymbol{\Sigma} = (\boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_K)$

Mélange de gaussiennes : estimation

Données : $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, $\mathbf{x}_n \in \mathbb{R}^D$

Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

avec $\boldsymbol{\pi} = (\pi_1, \dots, \pi_K)$, $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K)$ et $\boldsymbol{\Sigma} = (\boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_K)$

➔ La présence de la somme à l'intérieur du logarithme rend l'optimisation beaucoup plus complexe que dans le cas d'une seule gaussienne (pas de solutions analytiques)

Mélange de gaussiennes : estimation

Données : $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, $\mathbf{x}_n \in \mathbb{R}^D$

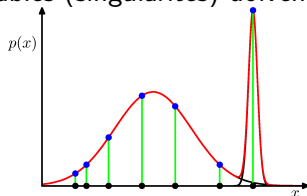
Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

avec $\boldsymbol{\pi} = (\pi_1, \dots, \pi_K)$, $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K)$ et $\boldsymbol{\Sigma} = (\boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_K)$

→ La présence de la somme à l'intérieur du logarithme rend l'optimisation beaucoup plus complexe que dans le cas d'une seule gaussienne (pas de solutions analytiques)

→ Des solutions indésirables (singularités) doivent être évitées :



Crédits figure : PRML C. Bishop

Formulation en termes de variable latente

Soit $\mathbf{z} \in \{0, 1\}^K$ un encodage 1 sur K : $z_k \in \{0, 1\}$ et $\sum_{k=1}^K z_k = 1$

Formulation en termes de variable latente

Soit $\mathbf{z} \in \{0, 1\}^K$ un encodage 1 sur K : $z_k \in \{0, 1\}$ et $\sum_{k=1}^K z_k = 1$

$\mathbf{z}$ est une variable latente discrète prenant K états possibles :

$$P(z_k = 1) = \pi_k \quad 1 \leq k \leq K$$

→ Semblables aux probabilités a priori $P(C_k)$ mais ici les classes ne sont pas observées

Formulation en termes de variable latente

Soit $\mathbf{z} \in \{0, 1\}^K$ un encodage 1 sur K : $z_k \in \{0, 1\}$ et $\sum_{k=1}^K z_k = 1$

$\mathbf{z}$ est une variable latente discrète prenant K états possibles :

$$P(z_k = 1) = \pi_k \quad 1 \leq k \leq K$$

→ Semblables aux probabilités a priori $P(C_k)$ mais ici les classes ne sont pas observées

Soit la densité conditionnelle de $\mathbf{x}$ étant donné la valeur de la variable latente :

$$p(\mathbf{x} | z_k = 1) = f(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

→ Semblables aux densités conditionnelles de classe $p(\mathbf{x} | C_k)$

Formulation en termes de variable latente

Soit $\mathbf{z} \in \{0, 1\}^K$ un encodage 1 sur K : $z_k \in \{0, 1\}$ et $\sum_{k=1}^K z_k = 1$

$\mathbf{z}$ est une variable latente discrète prenant K états possibles :

$$P(z_k = 1) = \pi_k \quad 1 \leq k \leq K$$

→ Semblables aux probabilités a priori $P(C_k)$ mais ici les classes ne sont pas observées

Soit la densité conditionnelle de $\mathbf{x}$ étant donné la valeur de la variable latente :

$$p(\mathbf{x} | z_k = 1) = f(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

→ Semblables aux densités conditionnelles de classe $p(\mathbf{x} | C_k)$

Alors le mélange de gaussiennes peut s'écrire :

$$\hat{p}(\mathbf{x}) = \sum_{k=1}^K P(z_k = 1) p(\mathbf{x} | z_k = 1) = \sum_{k=1}^K \pi_k f(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

La distribution conditionnelle de $\mathbf{z}$ sachant $\mathbf{x}$ est donnée par

$$\gamma(z_k) = P(z_k = 1|\mathbf{x}) = \frac{P(z_k = 1)p(\mathbf{x}|z_k = 1)}{\sum_{j=1}^K P(z_j = 1)p(\mathbf{x}|z_j = 1)}$$

Responsabilités et clustering

La distribution conditionnelle de $\mathbf{z}$ sachant $\mathbf{x}$ est donnée par

$$\gamma(z_k) = P(z_k = 1|\mathbf{x}) = \frac{P(z_k = 1)p(\mathbf{x}|z_k = 1)}{\sum_{j=1}^K P(z_j = 1)p(\mathbf{x}|z_j = 1)}$$

→ Les $\gamma(z_k)$ correspondent aux probabilités a posteriori $P(C_k|\mathbf{x})$ ou à la **responsabilité** de la composante k pour $\mathbf{x}$

→ Pour faire du **clustering**, on attribue la classe/groupe/cluster k à $\mathbf{x}$ si

$$\gamma(z_k) \geq \gamma(z_j) \quad \forall j \neq k$$

→ **Clustering probabiliste/soft**

Responsabilités et clustering

La distribution conditionnelle de $\mathbf{z}$ sachant $\mathbf{x}$ est donnée par

$$\gamma(z_k) = P(z_k = 1|\mathbf{x}) = \frac{P(z_k = 1)p(\mathbf{x}|z_k = 1)}{\sum_{j=1}^K P(z_j = 1)p(\mathbf{x}|z_j = 1)}$$

→ Les $\gamma(z_k)$ correspondent aux probabilités a posteriori $P(C_k|\mathbf{x})$ ou à la **responsabilité** de la composante k pour $\mathbf{x}$

→ Pour faire du **clustering**, on attribue la classe/groupe/cluster k à $\mathbf{x}$ si

$$\gamma(z_k) \geq \gamma(z_j) \quad \forall j \neq k$$

→ **Clustering probabiliste/soft**

→ La formulation en termes de variable latente permet de simplifier la maximisation de la log-vraisemblance

→ L'algorithme **EM** permet de maximiser la log-vraisemblance en faisant intervenir les responsabilités

Mélange de gaussiennes : échantillonnage

Le choix de la composante suit une loi multinomiale avec probabilités

$\pi_1, \dots, \pi_K$

→ On tire aléatoirement une valeur k en utilisant dans R la fonction `sample(K, probs = pik)`

Mélange de gaussiennes : échantillonnage

Le choix de la composante suit une loi multinomiale avec probabilités

$\pi_1, \dots, \pi_K$

→ On tire aléatoirement une valeur k en utilisant dans R la fonction `sample(K, probs = pik)`

On tire aléatoirement une valeur x de la loi gaussienne associée à la composante k de paramètres μ_k et Σ_k

→ En utilisant (par exemple), la fonction `mvrnorm` de la librairie MASS

Mélange de gaussiennes : échantillonnage

Le choix de la composante suit une loi multinomiale avec probabilités

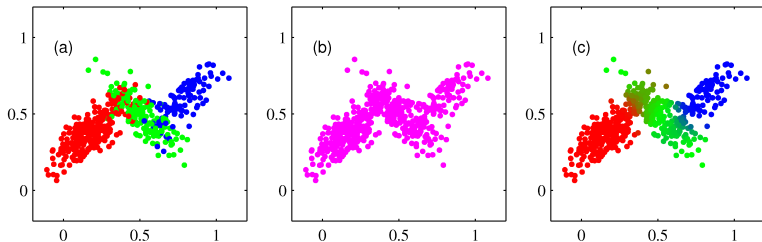
$\pi_1, \dots, \pi_K$

→ On tire aléatoirement une valeur k en utilisant dans R la fonction `sample(K, probs = pik)`

On tire aléatoirement une valeur x de la loi gaussienne associée à la composante k de paramètres μ_k et Σ_k

→ En utilisant (par exemple), la fonction `mvrnorm` de la librairie MASS

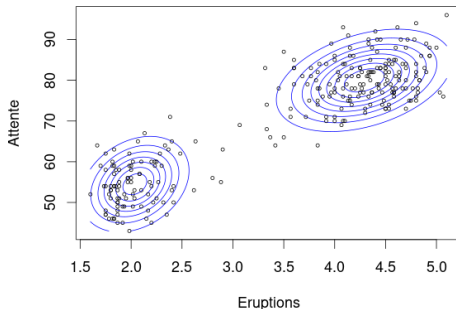
Échantillon avec ou sans le choix de la composante et responsabilités



Crédits figure : PRML C. Bishop

Exemple : données Old faithful

Mélange de 2 gaussiennes avec matrices de variance-covariance flexibles



→ Contours de densité constante pour chacune des gaussiennes

Exemple : données Old faithful

```
> library(mclust)
> data(faithful)
> dens2 <- densityMclust(faithful, G = 2, modelNames = "VVV")
> summary(dens2, parameters = TRUE)
-----
Density estimation via Gaussian finite mixture modeling
-----

Mclust VVV (ellipsoidal, varying volume, shape, and orientation)
model with 2 components:

  log-likelihood    n df          BIC          ICL
    -1130.264 272 11 -2322.192 -2322.697

Mixing probabilities:
      1      2
0.6440718 0.3559282
```

Exemple : données Old faithful

Means:

	[,1]	[,2]
eruptions	4.289781	2.036523
waiting	79.969549	54.479886

Variances:

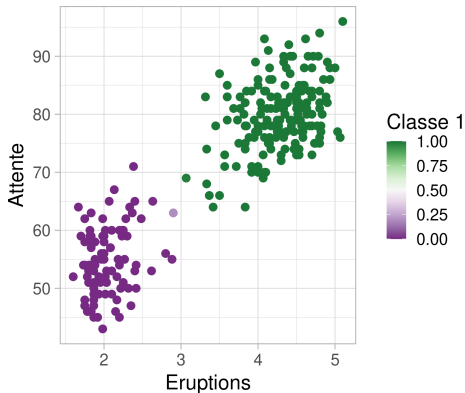
	[,1]	
		eruptions
eruptions	0.1698176	0.9386975
waiting	0.9386975	36.0247964

	[,2]	
		eruptions
eruptions	0.06927521	0.4363001
waiting	0.43630011	33.7051532

```
> plot(dens2, what = "density", data = faithful, grid = 200,  
points.cex = 0.5, drawlabels = FALSE, col = "blue",  
xlab = "Eruptions", ylab = "Attente")
```

Exemple : données Old faithful

Responsabilité de la classe latente 1 : $P(z_1 = 1|\mathbf{x})$



Les points sont attribués à la classe 1 lorsque $P(z_1 = 1|\mathbf{x}) > 0.5$

Exemple : données Old faithful

```
df.mclust <- data.frame(faithful, class = dens2$z)

library(ggplot2)
theme_set(theme_light(18))
library(RColorBrewer)

ggplot(df.mclust, aes(x = eruptions, y = waiting, col = class.1))+
  geom_point(size=3)+
  scale_colour_gradientn(colors=
  brewer.pal(n =11, name = "PRGn")[2:10], lim = c(0,1),
  name = "Classe 1") + xlab("Eruptions") + ylab("Attente")
```

Mclust : types de modèles

$$\Sigma_k = \lambda_k D_k A_k D_k^T$$

λ_k : *volume*

A_k : *forme*

matrice diagonale avec $|A_k| = 1$

D_k : *orientation*

matrice orthogonale

Model	Σ_k	Distribution	Volume	Shape	Orientation
EII	λI	Spherical	Equal	Equal	—
VII	$\lambda_k I$	Spherical	Variable	Equal	—
EEI	λA	Diagonal	Equal	Equal	Coordinate axes
VEI	$\lambda_k A$	Diagonal	Variable	Equal	Coordinate axes
EVI	λA_k	Diagonal	Equal	Variable	Coordinate axes
VI	$\lambda_k A_k$	Diagonal	Variable	Variable	Coordinate axes
EEE	$\lambda D A D^T$	Ellipsoidal	Equal	Equal	Equal
EVE	$\lambda D A_k D^T$	Ellipsoidal	Equal	Variable	Equal
VEE	$\lambda_k D A D^T$	Ellipsoidal	Variable	Equal	Equal
WE	$\lambda_k D A_k D^T$	Ellipsoidal	Variable	Variable	Equal
EEV	$\lambda D_k A D_k^T$	Ellipsoidal	Equal	Equal	Variable
VEV	$\lambda_k D_k A D_k^T$	Ellipsoidal	Variable	Equal	Variable
EVV	$\lambda D_k A_k D_k^T$	Ellipsoidal	Equal	Variable	Variable
VVV	$\lambda_k D_k A_k D_k^T$	Ellipsoidal	Variable	Variable	Variable

Table 3: Parameterisations of the within-group covariance matrix Σ_k for multidimensional data available in the **mclust** package, and the corresponding geometric characteristics.

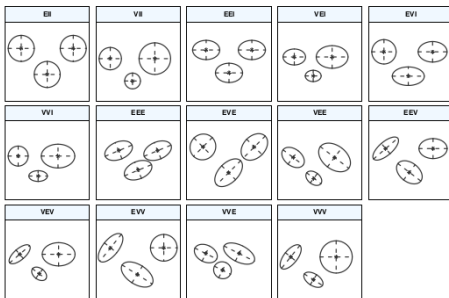


Figure 2: Ellipses of isodensity for each of the 14 Gaussian models obtained by eigen-decomposition in case of three groups in two dimensions.

- 1 Estimation de densité non-paramétrique
- 2 Mélange de distributions
- 3 Estimation de densité et clustering
- 4 Entraînement et choix de modèles**

Entraînement : trouver les paramètres π, μ, Σ qui minimisent la log-vraisemblance

→ Algorithme Expectation-Maximisation

Entraînement : trouver les paramètres π, μ, Σ qui minimisent la log-vraisemblance

→ Algorithme Expectation-Maximisation

Choix de modèles : établir le nombre de composantes dans le mélange et la forme des matrices de variance-covariance

→ Le critère d'information bayésien ("BIC") et interprétation du clustering

Mélange de gaussiennes : algorithme EM

Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

Mélange de gaussiennes : algorithme EM

Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

→ En calculant le gradient par rapport à $\boldsymbol{\pi}$, $\boldsymbol{\mu}$ et $\boldsymbol{\Sigma}$ et en cherchant des expressions pour ces paramètres qui annulent le gradient, on obtient :

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \quad \text{avec} \quad N_k = \underbrace{\sum_{n=1}^N \gamma(z_{nk})}_{\# \text{ effectif de points dans } C_k}$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T \quad \pi_k = \frac{N_k}{N}$$

Mélange de gaussiennes : algorithme EM

Fonction de coût pour l'estimation de densité : log-vraisemblance

$$\mathcal{L}(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}; \mathbf{x}_1, \dots, \mathbf{x}_N) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k f(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

→ En calculant le gradient par rapport à $\boldsymbol{\pi}$, $\boldsymbol{\mu}$ et $\boldsymbol{\Sigma}$ et en cherchant des expressions pour ces paramètres qui annulent le gradient, on obtient :

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \quad \text{avec} \quad N_k = \underbrace{\sum_{n=1}^N \gamma(z_{nk})}_{\# \text{ effectif de points dans } C_k}$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T \quad \pi_k = \frac{N_k}{N}$$

→ Si on remplace $\gamma(z_{nk})$ par t_{nk} , l'étiquette de classe dans le cadre supervisé, on retombe sur les estimateurs du modèle générateur probabiliste.

Mélange de gaussiennes : algorithme EM

Les paramètres π , μ et Σ apparaissent dans les expressions de $\gamma(z_{nk})$

Mélange de gaussiennes : algorithme EM

Les paramètres π , μ et Σ apparaissent dans les expressions de $\gamma(z_{nk})$

→ Approche itérative pour maximiser la log-vraisemblance :

- Initialisation : $\pi^{(0)}$, $\mu^{(0)}$ et $\Sigma^{(0)}$

Mélange de gaussiennes : algorithme EM

Les paramètres π , μ et Σ apparaissent dans les expressions de $\gamma(z_{nk})$

→ Approche itérative pour maximiser la log-vraisemblance :

- Initialisation : $\pi^{(0)}$, $\mu^{(0)}$ et $\Sigma^{(0)}$
- **E (expectation)** : calcul des responsabilités

$$\gamma^{(0)}(z_k) = \frac{\pi_k^{(0)} f(\mathbf{x}, \mu_k^{(0)}, \Sigma_k^{(0)})}{\sum_{j=1}^K \pi_j^{(0)} f(\mathbf{x}, \mu_j^{(0)}, \Sigma_j^{(0)})}$$

Mélange de gaussiennes : algorithme EM

Les paramètres π , μ et Σ apparaissent dans les expressions de $\gamma(z_{nk})$

→ Approche itérative pour maximiser la log-vraisemblance :

- Initialisation : $\pi^{(0)}$, $\mu^{(0)}$ et $\Sigma^{(0)}$
- **E (expectation)** : calcul des responsabilités

$$\gamma^{(0)}(z_k) = \frac{\pi_k^{(0)} f(\mathbf{x}, \mu_k^{(0)}, \Sigma_k^{(0)})}{\sum_{j=1}^K \pi_j^{(0)} f(\mathbf{x}, \mu_j^{(0)}, \Sigma_j^{(0)})}$$

- **M (maximisation)** :

$$\mu_k^{(m)} = \frac{1}{N_k} \sum_{n=1}^N \gamma^{(0)}(z_{nk}) \mathbf{x}_n \quad \text{avec} \quad N_k^{(m)} = \sum_{n=1}^N \gamma^{(0)}(z_{nk})$$

$$\Sigma_k^{(m)} = \frac{1}{N_k^{(m)}} \sum_{n=1}^N \gamma^{(0)}(z_{nk}) (\mathbf{x}_n - \mu_k^{(m)}) (\mathbf{x}_n - \mu_k^{(m)})^T \quad \pi_k^{(m)} = \frac{N_k^{(m)}}{N}$$

Mélange de gaussiennes : algorithme EM

Les paramètres π , μ et Σ apparaissent dans les expressions de $\gamma(z_{nk})$

→ Approche itérative pour maximiser la log-vraisemblance :

- Initialisation : $\pi^{(0)}$, $\mu^{(0)}$ et $\Sigma^{(0)}$
- **E (expectation)** : calcul des responsabilités

$$\gamma^{(0)}(z_k) = \frac{\pi_k^{(0)} f(\mathbf{x}, \mu_k^{(0)}, \Sigma_k^{(0)})}{\sum_{j=1}^K \pi_j^{(0)} f(\mathbf{x}, \mu_j^{(0)}, \Sigma_j^{(0)})}$$

- **M (maximisation)** :

$$\mu_k^{(m)} = \frac{1}{N_k} \sum_{n=1}^N \gamma^{(0)}(z_{nk}) \mathbf{x}_n \quad \text{avec} \quad N_k^{(m)} = \sum_{n=1}^N \gamma^{(0)}(z_{nk})$$

$$\Sigma_k^{(m)} = \frac{1}{N_k^{(m)}} \sum_{n=1}^N \gamma^{(0)}(z_{nk}) (\mathbf{x}_n - \mu_k^{(m)}) (\mathbf{x}_n - \mu_k^{(m)})^T \quad \pi_k^{(m)} = \frac{N_k^{(m)}}{N}$$

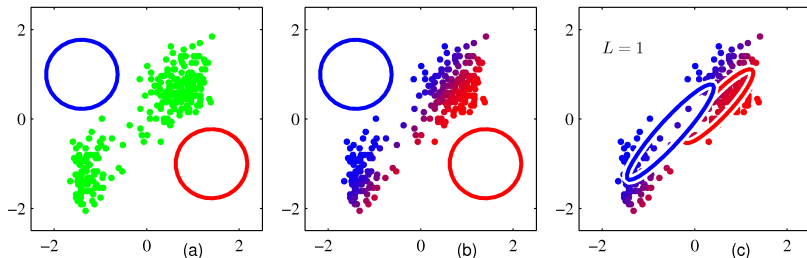
- Si l'algorithme n'a pas convergé :

$$\pi^{(0)} \leftarrow \pi^{(m)} \quad \mu^{(0)} \leftarrow \mu^{(m)} \quad \Sigma^{(0)} \leftarrow \Sigma^{(m)}$$

et retour aux étapes **E** et **M**.

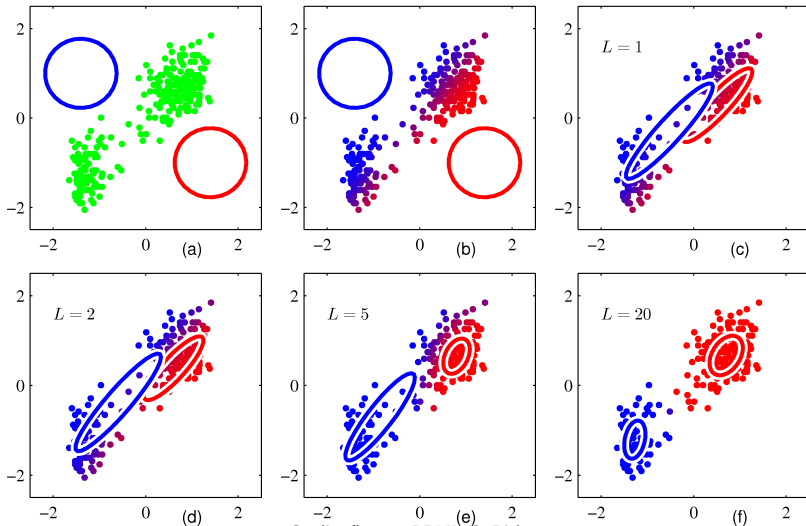
Mélanges de gaussiennes : données Old faithful

Données centrées et réduites (pas indispensable dans ce cas)



Mélanges de gaussiennes : données Old faithful

Données centrées et réduites (pas indispensable dans ce cas)



Beaucoup d'itérations de EM peuvent être nécessaire pour ajuster le mélange de gaussiennes

Il est utile de bien initialiser les paramètres :

- Faire tourner les K-moyennes sur les données
- Pour chaque groupe k , initialiser μ_k et Σ_k par les moyennes échantilonaes et les matrices de variance-covariance échantillonaale
- Les coefficients du mélange π_k , $1 \leq k \leq K$, sont initialisés par les proportions d'exemples attribués à chaque groupe.

Choix de modèles

Dans `mclust`, `G` contrôle le nombre de composantes et `modelName` le type de matrice de variance-covariance $\mathcal{M}$:

```
> dens2 <- densityMclust(faithful, G = 2, modelName = "VVV")
```

Choix de modèles

Dans `mclust`, `G` contrôle le nombre de composantes et `modelName` le type de matrice de variance-covariance $\mathcal{M}$:

```
> dens2 <- densityMclust(faithful, G = 2, modelName = "VVV")
```

Le choix de modèles se fait à l'aide du critère d'information bayésien (BIC) :

$$BIC(\mathcal{M}, G) = 2 \underbrace{\mathcal{L}_{\mathcal{M}, G}(\Psi | \{\mathbf{x}_n\}_{n=1}^N)}_{\text{log-vraisemblance}} - \underbrace{\nu \log(N)}_{\text{pénalité}}$$

Ψ : l'ensemble des paramètres du mélange avec G composantes et matrice de variance-covariance $\mathcal{M}$

ν : le nombre de paramètres du mélange (# d'éléments dans Ψ)

Choix de modèles

Dans `mclust`, `G` contrôle le nombre de composantes et `modelName` le type de matrice de variance-covariance $\mathcal{M}$:

```
> dens2 <- densityMclust(faithful, G = 2, modelName = "VVV")
```

Le choix de modèles se fait à l'aide du critère d'information bayésien (BIC) :

$$BIC(\mathcal{M}, G) = 2 \underbrace{\mathcal{L}_{\mathcal{M}, G}(\Psi | \{\mathbf{x}_n\}_{n=1}^N)}_{\text{log-vraisemblance}} - \underbrace{\nu \log(N)}_{\text{pénalité}}$$

Ψ : l'ensemble des paramètres du mélange avec G composantes et matrice de variance-covariance $\mathcal{M}$

ν : le nombre de paramètres du mélange (# d'éléments dans Ψ)

Log-vraisemblance : mesure d'adéquation du modèle que l'on cherche à maximiser (inverse du biais)

Pénalité : mesure de la complexité du modèle

➔ **BIC** : identifier un modèle faisant un compromis entre une bonne adéquation (faible biais) et une trop grande complexité

Choix de modèles : données Old faithful

```
> dens <- densityMclust(faithful)
> summary(dens)
-----
Density estimation via Gaussian finite mixture modeling
-----

Mclust EEE (ellipsoidal, equal volume, shape and orientation)
model with 3 components:

log-likelihood    n df          BIC          ICL
      -1126.326 272 11 -2314.316 -2357.824
```

Le nombre de degrés de liberté df :

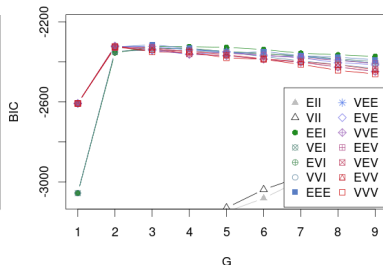
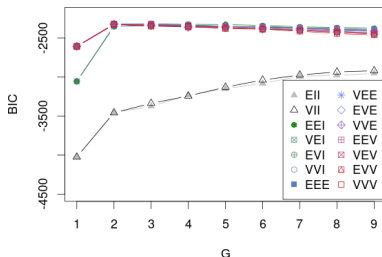
3 valeurs pour définir $\mathcal{M}$, 2×3 valeurs pour les centres de clusters, 2 valeurs pour les coefficients du mélange = 11 paramètres au total

Choix de modèles : données Old faithful

```
plot(dens, what = "BIC", xlab = "G", ylab = "BIC",  
     ylim = c(-4500, -2200))
```

Top 3 models based on the BIC criterion:

EEE,3	VVE,2	VEE,3
-2314.316	-2320.433	-2322.103

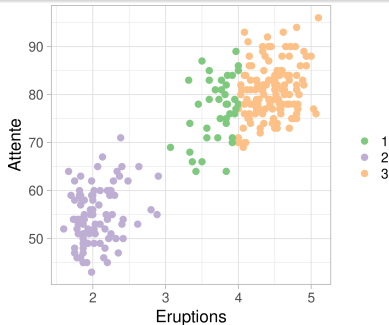


$$BIC(\mathcal{M}, G) = 2 \underbrace{\mathcal{L}_{\mathcal{M}, G}(\Psi | \{\mathbf{x}_n\}_{n=1}^N)}_{\text{log-vraisemblance}} - \underbrace{\nu \log(N)}_{\text{pénalité}}$$

Choix de modèles : données Old faithful

```
dens$param$mean
```

	[,1]	[,2]	[,3]
eruptions	3.793066	2.037596	4.463245
waiting	77.521051	54.491158	80.833439



MTH8304 - Apprentissage non-supervisé et séries chronologiques

11. Boîte à outils et lissage exponentiel

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

- 1 Séquence de modélisation

1 Séquence de modélisation

Séquence de modélisation

① Préparation des données

② Modélisation

① Visualisation

② Spécification du modèle

③ Estimation

④ Évaluation

boucle

③ Prévion

Modèles simples (parfois efficaces) : performance de référence

Données historiques : $x_1, \dots, x_T$

Prévision d'horizon h : $\hat{x}_{T+h|T}$

- Méthode de la moyenne :

$$\hat{x}_{T+h|T} = \bar{x}$$

- Méthode naïve simple :

$$\hat{x}_{T+h|T} = x_T$$

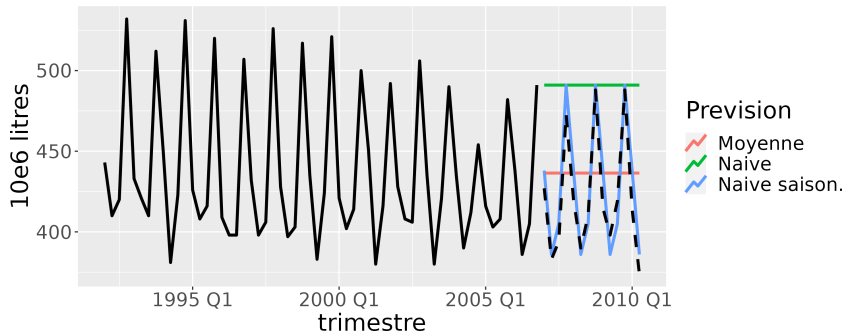
- Méthode naïve saisonnière :

$$\hat{x}_{T+h|T} = x_{T+h-m(k+1)}$$

m : période de la saison

$$k = \lfloor (h-1)/m \rfloor$$

Modèles de références : exemple



Valeurs ajustées et résidus

Valeurs ajustées :

$$\hat{x}_{t|t-1} = \hat{x}_t$$

prévision de x_t étant donné $x_1, \dots, x_{t-1}$

→ souvent pas une vraie prévision car le modèle est ajusté sur toutes les observations - par exemple avec la méthode de la moyenne

Valeurs ajustées et résidus

Valeurs ajustées :

$$\hat{x}_{t|t-1} = \hat{x}_t$$

prévision de x_t étant donné $x_1, \dots, x_{t-1}$

→ souvent pas une vraie prévision car le modèle est ajusté sur toutes les observations - par exemple avec la méthode de la moyenne

Résidus :

$$e_t = x_t - \hat{x}_t$$

Si une transformation a été utilisée : $w_t = \ln(x_t)$

Analyses sur les résidus transformés : $w_t - \hat{w}_t$ *innovation residuals*

→ y a-t-il des motifs dans les résidus ?

on utilise les résidus transformés $w_t - \hat{w}_t$

→ quelle méthode donne les prévisions les plus précises ?

on utilise e_t

Diagnostiques basés sur les résidus transformés

On cherche les propriétés suivantes pour w_t :

- absence de corrélation - la corrélation indique la présence d'information utile pour la prévision ;
- moyenne de zéro - une moyenne différente de zéro indique un biais.

→ Une méthode de prévision qui ne donne pas des résidus avec ces propriétés peut être amélioré e

Diagnostiques basés sur les résidus transformés

On cherche les propriétés suivantes pour w_t :

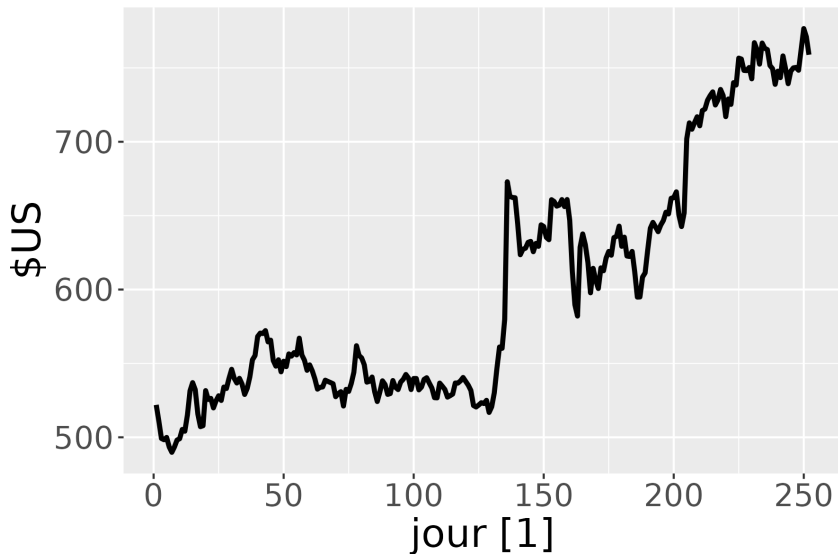
- absence de corrélation - la corrélation indique la présence d'information utile pour la prévision ;
- moyenne de zéro - une moyenne différente de zéro indique un biais.

→ Une méthode de prévision qui ne donne pas des résidus avec ces propriétés peut être améliorée

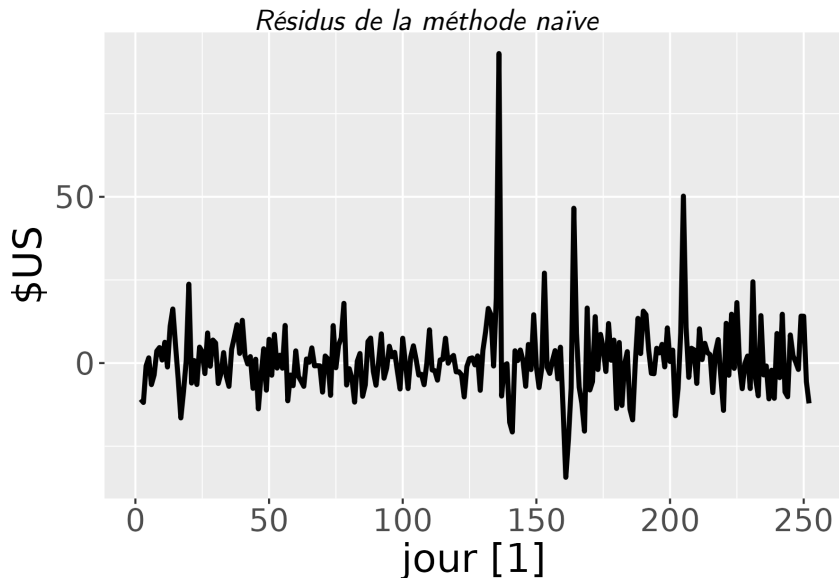
Autres propriétés pour w_t utiles pour le calcul des intervalles de prévision :

- Variance constante - homoscedasticité ;
- Distribués selon la loi normale.

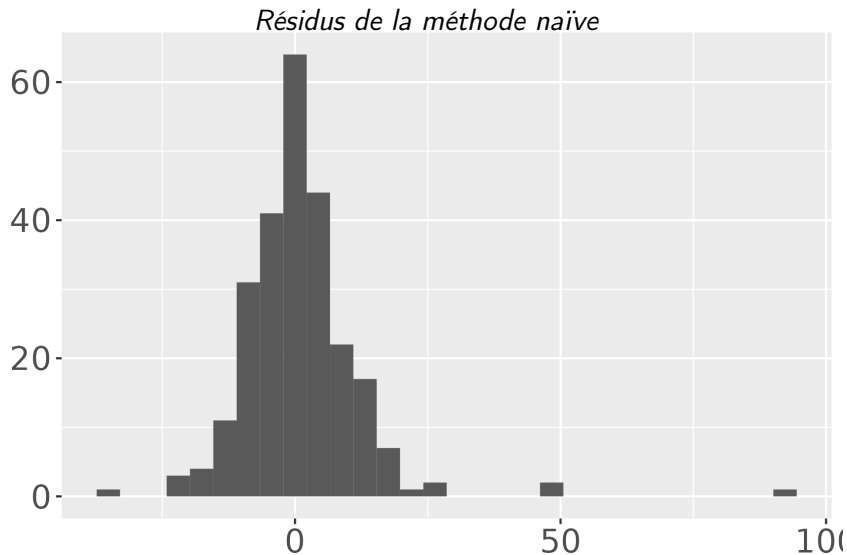
Exemple : prix de fermeture de Google en 2015



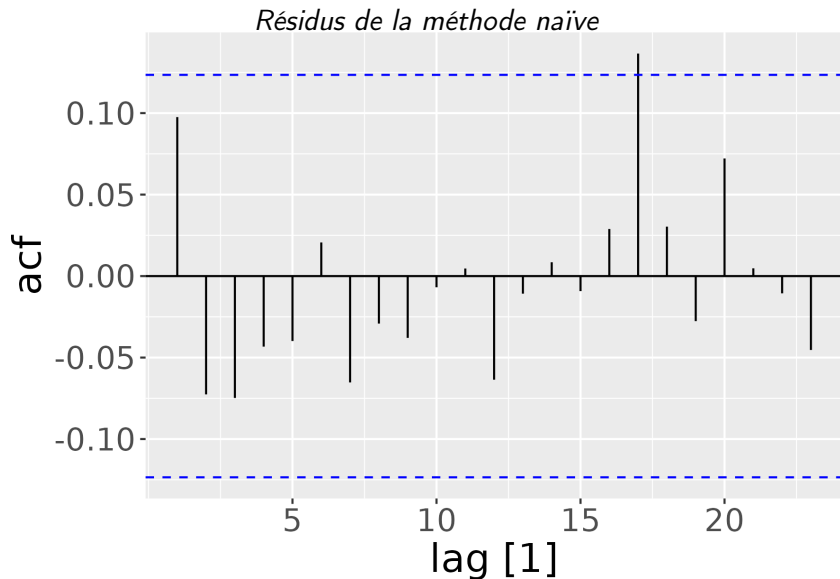
Exemple : prix de fermeture de Google en 2015



Exemple : prix de fermeture de Google en 2015



Exemple : prix de fermeture de Google en 2015



Tests de Portmanteau pour l'auto-corrélation

→ On regroupe les valeurs r_k , $1 \leq k \leq K$, d'auto-corrélation empirique

Test de Ljung-Box (d'autres test existent) :

$$Q^* = T(T+2) \sum_{k=1}^K (T-k)^{-1} r_k^2$$

Interprétation du test :

Si la série est un bruit blanc (non corrélée), $Q^* \sim \chi^2$ avec K degrés de liberté.

Prix de fermeture de Google en 2015 avec la méthode naïve :

$$Q^* = 7.91 \quad \text{valeur - p} = 0.637$$

→ Caractériser l'incertitude de $\hat{x}_{T+h|T} : P(X_{T+h}|X_1, \dots, X_T)$

→ Caractériser l'incertitude de $\hat{x}_{T+h|T} : P(X_{T+h}|X_1, \dots, X_T)$

Si la distribution est normale, un intervalle de confiance à 80% ou 95% est donné par :

$$\hat{x}_{T+h|T} \pm 1.28 \hat{\sigma}_h \quad \hat{x}_{T+h|T} \pm 1.96 \hat{\sigma}_h$$

où $\hat{\sigma}_h$ est un estimateur de l'écart-type de la distribution de prévision d'horizon h

→ Caractériser l'incertitude de $\hat{x}_{T+h|T} : P(X_{T+h}|X_1, \dots, X_T)$

Si la distribution est normale, un intervalle de confiance à 80% ou 95% est donné par :

$$\hat{x}_{T+h|T} \pm 1.28 \hat{\sigma}_h \quad \hat{x}_{T+h|T} \pm 1.96 \hat{\sigma}_h$$

où $\hat{\sigma}_h$ est un estimateur de l'écart-type de la distribution de prévision d'horizon h

→ Permet de déterminer la précision de la prévision

Intervalle de prévision d'horizon h

Pour $h = 1$:

→ Estimation de l'écart-type à partir des résidus e_t :

$$\hat{\sigma} = \sqrt{\frac{1}{T - K - M} \sum_{t=1}^T e_t^2}$$

où K est le nombre de paramètres de la méthode de prévision
et M est le nombre de valeurs manquantes ($M = 1$ pour la méthode naïve)

Intervalle de prévision d'horizon h

Pour $h = 1$:

→ Estimation de l'écart-type à partir des résidus e_t :

$$\hat{\sigma} = \sqrt{\frac{1}{T - K - M} \sum_{t=1}^T e_t^2}$$

où K est le nombre de paramètres de la méthode de prévision
et M est le nombre de valeurs manquantes ($M = 1$ pour la méthode naïve)

Pour $h > 1$:

Méthode de la moyenne : $\hat{\sigma}_h = \hat{\sigma} \sqrt{1 + 1/T}$

Méthode naïve : $\hat{\sigma}_h = \hat{\sigma} \sqrt{h}$

Méthode naïve saisonnière : $\hat{\sigma}_h = \hat{\sigma} \sqrt{k + 1}$ avec $k = \lfloor (h-1)/m \rfloor$

Intervalle de prévision d'horizon h

Pour $h = 1$:

→ Estimation de l'écart-type à partir des résidus e_t :

$$\hat{\sigma} = \sqrt{\frac{1}{T - K - M} \sum_{t=1}^T e_t^2}$$

où K est le nombre de paramètres de la méthode de prévision
et M est le nombre de valeurs manquantes ($M = 1$ pour la méthode naïve)

Pour $h > 1$:

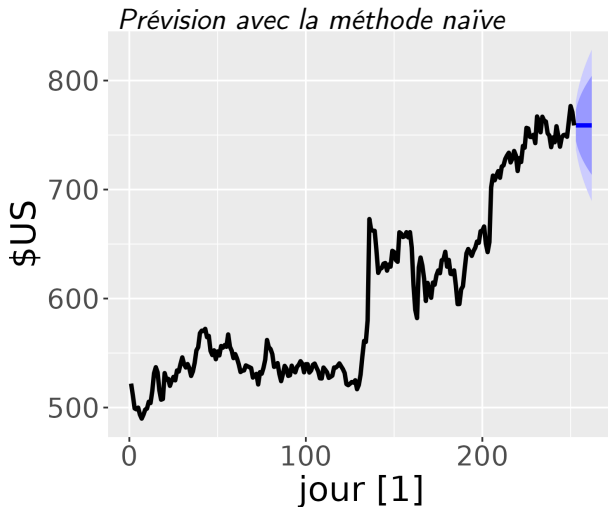
Méthode de la moyenne : $\hat{\sigma}_h = \hat{\sigma} \sqrt{1 + 1/T}$

Méthode naïve : $\hat{\sigma}_h = \hat{\sigma} \sqrt{h}$

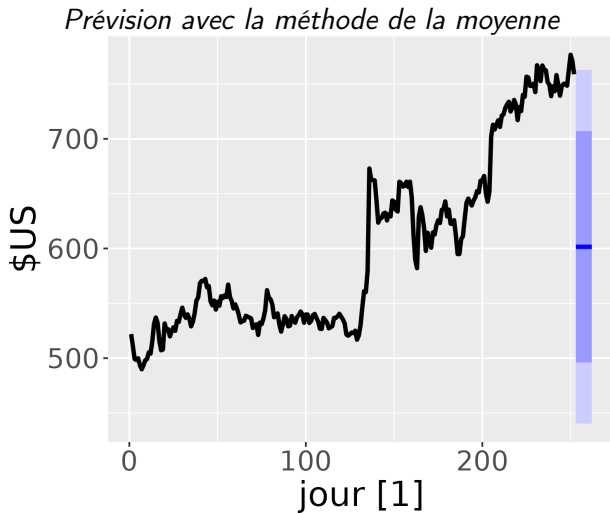
Méthode naïve saisonnière : $\hat{\sigma}_h = \hat{\sigma} \sqrt{k + 1}$ avec $k = \lfloor (h-1)/m \rfloor$

→ On s'attend à ce que l'intervalle de prévision augmente avec h
mais ce n'est pas toujours le cas.

Exemple : prix de fermeture de Google en 2015



Exemple : prix de fermeture de Google en 2015



- Lorsqu'on ne peut pas supposer la normalité de la distribution

→ Lorsqu'on ne peut pas supposer la normalité de la distribution

Méthode naïve

Soit $e_t = x_t - \hat{x}_{t|t-1}$ l'erreur de prévision avec $h = 1$

Puisque $\hat{x}_{t|t-1} = x_{t-1}$:

$$x_t = x_{t-1} + e_t$$

Intervalle de prévision bootstrap

➔ Lorsqu'on ne peut pas supposer la normalité de la distribution

Méthode naïve

Soit $e_t = x_t - \hat{x}_{t|t-1}$ l'erreur de prévision avec $h = 1$

Puisque $\hat{x}_{t|t-1} = x_{t-1}$:

$$x_t = x_{t-1} + e_t$$

Soit $t = T + 1$, on tire e_{T+1}^* parmi $e_1, \dots, e_T$ avec remise

On peut donc simuler x_{T+1} avec

$$x_{T+1}^* = x_T + e_{T+1}^*$$

Intervalle de prévision bootstrap

→ Lorsqu'on ne peut pas supposer la normalité de la distribution

Méthode naïve

Soit $e_t = x_t - \hat{x}_{t|t-1}$ l'erreur de prévision avec $h = 1$

Puisque $\hat{x}_{t|t-1} = x_{t-1}$:

$$x_t = x_{t-1} + e_t$$

Soit $t = T + 1$, on tire e_{T+1}^* parmi $e_1, \dots, e_T$ avec remise

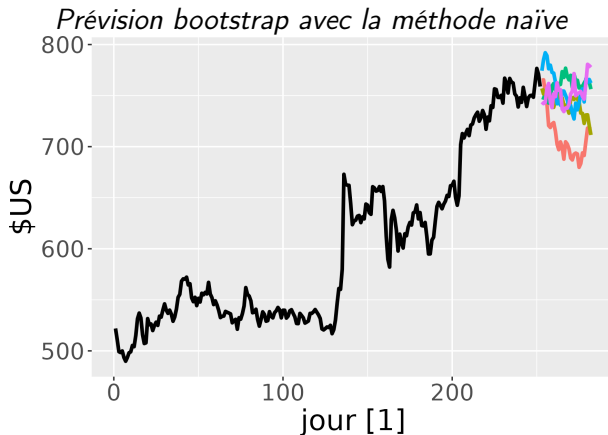
On peut donc simuler x_{T+1} avec

$$x_{T+1}^* = x_T + e_{T+1}^*$$

On répète : on tire e_{T+2}^* parmi $e_1, \dots, e_T$ pour simuler x_{T+2} :

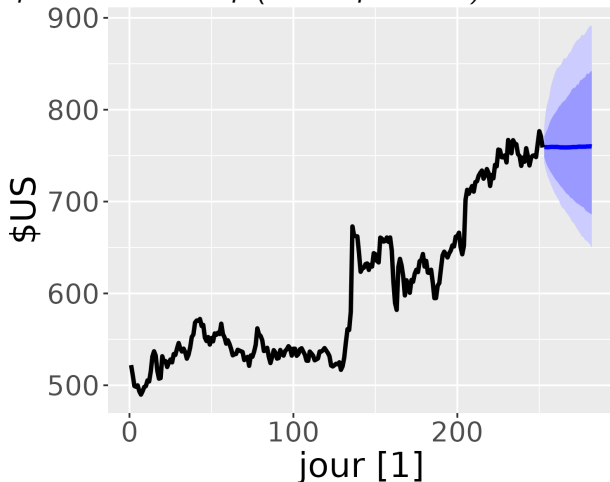
$$x_{T+2}^* = x_{T+1}^* + e_{T+2}^*$$

Exemple : prix de fermeture de Google en 2015



Exemple : prix de fermeture de Google en 2015

Intervalle de prévision bootstrap (5000 réplifications) avec la méthode naïve



- ➔ Intervalles ne sont plus parfaitement symétriques
- ➔ La moyenne des réplifications bootstrap peut être différente de la prévision originale

Prévision avec transformation

Soit $w_t = \ln x_t$, $1 \leq t \leq T$

Pour un t donné, supposons que $\hat{\sigma} = 1$ et $\hat{w}_t = 0$

Alors l'intervalle de prévision 95% est donné par : $[-1.96, 1.96]$

Prévision avec transformation

Soit $w_t = \ln x_t$, $1 \leq t \leq T$

Pour un t donné, supposons que $\hat{\sigma} = 1$ et $\hat{w}_t = 0$

Alors l'intervalle de prévision 95% est donné par : $[-1.96, 1.96]$

En revenant à l'échelle des données : $\hat{x}_t = \exp(\hat{w}_t) = \exp(0) = 1$

Et l'intervalle de prévision devient :

$$[\exp(-1.96), \exp(1.96)] = [0.1409, 7.0993]$$

Prévision avec transformation

Soit $w_t = \ln x_t$, $1 \leq t \leq T$

Pour un t donné, supposons que $\hat{\sigma} = 1$ et $\hat{w}_t = 0$

Alors l'intervalle de prévision 95% est donné par : $[-1.96, 1.96]$

En revenant à l'échelle des données : $\hat{x}_t = \exp(\hat{w}_t) = \exp(0) = 1$

Et l'intervalle de prévision devient :

$$[\exp(-1.96), \exp(1.96)] = [0.1409, 7.0993]$$

→ $\hat{x}_t$ n'est pas la moyenne de la distribution de la prévision

Car pour une loi log-normale de paramètre μ et σ^2 , l'espérance est donné par :

$$\exp\left(\mu + \frac{\sigma^2}{2}\right) = \exp\left(0 + \frac{1}{2}\right) \approx 1.65$$

On a que sa médiane est donnée par $\exp(\mu)$.

Prévision avec transformation

Soit $w_t = \ln x_t$, $1 \leq t \leq T$

Pour un t donné, supposons que $\hat{\sigma} = 1$ et $\hat{w}_t = 0$

Alors l'intervalle de prévision 95% est donné par : $[-1.96, 1.96]$

En revenant à l'échelle des données : $\hat{x}_t = \exp(\hat{w}_t) = \exp(0) = 1$

Et l'intervalle de prévision devient :

$$[\exp(-1.96), \exp(1.96)] = [0.1409, 7.0993]$$

→ $\hat{x}_t$ n'est pas la moyenne de la distribution de la prévision

Car pour une loi log-normale de paramètre μ et σ^2 , l'espérance est donné par :

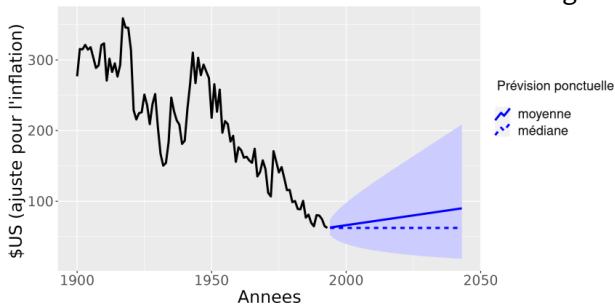
$$\exp\left(\mu + \frac{\sigma^2}{2}\right) = \exp\left(0 + \frac{1}{2}\right) \approx 1.65$$

On a que sa médiane est donnée par $\exp(\mu)$.

→ Possibilité de faire une correction pour le calcul de la moyenne

Exemple : prix annuel des oeufs

Prévision avec la méthode naïve sur transformation logarithmique



Application de la décomposition pour la prévision

Structure additive : $X_t = T_t + S_t + I_t = A_t + S_t$

Structure multiplicative : $X_t = T_t \times S_t \times I_t = A_t \times S_t$

Application de la décomposition pour la prévision

Structure additive : $X_t = T_t + S_t + I_t = A_t + S_t$

Structure multiplicative : $X_t = T_t \times S_t \times I_t = A_t \times S_t$

Composante saisonnière S_t : prévision avec la méthode naïve

→ Hypothèse : pas ou peu de changement dans S_t

Application de la décomposition pour la prévision

Structure additive : $X_t = T_t + S_t + I_t = A_t + S_t$

Structure multiplicative : $X_t = T_t \times S_t \times I_t = A_t \times S_t$

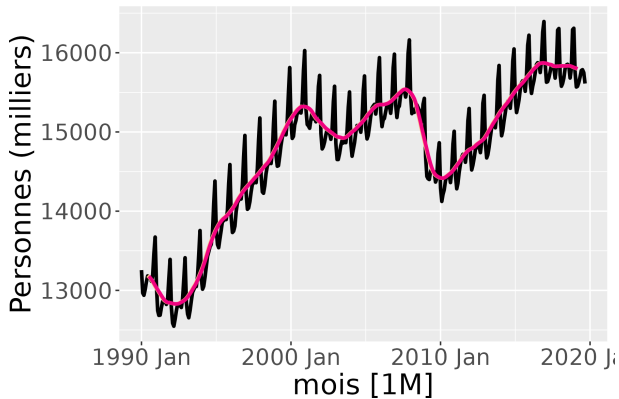
Composante saisonnière S_t : prévision avec la méthode naïve

→ Hypothèse : pas ou peu de changement dans S_t

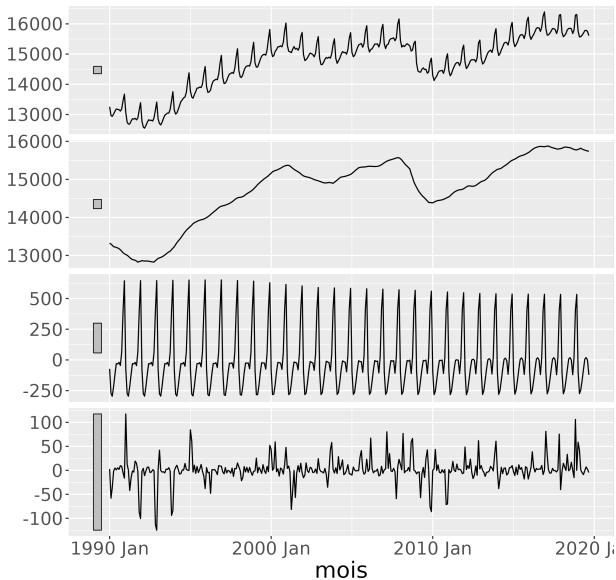
Composante ajustée pour la saison A_t : modèle de prévision non-saisonnier

→ Par exemple : modèle de Holt (lissage exponentiel avec tendance) ou ARIMA non-saisonnier

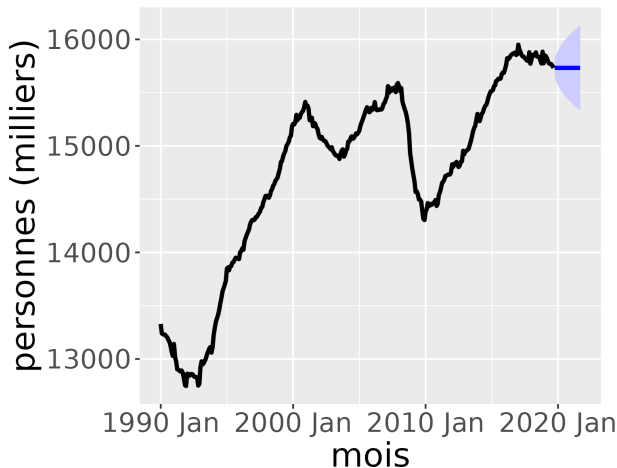
Exemple : emplois vente de détail



Exemple : emplois vente de détail

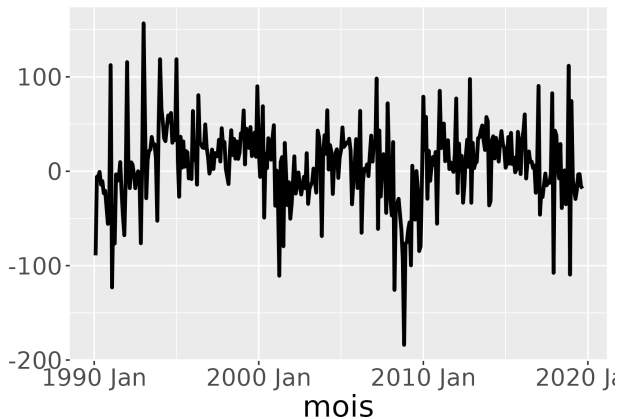


Exemple : emplois vente de détail



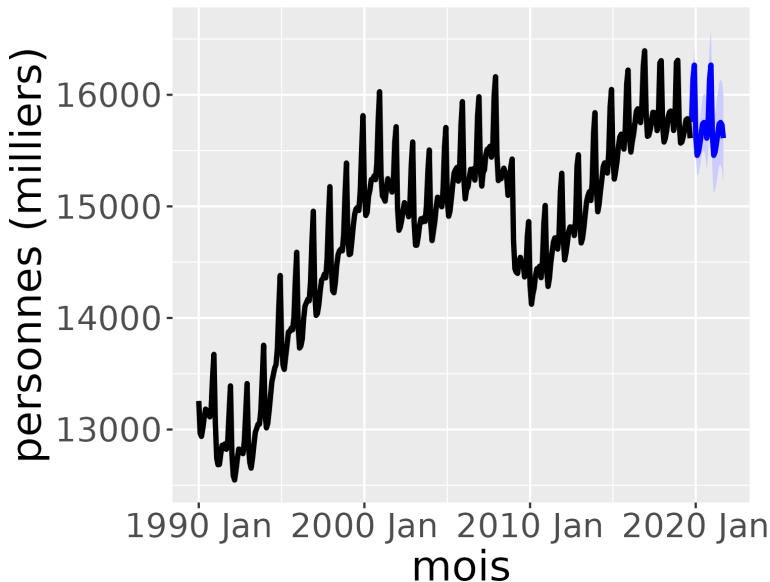
- Prédiction avec la méthode naïve de A_t , composante ajustée pour la saison
- On verra d'autres méthodes possibles de prévision

Exemple : emplois vente de détail



→ Deux propriétés à vérifier : moyenne est zéro et absence de corrélation

Exemple : emplois vente de détail



Ensembles d'entraînement et de test

On ne peut pas choisir un modèle seulement à l'analyse des résidus

Évaluation de la performance sur un ensemble de test : des données qui n'ont pas servi à ajuster le modèle

Règle générale : l'ensemble de test contient 20% des données initiales

Dépend de l'horizon de prévision envisagé

Doit respecter la chronologie des données

Ensembles d'entraînement et de test

On ne peut pas **choisir** un modèle seulement à l'**analyse des résidus**

Évaluation de la performance sur un **ensemble de test** : des données qui n'ont pas servi à ajuster le modèle

Règle générale : l'ensemble de test contient 20% des données initiales

Dépend de l'**horizon de prévision** envisagé

Doit respecter la **chronologie** des données

Sur-apprentissage versus biais :

- un modèle très bien ajusté sur les données d'entraînement ne fournit pas nécessairement de bonnes prévisions
- on peut toujours obtenir un ajustement parfait avec un modèle avec assez de paramètres
- le sur-apprentissage est aussi problématique que la biais (rater des motifs systématiques)

Évaluation de la performance : prévision ponctuelle

Ensemble d'entraînement : $\{x_1, \dots, x_T\}$ $\{x_{T+1}, \dots, x_{T+n}\}$

Erreurs de prévision : $e_{T+h} = x_{T+h} - \hat{x}_{T+h}$

Différences avec les résidus :

- les erreurs de prévisions sont calculées sur l'ensemble de test
- l'horizon de prévision peut varier (pas forcément un pas de un)

Critères de performance : régression

- La racine carrée de l'erreur quadratique moyenne (RMSE)

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (e_{T+i})^2}$$

- L'erreur absolue moyenne (MAE)

$$\frac{1}{n} \sum_{i=1}^n |e_{T+i}|$$

- L'erreur absolue relative moyenne en pourcentage (MAPE)

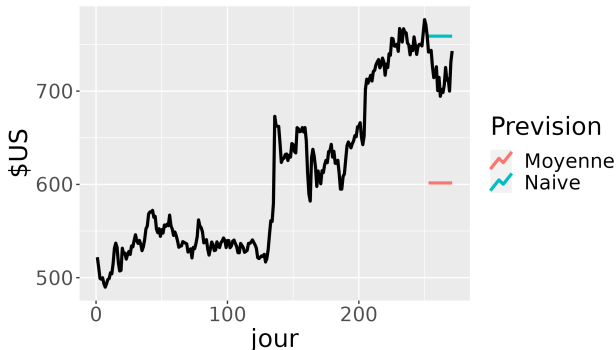
$$\frac{1}{n} \sum_{i=1}^n \frac{|e_{T+i}|}{|x_{T+i}|} \times 100$$

➔ RMSE et MAE sont à l'échelle des données

➔ MAPE est sans échelle

➔ Plusieurs autres possibilités : en particulier, pour évaluer la distribution de la prévision

Exemple : prix de fermeture de Google en 2015



Méthode	RMSE	MAE	MAPE
Moyenne	118	117	16.2
Naive	43.4	40.4	5.67

Validation séquentielle : plusieurs partitions

- Dans une partition entraînement-test, si l'ensemble de test est long, le modèle devra faire la prévision pour un horizon temporel très éloigné.
- La validation séquentielle suit le même type de stratégie que la validation croisée mais avec prise en compte de la chronologie des observations.

Validation séquentielle : plusieurs partitions

- Dans une partition entraînement-test, si l'ensemble de test est long, le modèle devra faire la prévision pour un horizon temporel très éloigné.
- La validation séquentielle suit le même type de stratégie que la validation croisée mais avec prise en compte de la chronologie des observations.

Deux façons de procéder :

① Ensemble d'entraînement incrémental :

- les N observations initiales forment l'ensemble d'entraînement initial ;
- les n observations suivantes forment l'ensemble de test initial ;
- l'ensemble d'entraînement suivant comprend les ensembles d'entraînement et de test initiaux ;
- on itère jusqu'à avoir couvert tout le jeu de données.

Validation séquentielle : plusieurs partitions

- Dans une partition entraînement-test, si l'ensemble de test est long, le modèle devra faire la prévision pour un horizon temporel très éloigné.
- La validation séquentielle suit le même type de stratégie que la validation croisée mais avec prise en compte de la chronologie des observations.

Deux façons de procéder :

① Ensemble d'entraînement incrémental :

- les N observations initiales forment l'ensemble d'entraînement initial ;
- les n observations suivantes forment l'ensemble de test initial ;
- l'ensemble d'entraînement suivant comprend les ensembles d'entraînement et de test initiaux ;
- on itère jusqu'à avoir couvert tout le jeu de données.

② Ensemble d'entraînement mobile :

un ensemble d'entraînement est créé en faisant glisser vers l'avant de n pas de temps l'ensemble d'entraînement précédent.

Validation séquentielle : plusieurs partitions

N doit être choisi de façon à ce que l'ensemble d'entraînement initial contiennent les motifs représentatifs de la série.

n peut coïncider avec l'horizon avec lequel le modèle devra faire la prévision en pratique.

Validation séquentielle : plusieurs partitions

N doit être choisi de façon à ce que l'ensemble d'entraînement initial contiennent les motifs représentatifs de la série.

n peut coïncider avec l'horizon avec lequel le modèle devra faire la prévision en pratique.

L'ensemble d'entraînement incrémental peut être justifié si le motif saisonnier et la tendance sont stables.

Cependant, les ensembles d'entraînement sont de tailles différentes. En principe, si la série est stationnaire, plus l'ensemble d'entraînement est grand meilleur l'apprentissage.

Validation séquentielle : plusieurs partitions

N doit être choisi de façon à ce que l'ensemble d'entraînement initial contiennent les motifs représentatifs de la série.

n peut coïncider avec l'horizon avec lequel le modèle devra faire la prévision en pratique.

L'ensemble d'entraînement incrémental peut être justifié si le motif saisonnier et la tendance sont stables.

Cependant, les ensembles d'entraînement sont de tailles différentes. En principe, si la série est stationnaire, plus l'ensemble d'entraînement est grand meilleur l'apprentissage.

L'approche avec l'ensemble d'entraînement mobile permet d'avoir des ensembles d'entraînement tous de même taille et peut être plus adéquate en présence de changements structurels dans la série.

Trois types de lissage exponentiel (conceptuellement semblables aux moyennes mobiles) :

- lissage exponentiel simple ;
 - suppose un niveau (moyenne) constant : ne permet pas de composantes tendance et/ou saisonnière.
- modèle de Holt ;
 - Permet une composante tendance.
- modèle de Holt-Winters.
 - Permet une composante tendance et une composante saisonnière.

Lissage exponentiel simple

Moyenne pondérée **de façon exponentielle selon le lag** : plus l'observation est loin dans le temps plus le poids est faible.

Contrairement aux moyennes mobiles, toutes les observations entrent dans le calcul de la moyenne.

Lissage exponentiel simple

Moyenne pondérée **de façon exponentielle selon le lag** : plus l'observation est loin dans le temps plus le poids est faible.

Contrairement aux moyennes mobiles, toutes les observations entrent dans le calcul de la moyenne.

Deux équations avec paramètre de lissage $0 \leq \alpha \leq 1$

$$\text{Prévision } \hat{x}_{t+h|t} = L_t$$

$$\text{Lissage exponentiel } L_t = \alpha x_t + (1 - \alpha)L_{t-1}$$

Valeurs ajustées : $h = 1$

Prévisions : $t = T$

Lissage exponentiel simple

Moyenne pondérée **de façon exponentielle selon le lag** : plus l'observation est loin dans le temps plus le poids est faible.

Contrairement aux moyennes mobiles, toutes les observations entrent dans le calcul de la moyenne.

Deux équations avec paramètre de lissage $0 \leq \alpha \leq 1$

$$\begin{array}{ll} \text{Prévision} & \hat{x}_{t+h|t} = L_t \\ \text{Lissage exponentiel} & L_t = \alpha x_t + (1 - \alpha)L_{t-1} \end{array}$$

Valeurs ajustées : $h = 1$ Prévisions : $t = T$

→ Intermédiaire entre la méthode de la moyenne et la méthode naïve

Lissage exponentiel simple

On remplace L_{t-1} par la même expression de façon récursive :

$$\begin{aligned}L_t &= \alpha x_t + (1 - \alpha) (\alpha x_{t-1} + (1 - \alpha) L_{t-2}) \\&= \alpha x_t + (1 - \alpha) \alpha x_{t-1} + (1 - \alpha)^2 L_{t-2} \\&= \alpha x_t + (1 - \alpha) \alpha x_{t-1} + (1 - \alpha)^2 \alpha x_{t-2} + \cdots + \\&\quad (1 - \alpha)^{t-2} \alpha x_2 + (1 - \alpha)^{t-1} L_1.\end{aligned}$$

On peut ré-écrire les expressions précédentes de la façon suivante :

$$L_t = \sum_{i=0}^{t-2} \alpha (1 - \alpha)^i x_{t-i} + (1 - \alpha)^{t-1} L_1.$$

Il faut initialiser L_1 et optimiser α (minimisation des carrés des erreurs).

Lissage exponentiel simple

On remplace L_{t-1} par la même expression de façon récursive :

$$\begin{aligned}L_t &= \alpha x_t + (1 - \alpha) (\alpha x_{t-1} + (1 - \alpha) L_{t-2}) \\&= \alpha x_t + (1 - \alpha) \alpha x_{t-1} + (1 - \alpha)^2 L_{t-2} \\&= \alpha x_t + (1 - \alpha) \alpha x_{t-1} + (1 - \alpha)^2 \alpha x_{t-2} + \cdots + \\&\quad (1 - \alpha)^{t-2} \alpha x_2 + (1 - \alpha)^{t-1} L_1.\end{aligned}$$

On peut ré-écrire les expressions précédentes de la façon suivante :

$$L_t = \sum_{i=0}^{t-2} \alpha (1 - \alpha)^i x_{t-i} + (1 - \alpha)^{t-1} L_1.$$

Il faut initialiser L_1 et optimiser α (minimisation des carrés des erreurs).

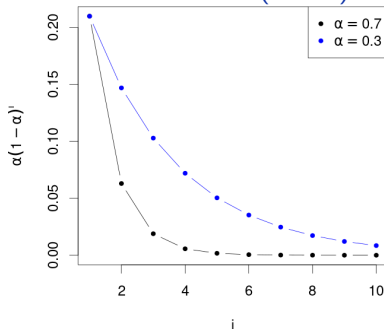
La prévision d'horizon h est donné par :

$$\hat{x}_{T+h|T} = \hat{x}_{T+1|T} = L_T \quad h = 1, 2, 3, \dots$$

Lissage exponentiel simple

$$L_t = \sum_{i=0}^{t-2} \alpha(1-\alpha)^i x_{t-i} + (1-\alpha)^{t-1} L_1.$$

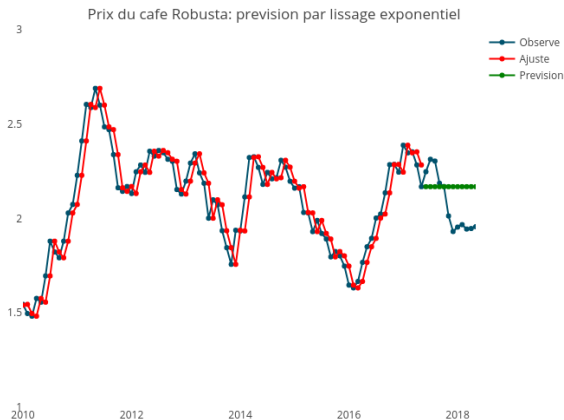
Décroissance de $\alpha(1-\alpha)^i$



→ $\alpha \downarrow 0$: méthode moyenne

→ $\alpha \uparrow 1$: méthode naïve

Exemple : prix du café Robusta



→ La valeur initiale $\hat{L}_1 = 0.6957$ s'approche de $x_1 = 0.696$.

→ $\hat{\alpha} = 0.9999$: pratiquement le modèle naïf.

Modèle de Holt : tendance linéaire

Trois équations avec deux paramètres de lissage $0 \leq \alpha, \beta \leq 1$

$$\text{Prévision } \hat{x}_{t+h|t} = L_t + h B_t$$

$$\text{Lissage exponentiel (niveau) } L_t = \alpha \underbrace{x_t}_{\text{observée}} + (1 - \alpha) \underbrace{(L_{t-1} + B_{t-1})}_{\text{prédite}}$$

$$\text{Lissage exponentiel (pente) } B_t = \beta(L_t - L_{t-1}) + (1 - \beta)B_{t-1}$$

Modèle de Holt : tendance linéaire

Trois équations avec deux paramètres de lissage $0 \leq \alpha, \beta \leq 1$

$$\text{Prévision } \hat{x}_{t+h|t} = L_t + h B_t$$

$$\text{Lissage exponentiel (niveau)} \quad L_t = \alpha \underbrace{x_t}_{\text{observée}} + (1 - \alpha) \underbrace{(L_{t-1} + B_{t-1})}_{\text{prédite}}$$

$$\text{Lissage exponentiel (pente)} \quad B_t = \beta(L_t - L_{t-1}) + (1 - \beta)B_{t-1}$$

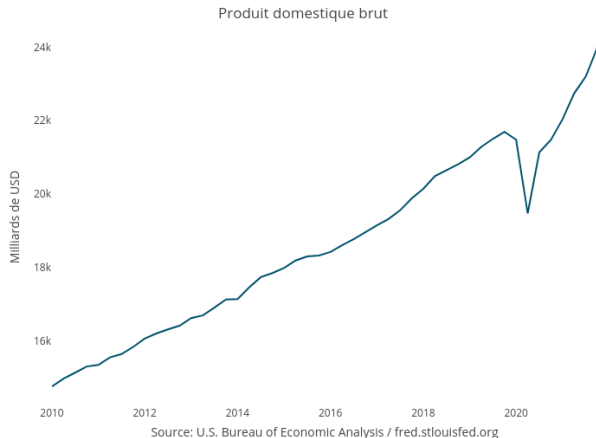
Processus récursif : il faut initialiser L_2 et B_2 .

→ Un choix naturel $L_2 = x_2$ et $B_2 = x_2 - x_1$.

→ Estimation des paramètres de lissage α et β par minimisation de l'erreur quadratique :

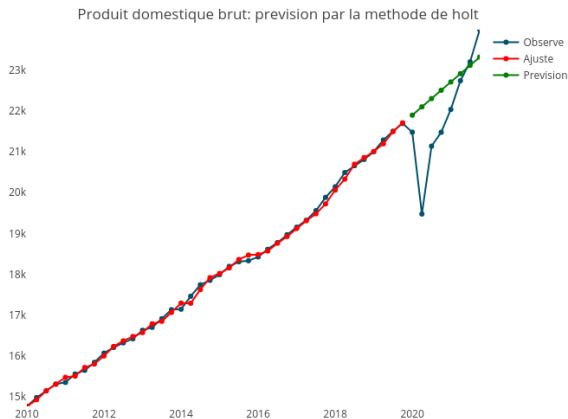
$$\sum_{i=3}^T (x_i - \hat{x}_i)^2.$$

Modèle de Holt : série du produit domestique brut



➔ La série a été dé-saisonnalisée, elle comporte une tendance linéaire forte et un saut au premier trimestre de 2020.

Modèle de Holt : série du produit domestique brut



→ Comment faire pour modéliser le creux du premier trimestre 2020 ?

Modèle de Holt-Winters

Généralisation de la prévision à l'horizon h en présence de tendance linéaire et de composante saisonnière de période m :

$$\hat{x}_{t+h|t} = L_t + hB_t + S_{t+h-m(k+1)} \quad \text{où } k = \lfloor (h-1)/m \rfloor$$

où L_t est le niveau, B_t est la pente et $S_{t+h-m(k+1)}$ est la composante saisonnière associés au pas de temps t .

Modèle de Holt-Winters

Généralisation de la prévision à l'horizon h en présence de tendance linéaire et de composante saisonnière de période m :

$$\hat{x}_{t+h|t} = L_t + hB_t + S_{t+h-m(k+1)} \quad \text{où } k = \lfloor (h-1)/m \rfloor$$

où L_t est le niveau, B_t est la pente et $S_{t+h-m(k+1)}$ est la composante saisonnière associés au pas de temps t .

Trois équations de lissage

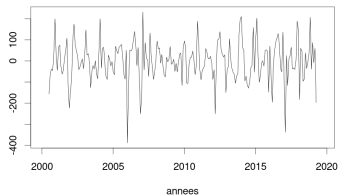
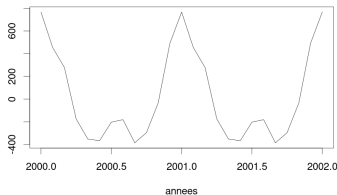
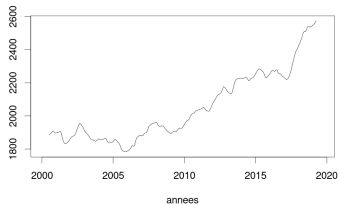
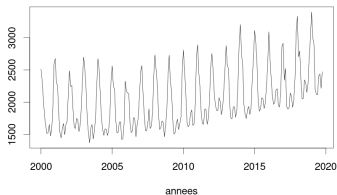
Les valeurs de L_t , B_t et S_t , avec $t = (m+2), \dots, T$ sont obtenues par récursion :

$$\begin{aligned} L_t &= \alpha(x_t - S_{t-m}) + (1 - \alpha)(L_{t-1} + B_{t-1}) \\ B_t &= \beta(L_t - L_{t-1}) + (1 - \beta)B_{t-1} \\ S_t &= \underbrace{\gamma(x_t - L_t - B_t)}_{\text{actuel}} + (1 - \gamma) \underbrace{(S_{t-m})}_{\text{précédent}}. \end{aligned}$$

→ Il faut initialiser : L_{m+1} , B_{m+1} et S_j avec $1 \leq j \leq m$.

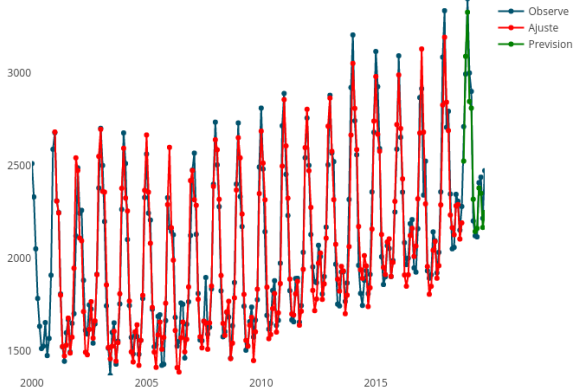
Modèle de Holt-Winters : consommation de gaz mensuel US

Décomposition (fonction `decompose`) :
séries - tendance - composante saisonnière - composante irrégulière



Modèle de Holt-Winters : consommation de gaz mensuel US

Consommation de gaz mensuel USA: prevision par la methode de Holt-Winters



Modèle de Holt-Winters : consommation de gaz mensuel US

Holt-Winters exponential smoothing with trend and additive seasonal component.

Smoothing parameters:

alpha: 0.371213

beta : 0

gamma: 0.4422456

→ Les composantes niveau et saisonnière ont des paramètres de lissage non nuls.

Coefficients:

[,1]

a 2491.9930104

b -0.1287005

s1 32.0972651

s2 597.1088003

s3 834.9628439

s4 353.8593860

s5 318.1927058

s6 -173.0721496

s7 -346.6229990

s8 -329.7169608

s9 -112.1664217

s10 -140.3186476

s11 -324.5343787

s12 -243.9334551

MTH8304 - Apprentissage non-supervisé et séries chronologiques

10. Séries chronologiques: analyses préliminaires

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

Analyse des séries chronologiques

Objectif : extraire de l'information d'une série temporelle pour en comprendre les mécanismes et faire des prévisions

Données : espacées régulièrement dans le temps

Décomposition : identifier des structures ou motifs récurrents (saisonnalité, tendance, cycles)

Bruit blanc : composante irrégulière

Prévision : lissage exponentiel, moyenne mobile, ARIMA

Analyse des séries chronologiques

Objectif : extraire de l'information d'une série temporelle pour en comprendre les mécanismes et faire des prévisions

Données : espacées régulièrement dans le temps

Décomposition : identifier des structures ou motifs récurrents (saisonnalité, tendance, cycles)

Bruit blanc : composante irrégulière

Prévision : lissage exponentiel, moyenne mobile, ARIMA

➔ Associée à quelle catégorie de l'apprentissage statistique ?

1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

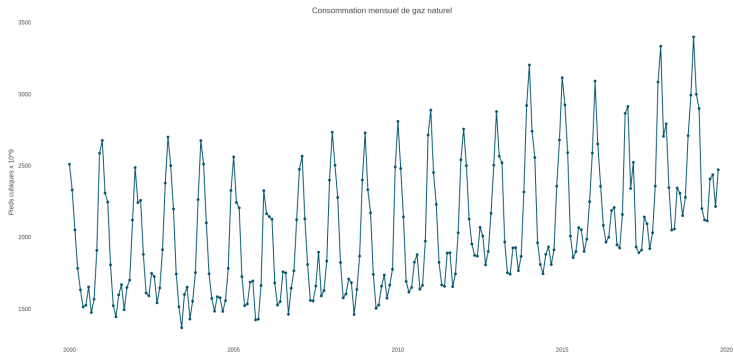
3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

Données de consommation mensuelle de gaz

Séries de données indexées temporellement :

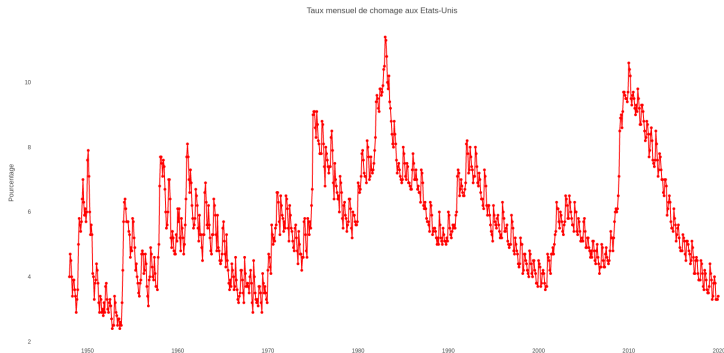
$$\{x_{t_1}, x_{t_2}, \dots, x_{t_n}\} \quad t_i \in [T_i, T_f]$$



→ Saisonnalité : cycles récurrents sur l'année

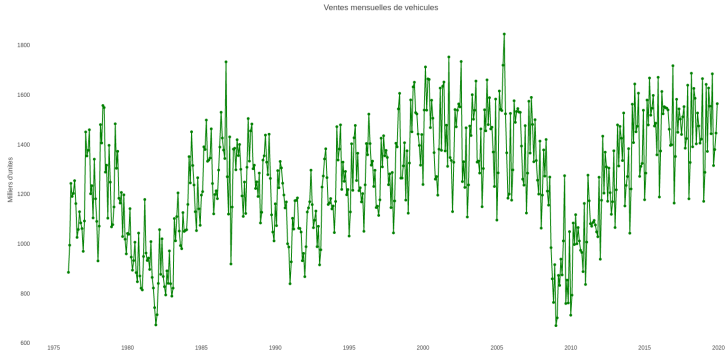
→ Tendence

Données du taux mensuel de chômage



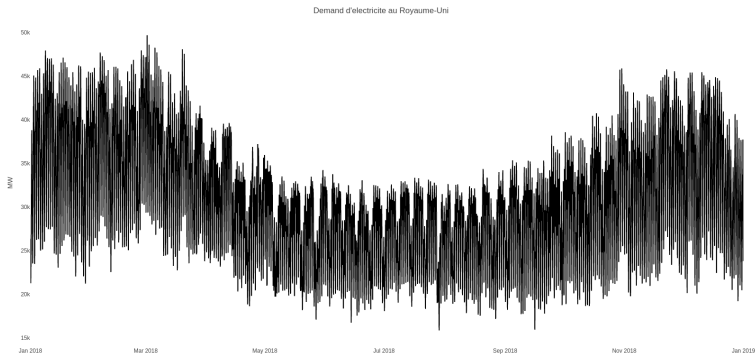
→ Cycles : motifs temporels récurrents

Données de ventes mensuelles de véhicules



- Saisonnalité : cycles récurrents sur l'année
- Cycles : motifs temporels récurrents

Données de la demande horaire en électricité



→ Saisonnalité : cycles récurrents sur la journée et sur l'année

- Tendance : croissance/décroissance sur le long terme
→ pas forcément linéaire
- Saisonnalité : effets de facteurs saisonniers (le jour de l'année ou de la semaine)
→ période récurrente fixe et connue
- Cycle : hausses et baisses sans fréquence fixe
→ Causé par les conditions économiques, durée ≥ 2 ans

1 Introduction

2 Analyses graphiques

- Graphiques temporels
- **Graphiques saisonniers**
- Lags et auto-corrélation

3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

Saisonnalité simple ou multiple

Saisonnalité :

- dépendance à l'orbite de la Terre autour du Soleil (cycles annuels - 365.25 jours)
- dépendance à la rotation de la Terre (cycles diurnes - 24h)

Saisonnalité simple ou multiple

Saisonnalité :

- dépendance à l'orbite de la Terre autour du Soleil (cycles annuels - 365.25 jours)
- dépendance à la rotation de la Terre (cycles diurnes - 24h)

→ La température de l'air affiche des cycles annuels et diurnes

Saisonnalité simple ou multiple

Saisonnalité :

- dépendance à l'orbite de la Terre autour du Soleil (cycles annuels - 365.25 jours)
- dépendance à la rotation de la Terre (cycles diurnes - 24h)

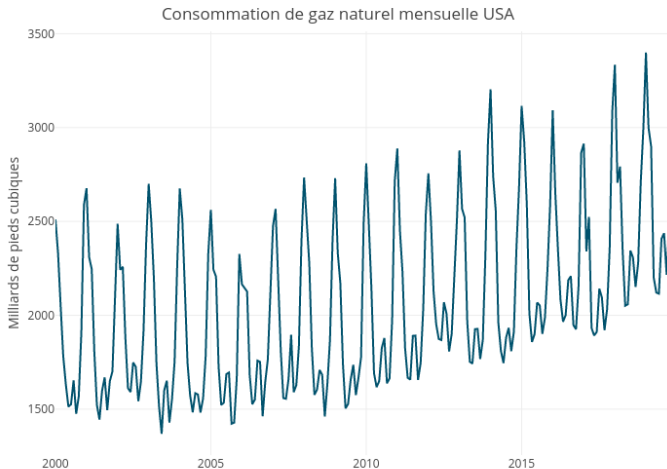
→ La température de l'air affiche des cycles annuels et diurnes

Saisonnalité simple : un seul motif saisonnier dans la série chronologique

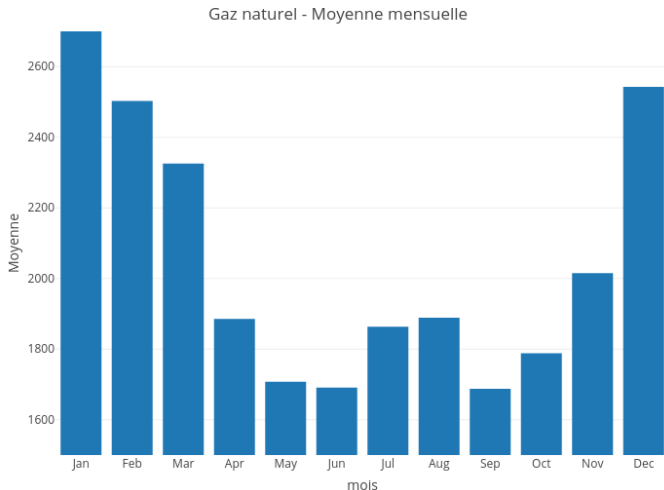
Saisonnalité multiple : plusieurs motifs saisonniers dans la série chronologique

→ Se produit plutôt pour des séries à haute fréquence, e.g., horaire

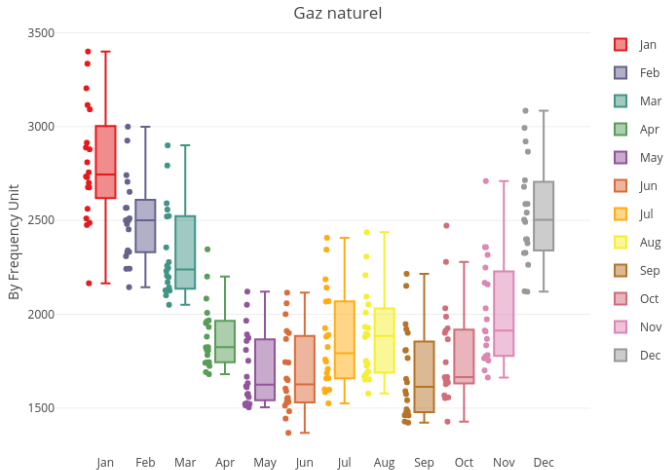
Différents graphiques : mettre en évidence la saisonnalité



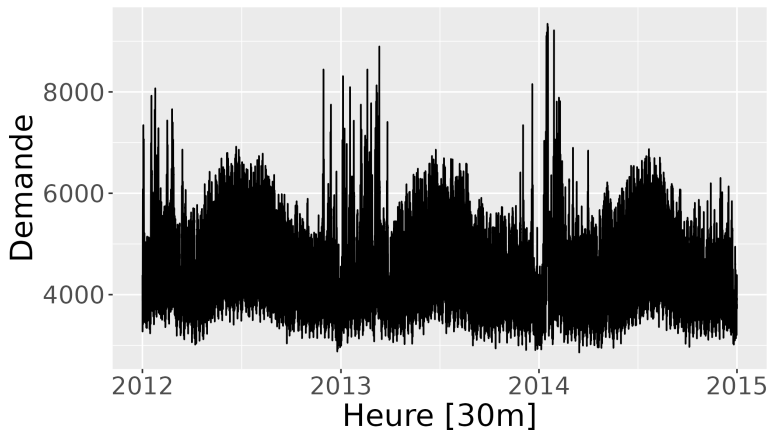
Différents graphiques : mettre en évidence la saisonnalité



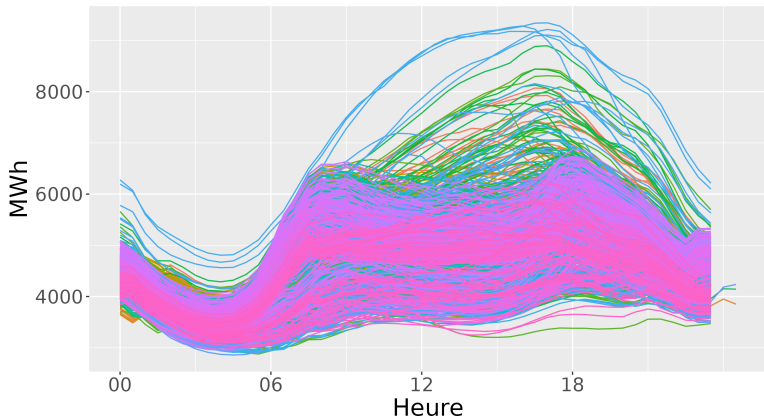
Différents graphiques : mettre en évidence la saisonnalité



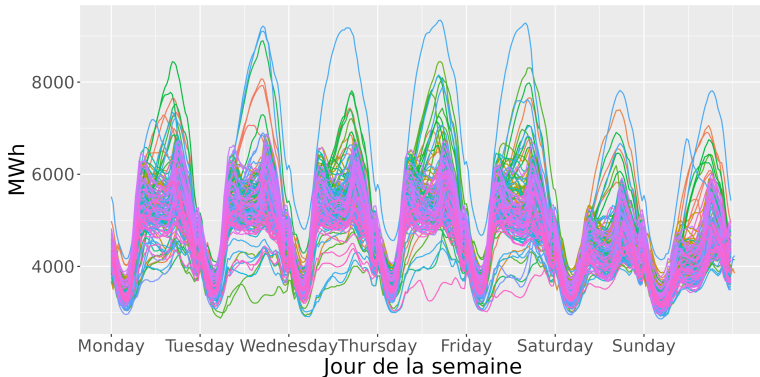
Demande électricité - état de Victoria, Australie



Demande électricité - état de Victoria, Australie



Demande électricité - état de Victoria, Australie



1 Introduction

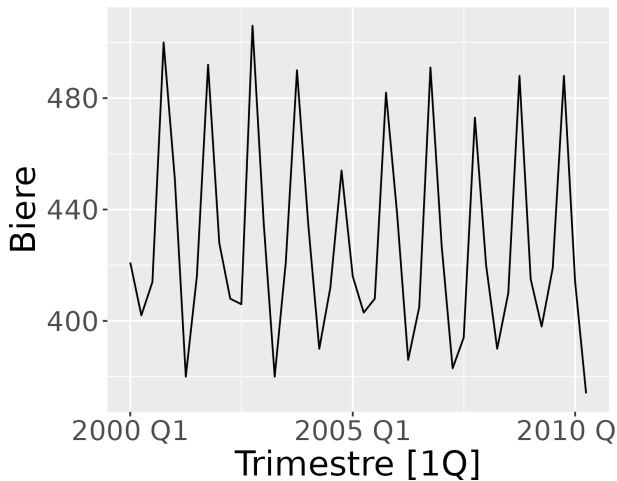
2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

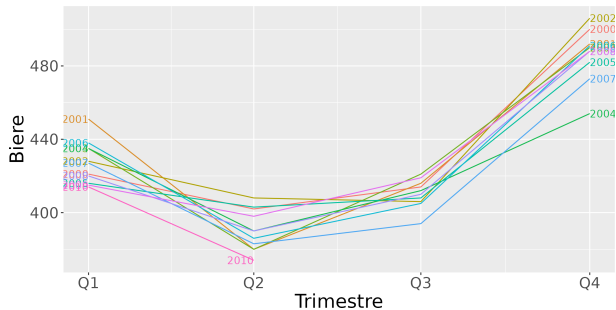
3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

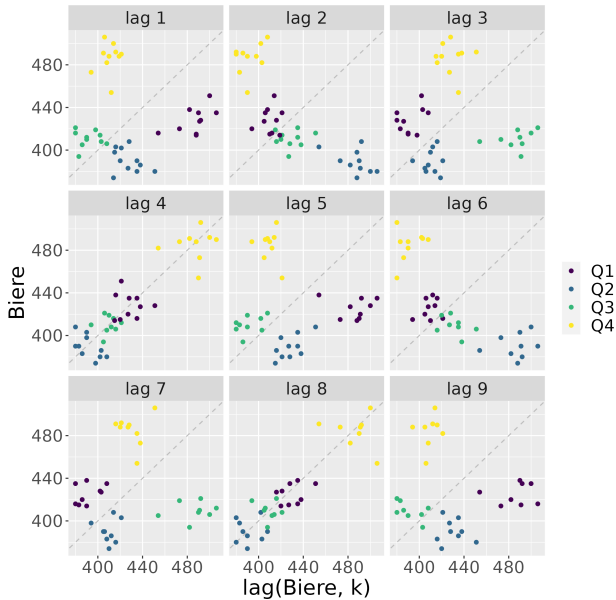
Production de bières



Production de bières

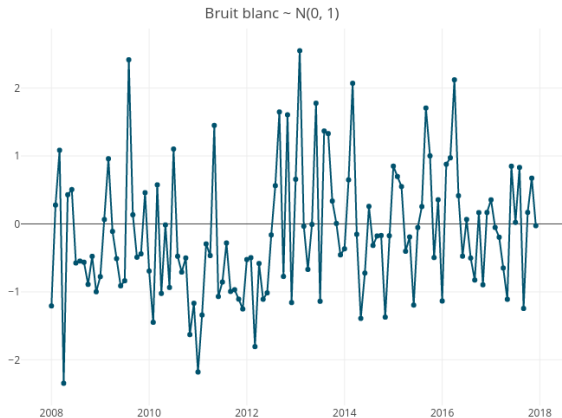


Production de bières



Bruit blanc

Séries de v.a. non-corrélées avec $\mu = 0$ et $\sigma = 1$



Fonction d'auto-corrélation

→ Un outil permettant de détecter un bruit blanc et la présence de motifs

Rappel : Corrélation entre v.a. X et Y

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Cadre d'une série stationnaire : $\{X_t\}_{t=1}^T$

Covariance de lag k :

$$\gamma_X(k) = \text{Cov}(X_{t-k}, X_t) \Rightarrow \text{ne dépend pas de } t$$

Fonction d'auto-corrélation

→ Un outil permettant de détecter un bruit blanc et la présence de motifs

Rappel : Corrélation entre v.a. X et Y

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Cadre d'une série stationnaire : $\{X_t\}_{t=1}^T$

Covariance de lag k :

$$\gamma_X(k) = \text{Cov}(X_{t-k}, X_t) \Rightarrow \text{ne dépend pas de } t$$

Fonction d'auto-corrélation de lag k :

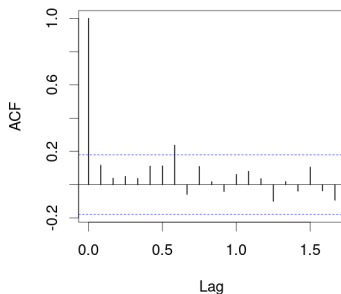
$$\rho_X(k) = \frac{\gamma_X(k)}{\gamma_X(0)}$$

Estimateur échantillonnal :

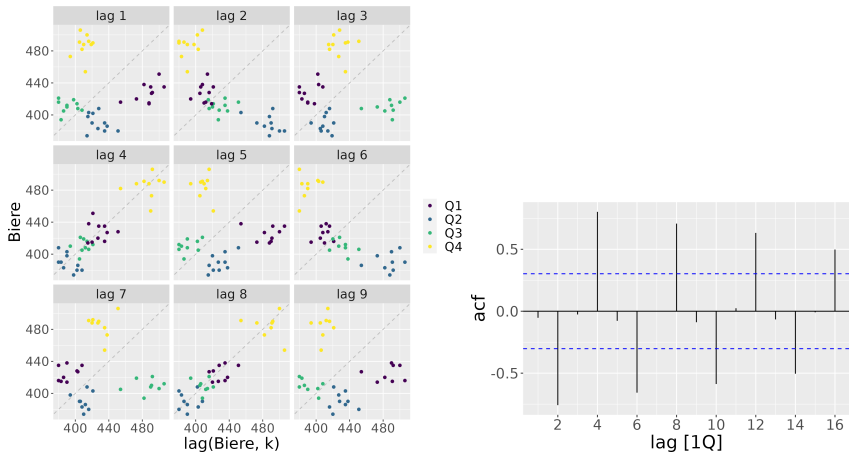
$$r_k = \frac{\sum_{t=k+1}^T (x_{t-k} - \bar{x})(x_t - \bar{x})}{\sum_{t=1}^T (x_t - \bar{x})^2}$$

où $\{x_1, x_2, \dots, x_T\}$ est un échan-

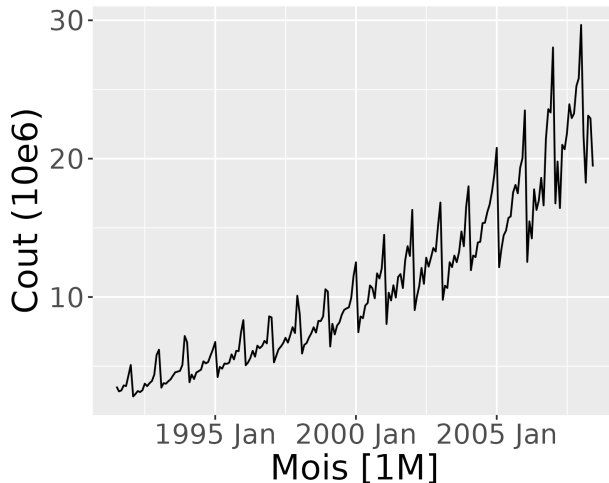
Bruit blanc



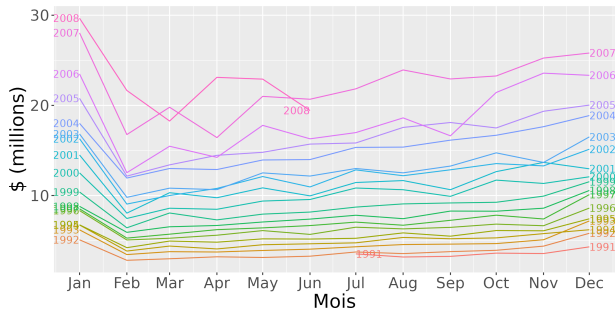
Production de bières



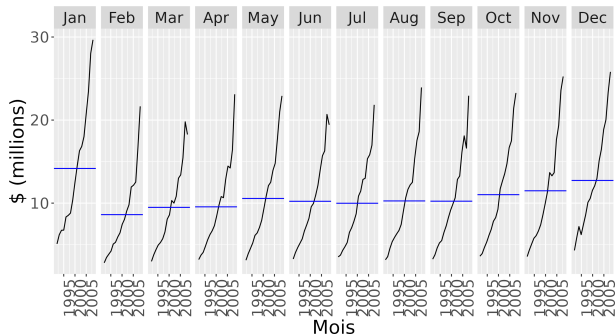
Ventes de médicaments contre le diabète



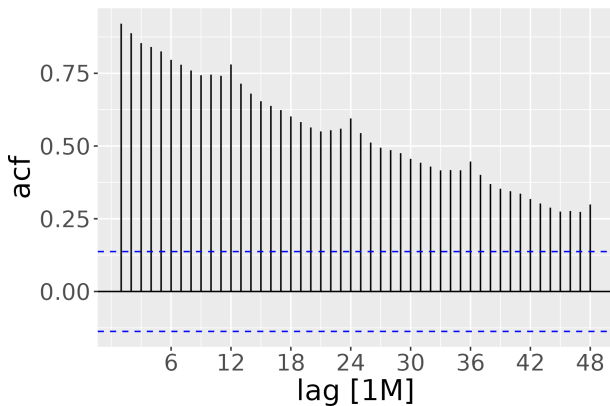
Ventes de médicaments contre le diabète



Ventes de médicaments contre le diabète



Ventes de médicaments contre le diabète



1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

3 Décomposition

- **Types de structures**
- Moyennes mobiles
- Approche classique

Trois types de motifs : tendance, saisonnalité et cycles

Séries chronologiques se décomposent en :

- ① tendance-cycle
- ② composante saisonnière
- ③ composante aléatoire ou résiduelle

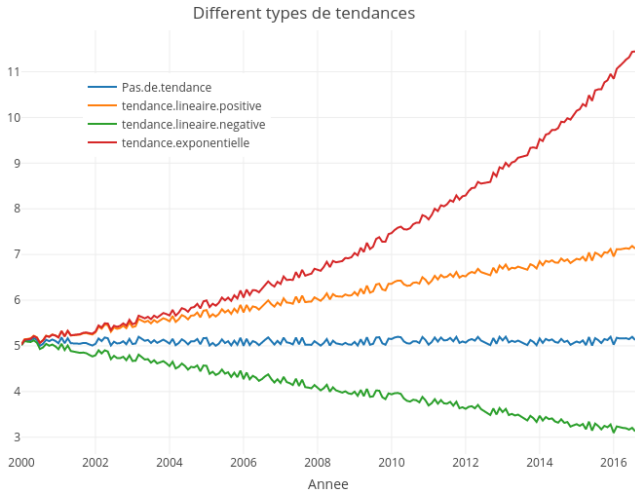
Décomposition en trois composantes :

- meilleure compréhension des structures présentes dans la série
- meilleure prévision

Tendance

Direction générale de la série dans le temps

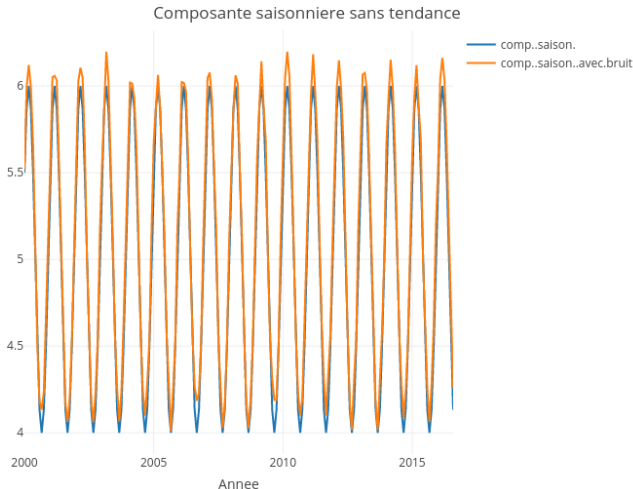
➔ par exemple : décroissance ou croissance



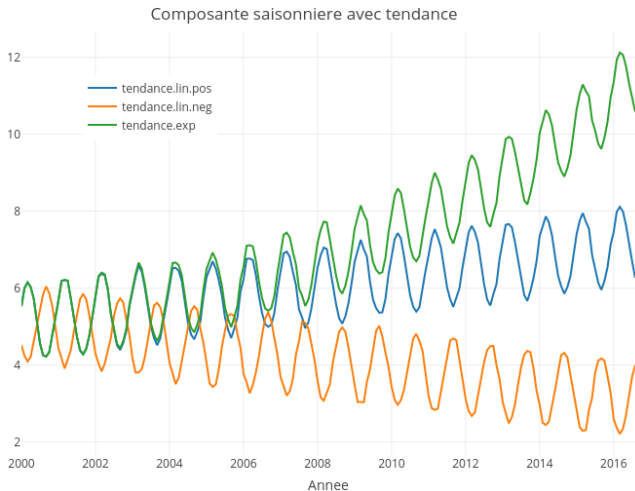
Composante saisonnière

Variations se produisant avec une certaine fréquence

→ généralement reliées au cycle saisonnier ou au cycle diurne



Composante saisonnière avec tendance



Décomposition d'une série chronologique

Composantes structurelles

- T_t : tendance
- S_t : composante saisonnière

Composante non-structurelle

- I_t : composante aléatoire ou résiduelle

Décomposition d'une série chronologique

Composantes structurelles

- T_t : tendance
- S_t : composante saisonnière

Composante non-structurelle

- I_t : composante aléatoire ou résiduelle

Séries chronologiques avec une **structure additive** :

$$X_t = T_t + S_t + I_t$$

Séries chronologiques avec une **structure multiplicative** :

$$X_t = T_t \times S_t \times I_t$$

Décomposition d'une série chronologique

Composantes structurelles

- T_t : tendance
- S_t : composante saisonnière

Composante non-structurale

- I_t : composante aléatoire ou résiduelle

Séries chronologiques avec une **structure additive** :

$$X_t = T_t + S_t + I_t$$

Séries chronologiques avec une **structure multiplicative** :

$$X_t = T_t \times S_t \times I_t$$

Ajustement saisonnier : $X_t - S_t = T_t + I_t$ ou $X_t/S_t = T_t \times I_t$

→ Par exemple, pour mettre en évidence l'état de l'économie

Composante aléatoire ou résiduelle

Se définit comme ce qui reste après la prise en compte des composantes structurelles

Génère des fluctuations irrégulières dans les observations

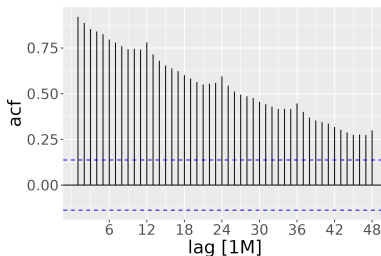
Composante aléatoire ou résiduelle

Se définit comme ce qui reste après la prise en compte des composantes structurelles

Génère des fluctuations irrégulières dans les observations

Fournit de l'information sur la justesse de l'estimation des composantes structurelles

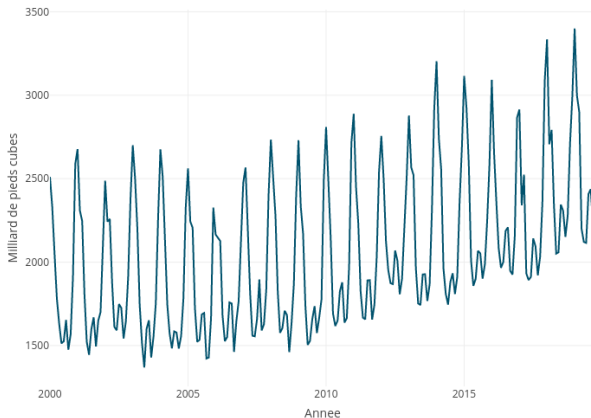
Par exemple, si une forte auto-corrélation est présente, il se peut qu'une partie de la structure n'a pas été prise en compte



Modèle additif

$$X_t = T_t + S_t + I_t$$

Consommation Gaz Naturel USA

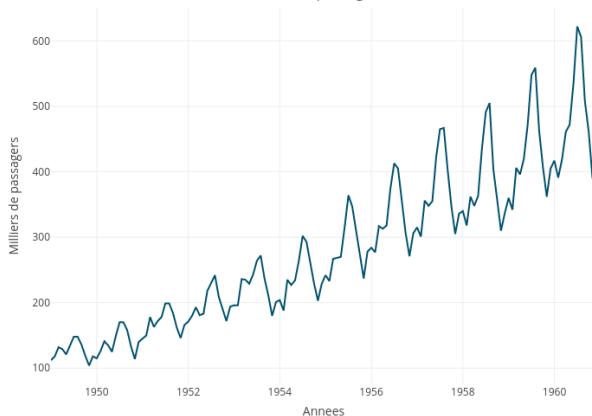


Une série a un comportement “additif” si le taux de croissance (tendance) et la composante saisonnière sont constants

Modèle multiplicatif

$$X_t = T_t \times S_t \times I_t$$

Nombre mensuel de passagers arien



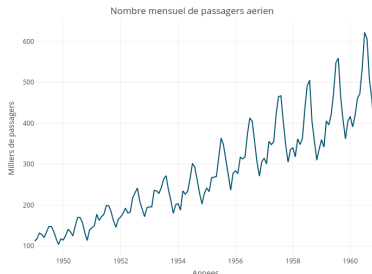
Une série a un comportement “multiplicatif” si le taux de croissance (tendance) et/ou la composante saisonnière augmentent

Transformation préliminaire

But : Simplifier la décomposition et les analyses

Quand ?

La variation augmente ou diminue avec le niveau de la série



Transformation logarithmique (séries positives) :

modèle multiplicatif → modèle additif

Transformation de puissance : racine carrée ou cubique

Transformation de Box & Cox

$$x'_t = \begin{cases} \frac{x_t^\lambda - 1}{\lambda} & \text{si } \lambda \neq 0 \\ \ln(x_t) & \text{si } \lambda = 0 \end{cases}$$

→ pour des séries positives

Si $\lambda = 0$: logarithme en base naturel

Si $\lambda = 1$: décalage de 1 (pas d'effets)

Autres valeurs de λ : fonctions de puissance

Transformation de Box & Cox

$$x'_t = \begin{cases} \frac{x_t^\lambda - 1}{\lambda} & \text{si } \lambda \neq 0 \\ \ln(x_t) & \text{si } \lambda = 0 \end{cases}$$

Si $\lambda = 0$: logarithme en base naturel

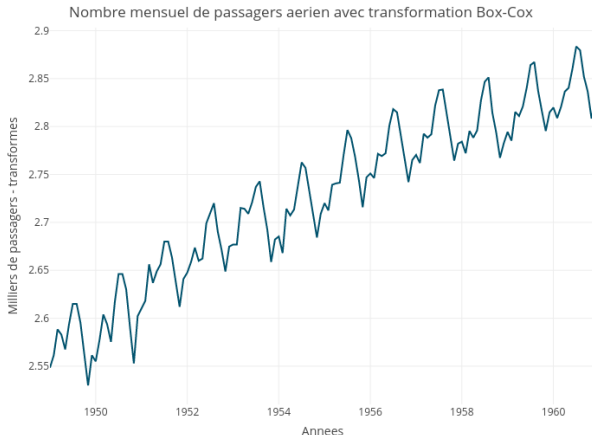
Si $\lambda = 1$: décalage de 1 (pas d'effets)

Autres valeurs de λ : fonctions de puissance

→ pour des séries positives

$$\lambda_{\text{opt}} = -0.2947$$

Choix automatique de λ



1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

3 Décomposition

- Types de structures
- **Moyennes mobiles**
- Approche classique

Technique permettant de lisser un signal :

- enlever le bruit (composante aléatoire)
- laisser la structure (composante systématique)
- estimation de la composante tendance-cycle

Technique permettant de lisser un signal :

- enlever le bruit (composante aléatoire)
- laisser la structure (composante systématique)
- estimation de la composante tendance-cycle

Deux composantes à fixer :

- la forme et la largeur de la fenêtre
- la façon de calculer la moyenne (arithmétique ou pondérée)

Moyenne mobile : forme de la fenêtre

Observations : $\{x_1, x_2, \dots, x_n\}$ régulièrement indexées dans $[T_i, T_f]$

Moyenne mobile d'un seul côté (passé) de fenêtre m :

$$\hat{x}_k = \frac{1}{m} \sum_{i=1}^m x_{k-i+1} \quad \text{où } m \leq k$$

Moyenne mobile : forme de la fenêtre

Observations : $\{x_1, x_2, \dots, x_n\}$ régulièrement indexées dans $[T_i, T_f]$

Moyenne mobile d'un seul côté (passé) de fenêtre m :

$$\hat{x}_k = \frac{1}{m} \sum_{i=1}^m x_{k-i+1} \quad \text{où } m \leq k$$

Moyenne mobile de deux côtés de fenêtre $m = n_1 + n_2 + 1$:

$$\hat{x}_k = \frac{1}{m} \sum_{i=1}^m x_{k-n_1-1+i} \quad \text{où } n_1 < k < n - n_2$$

➔ Moyenne mobile centrée d'ordre $m = 2n + 1$

Moyenne mobile : forme de la fenêtre

Observations : $\{x_1, x_2, \dots, x_n\}$ régulièrement indexées dans $[T_i, T_f]$

Moyenne mobile d'un seul côté (passé) de fenêtre m :

$$\hat{x}_k = \frac{1}{m} \sum_{i=1}^m x_{k-i+1} \quad \text{où } m \leq k$$

Moyenne mobile de deux côtés de fenêtre $m = n_1 + n_2 + 1$:

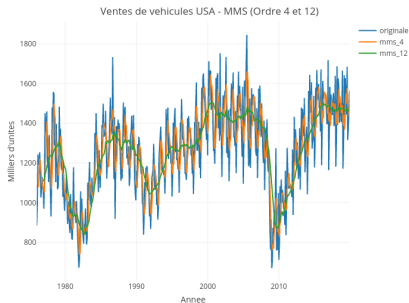
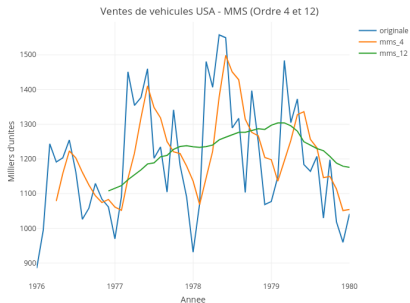
$$\hat{x}_k = \frac{1}{m} \sum_{i=1}^m x_{k-n_1-1+i} \quad \text{où } n_1 < k < n - n_2$$

➔ Moyenne mobile centrée d'ordre $m = 2n + 1$

➔ Choix de la taille de la fenêtre m influence le niveau de lissage et la perte associée (le nombre d'observations éliminées au début)

m : taille de voisinage

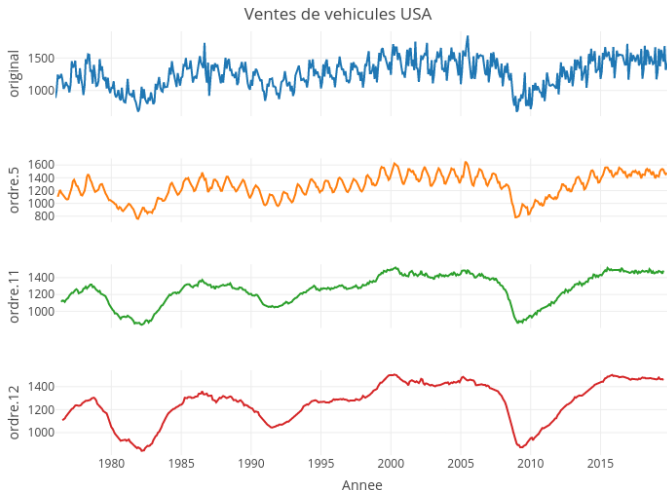
Moyenne mobile simple : version d'un seul côté - utile en prévision



- ➔ Avec une fenêtre de 4 mois : réduction des oscillations et du bruit
- ➔ Avec une fenêtre de 12 mois : réduction de plus des cycles saisonniers
- ➔ Permet de faire apparaître des tendances et des cycles

Effet : forme de fenêtre avec deux côtés

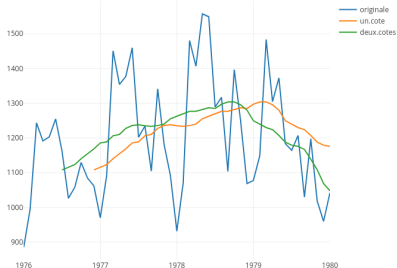
Fenêtres centrées (5 et 11) et non-centrée (12) - utile pour analyses



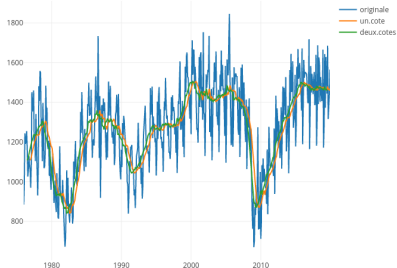
➔ A vue d'oeil, pas de grandes différences avec la structure de fenêtre avec un seul côté

Comparaison : forme de fenêtre avec un ou deux côtés

Moyenne mobile a un cote vs. deux cotes - Ordre 12



Moyenne mobile a un cote vs. deux cotes - Ordre 12



- La moyenne mobile à deux côtés précède celle qui a un côté
- Version à deux côtés : plus appropriée pour créer un filtre/lissage
- Version à un côté : plus appropriée si on veut se baser sur des observations récentes
- Coût associé à chaque version est différent

Moyenne mobile pondérée

Moyenne mobile d'un seul côté (passé) de fenêtre m avec poids $w_1, \dots, w_m \geq 0$:

$$\hat{x}_k = \sum_{i=1}^m w_i x_{k-i+1} \quad \text{où } m \leq k$$

avec $\sum_{i=1}^m w_i = 1$

➔ Utile pour pondérer en fonction de la distance dans le temps

Pour obtenir la moyenne arithmétique : $w_i = \frac{1}{m}$

Composition de moyennes mobiles

Moyenne mobile centrée avec $m = 2n + 1$ où m un nombre impair

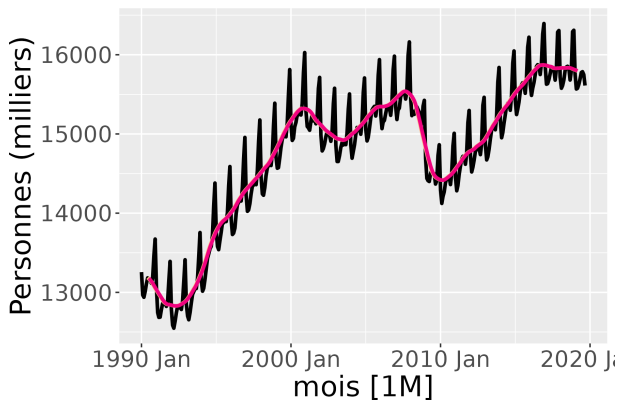
La composition de deux moyennes mobiles avec fenêtres paires est une moyenne mobile centrée :

$$\begin{aligned}\hat{x}_k^{(2-4)} &= \frac{1}{2} \left[\underbrace{\frac{1}{4}(x_{k-2} + x_{k-1} + x_k + x_{k+1})}_{\hat{x}_k^{(4)}} + \underbrace{\frac{1}{4}(x_{k-1} + x_k + x_{k+1} + x_{k+2})}_{\hat{x}_{k+1}^{(4)}} \right] \\ &= \frac{1}{8}x_{k-2} + \frac{1}{4}x_{k-1} + \frac{1}{4}x_k + \frac{1}{4}x_{k+1} + \frac{1}{8}x_{k+2}\end{aligned}$$

➔ Moyenne mobile pondérée et centrée

➔ Pour une série trimestrielle : enlève la variation saisonnière (même poids pour le même trimestre dans deux années consécutives)

Moyenne mobile 2-12



1 Introduction

2 Analyses graphiques

- Graphiques temporels
- Graphiques saisonniers
- Lags et auto-corrélation

3 Décomposition

- Types de structures
- Moyennes mobiles
- Approche classique

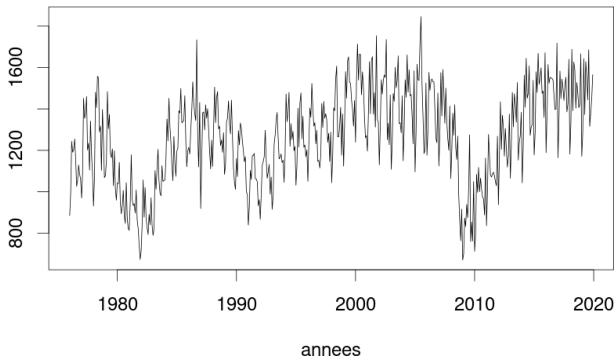
Décomposition additive

Soit m la période de la saison

- ① Si m est pair, calculer la composante tendance-cycle $\hat{T}_t$ avec une moyenne mobile centrée $2 - m$
Si m est impair, utiliser une moyenne mobile centrée m
- ② Enlever la composante tendance : $x_t - \hat{T}_t$
- ③ Estimer la composante saisonnière :
 - ① Calculer la moyenne inter- m périodes
 - ② Ajuster pour que la moyenne de la composante saisonnière soit zéro
 - ③ $\hat{S}_t$ répète ces valeurs pour couvrir la période temporelle de la série
- ④ Composante résiduelle : $x_t - \hat{T}_t - \hat{S}_t$

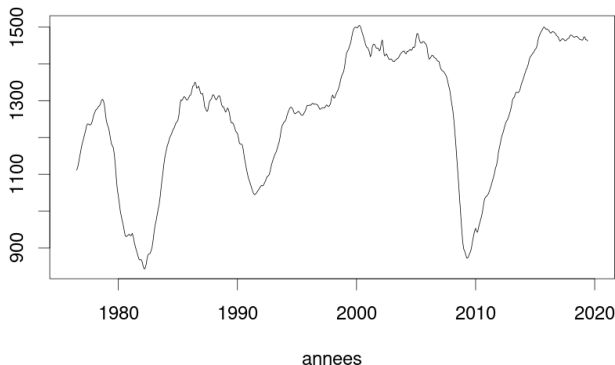
Décomposition : série initiale

Modèle additif pour les ventes de véhicules $X_t = T_t + S_t + I_t$



Décomposition : composante tendance

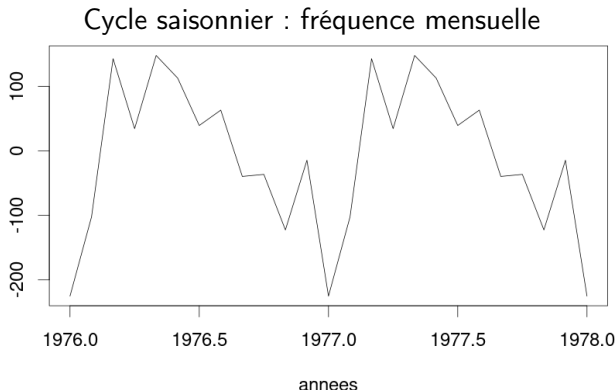
→ Moyenne mobile centrée



→ Prochaine étape, on travaille avec $X_t - \hat{T}_t = S_t + I_t$

Décomposition : composante saisonnière

→ Pour chaque mois : calcul de la moyenne inter-annuelle + ajustement moyenne

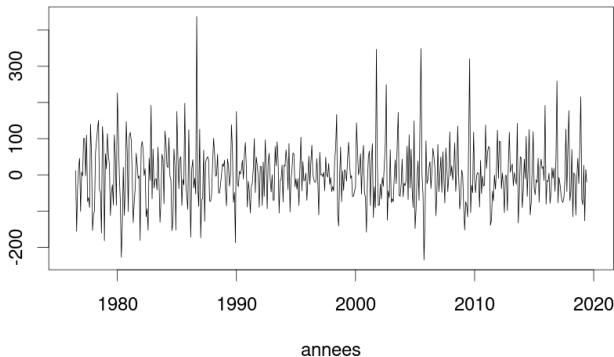


Cycle saisonnier constant : approche mieux adaptée au modèle additif

→ Prochaine étape, on travaille avec $X_t - \hat{T}_t - \hat{S}_t = I_t$

Décomposition : composante résiduelle

→ La composante irrégulière est déduite des étapes précédentes



→ La présence de structure dans la composante résiduelle est une indication que l'estimation des composantes structurelles est imparfaite

Décomposition multiplicative

Soit m la période de la saison

- ① Si m est pair, calculer la composante tendance-cycle $\hat{T}_t$ avec une moyenne mobile centrée $2 - m$
Si m est impair, utiliser une moyenne mobile centrée m
- ② Enlever la composante tendance : $x_t / \hat{T}_t$
- ③ Estimer la composante saisonnière :
 - ① Calculer la moyenne inter- m périodes
 - ② Ajuster pour que la moyenne de la composante saisonnière soit m
 - ③ $\hat{S}_t$ répète ces valeurs pour couvrir la période temporelle de la série
- ④ Composante résiduelle : $x_y / (\hat{T}_t \hat{S}_t)$

Décomposition STL

STL : Seasonal and Trend using Loess

Loess : forme de régression non-paramétrique

STL : Seasonal and Trend using Loess

Loess : forme de régression non-paramétrique

Avantages :

- Permet de prendre en compte tous les types de saisonnalité (mois, trimestre, semaine)
- la composante saisonnière peut changer dans le temps avec un taux contrôlable
- le niveau de lissage de la tendance-cycle est contrôlable
- robustesse aux valeurs aberrantes

Inconvénient : → ne s'applique qu'à la décomposition additive, pour généraliser, il faut faire une transformation

STL : Seasonal and Trend using Loess

Loess : forme de régression non-paramétrique

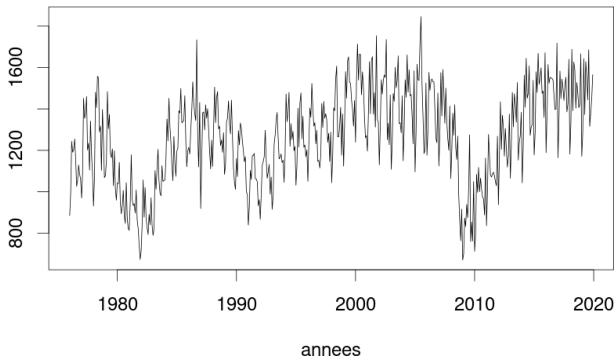
Avantages :

- Permet de prendre en compte tous les types de saisonnalité (mois, trimestre, semaine)
- la composante saisonnière peut changer dans le temps avec un taux contrôlable
- le niveau de lissage de la tendance-cycle est contrôlable
- robustesse aux valeurs aberrantes

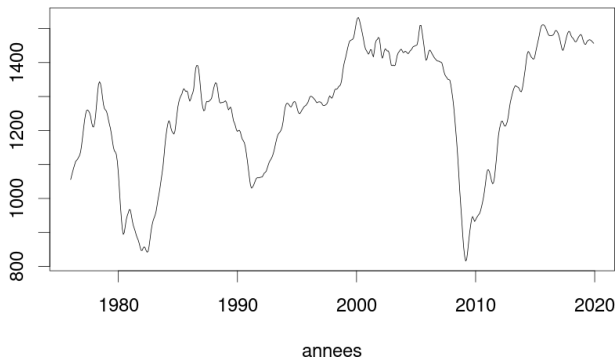
Inconvénient : → ne s'applique qu'à la décomposition additive, pour généraliser, il faut faire une transformation

Deux hyper-paramètres : contrôle du niveau de lissage de la composante tendance et de la composante saisonnière

Modèle additif pour les ventes de véhicules $X_t = T_t + S_t + I_t$

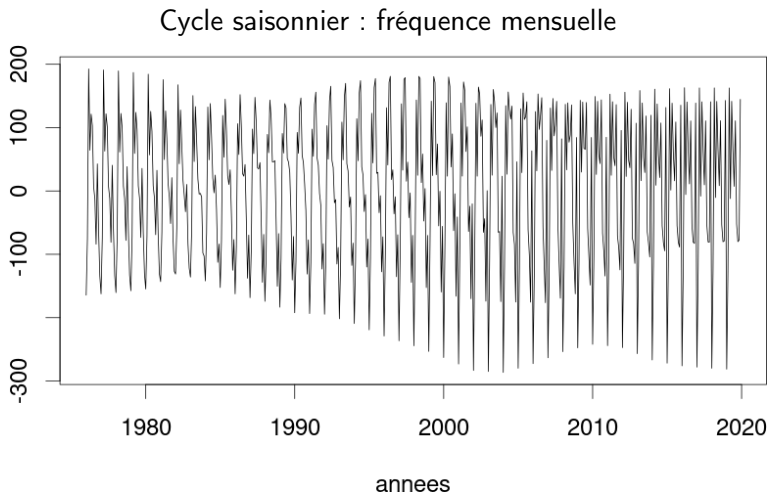


STL : composante tendance



→ Prochaine étape, on travaille avec $X_t - \hat{T}_t = S_t + I_t$

STL : composante saisonnière

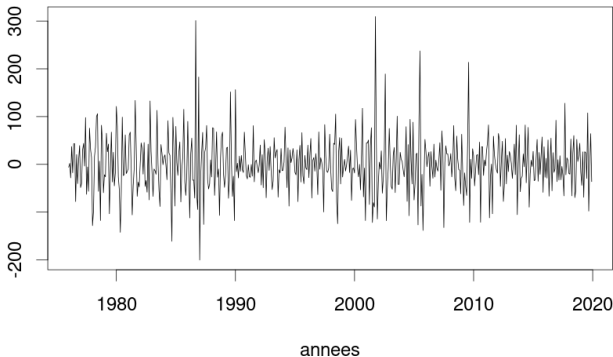


Cycle saisonnier constant : approche mieux adaptée au modèle additif

→ Prochaine étape, on travaille avec $X_t - \hat{T}_t - \hat{S}_t = I_t$

STL : composante résiduelle

→ La composante irrégulière est déduite des étapes précédentes



→ On cherche la décomposition qui laisse le moins de structure possible dans la composante résiduelle

MTH8304 - Apprentissage non-supervisé et séries chronologiques

12. Modèles ARIMA

Julie Carreau
julie.carreau@polymtl.ca

Département Mathématiques et Génie Industriel

Plan

- 1 Stationnarité et différenciation
- 2 Processus ARMA et ARIMA non-saisonniers
- 3 Modélisation et prévision
- 4 Autres modèles

- 1 Stationnarité et différenciation
- 2 Processus ARMA et ARIMA non-saisonniers
- 3 Modélisation et prévision
- 4 Autres modèles

Définition de série stationnaire

Intuitivement, une série chronologique $\{X_t; t = 0, \pm 1, \dots\}$ est **stationnaire** si ses propriétés statistiques sont les mêmes que celles de $\{X_{t+h}; t = 0, \pm 1, \dots\}$ pour tout $h \in \mathbb{N}$.

Définition de série stationnaire

Intuitivement, une série chronologique $\{X_t; t = 0, \pm 1, \dots\}$ est **stationnaire** si ses propriétés statistiques sont les mêmes que celles de $\{X_{t+h}; t = 0, \pm 1, \dots\}$ pour tout $h \in \mathbb{N}$.

En se restreignant aux propriétés liées **aux premier et au deuxième moments** de $\{X_t\}$ avec $E(X_t^2) < \infty$, on obtient :

- La fonction de **moyenne** de $\{X_t\}$

$$\mu_X(t) = E(X_t).$$

- La fonction de **covariance** de $\{X_t\}$

$$\gamma_X(r, s) = \text{Cov}(X_r, X_s) = E[(X_r - \mu_X(r))(X_s - \mu_X(s))] \quad \forall r, s \in \mathbb{N}.$$

Définition de série stationnaire

La série $\{X_t\}$ est dite **stationnaire** si

- ① $\mu_X(t)$ ne dépend pas de t (c'est une constante) ;
- ② $\gamma_X(t+h, t)$ ne dépend pas de t pour tout h .

Définition de série stationnaire

La série $\{X_t\}$ est dite **stationnaire** si

- ① $\mu_X(t)$ ne dépend pas de t (c'est une constante) ;
- ② $\gamma_X(t+h, t)$ ne dépend pas de t pour tout h .

➔ Graphiquement, une série stationnaire oscille autour d'une ligne horizontale avec un écart-type constant.

➔ Une série ayant une tendance ou une composante saisonnière dont l'amplitude change dans le temps est non-stationnaire.

Définition de fonctions d'autocovariance et d'autocorrélation

Pour $\{X_t\}$ une série **stationnaire**, on définit la **fonction d'autocovariance** de lag h :

$$\gamma_X(h) = \text{Cov}(X_{t+h}, X_t).$$

Propriétés

$$\gamma(0) \geq 0$$

$$|\gamma(h)| \leq \gamma(0) \quad \forall h$$

$$\gamma(h) = \gamma(-h) \quad \forall h \quad \text{c'est une fonction paire}$$

→ $\gamma(\cdot)$ est définie non-négative.

Définition de fonctions d'autocovariance et d'autocorrélation

Pour $\{X_t\}$ une série **stationnaire**, on définit la **fonction d'autocovariance** de lag h :

$$\gamma_X(h) = \text{Cov}(X_{t+h}, X_t).$$

Propriétés

$$\begin{aligned}\gamma(0) &\geq 0 \\ |\gamma(h)| &\leq \gamma(0) \quad \forall h \\ \gamma(h) &= \gamma(-h) \quad \forall h \quad \text{c'est une fonction paire}\end{aligned}$$

→ $\gamma(\cdot)$ est définie non-négative.

La **fonction d'autocorrélation** de lag h est :

$$\rho_X(h) = \frac{\gamma_X(h)}{\gamma_X(0)} = \text{Cor}(X_{t+h}, X_t).$$

Bruit blanc :

Soit $\{X_t\}$ une série de v.a. **non-correlées** telles que $E[X_t] = 0 \ \forall t$ et $E(X_t^2) = \sigma^2 < \infty$.

Alors $\{X_t\}$ est stationnaire et :

$$\gamma_X(h) = \begin{cases} \sigma^2 & \text{si } h = 0 \\ 0 & \text{sinon.} \end{cases}$$

Exemples de séries

Marche aléatoire :

Soit $\{X_t\}$ un bruit i.i.d : les v.a. sont **indépendantes**.

Une marche aléatoire $\{S_t; t = 0, 1, 2, \dots\}$ est définie par :

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

Exemples de séries

Marche aléatoire :

Soit $\{X_t\}$ un bruit i.i.d : les v.a. sont **indépendantes**.

Une marche aléatoire $\{S_t; t = 0, 1, 2, \dots\}$ est définie par :

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

Alors $E[S_t] = 0$ et $E[S_t^2] = t\sigma^2 < \infty \quad \forall t$.

Pour $h > 0$:

$$\begin{aligned}\gamma_S(t+h, t) &= \text{Cov}(S_{t+h}, S_t) \\ &= \text{Cov}(S_t + X_{t+1} + \dots + X_{t+h}, S_t) \\ &= \text{Cov}(S_t, S_t) = t\sigma^2.\end{aligned}$$

→ $\{S_t\}$ est **non-stationnaire**.

Exemples de séries

Moyenne mobile (MA) d'ordre 1 :

Soit la série définie par

$$X_t = Z_t + \theta Z_{t-1}, \quad t = 0, \pm 1, \dots$$

Avec $\{Z_t\}$ un bruit blanc de variance σ^2 et $\theta \in \mathbb{R}$.

Moyenne mobile (MA) d'ordre 1 :

Soit la série définie par

$$X_t = Z_t + \theta Z_{t-1}, \quad t = 0, \pm 1, \dots$$

Avec $\{Z_t\}$ un bruit blanc de variance σ^2 et $\theta \in \mathbb{R}$.

Alors $E[X_t] = 0$, $E[X_t^2] = \sigma^2(1 + \theta^2)$ et

$$\gamma_X(t+h, t) = \begin{cases} \sigma^2(1 + \theta^2) & \text{si } h = 0 \\ \sigma^2\theta & \text{si } h = \pm 1 \\ 0 & \text{sinon.} \end{cases}$$

Exemples de séries

Moyenne mobile (MA) d'ordre 1 :

Soit la série définie par

$$X_t = Z_t + \theta Z_{t-1}, \quad t = 0, \pm 1, \dots$$

Avec $\{Z_t\}$ un bruit blanc de variance σ^2 et $\theta \in \mathbb{R}$.

Alors $E[X_t] = 0$, $E[X_t^2] = \sigma^2(1 + \theta^2)$ et

$$\gamma_X(t+h, t) = \begin{cases} \sigma^2(1 + \theta^2) & \text{si } h = 0 \\ \sigma^2\theta & \text{si } h = \pm 1 \\ 0 & \text{sinon.} \end{cases}$$

→ $\{X_t\}$ est **stationnaire** et

$$\rho_X(h) = \begin{cases} 1 & \text{si } h = 0 \\ \frac{\theta}{1+\theta^2} & \text{si } h = \pm 1 \\ 0 & \text{sinon.} \end{cases}$$

Processus auto-régressif (AR) d'ordre 1 :

Supposons que $\{X_t\}$ soit une série stationnaire telle que

$$X_t = \phi X_{t-1} + Z_t \quad t = 0, \pm 1, \dots$$

Avec $\{Z_t\}$ un bruit blanc de variance σ^2 non-corrélé avec $\{X_t\}$ et $|\phi| < 1$.

Processus auto-régressif (AR) d'ordre 1 :

Supposons que $\{X_t\}$ soit une série **stationnaire** telle que

$$X_t = \phi X_{t-1} + Z_t \quad t = 0, \pm 1, \dots$$

Avec $\{Z_t\}$ un bruit blanc de variance σ^2 non-corrélé avec $\{X_t\}$ et $|\phi| < 1$.

Alors $E[X_t] = 0$ et $\gamma_X(h) = \phi^h \gamma_X(0)$.

Puisque $\gamma_X(h) = \gamma_X(-h)$, on a $\rho_X(h) = \phi^{|h|}$.

Fonction d'autocorrélation échantionnale

Soient $\{x_1, x_2, \dots, x_n\}$ des données observées.

Fonction d'autocorrélation échantionnale

Soient $\{x_1, x_2, \dots, x_n\}$ des données observées.

La fonction d'autocorrélation (ACF) échantillonnale permet d'estimer l'autocorrélation si la série est stationnaire et d'identifier un modèle.

→ Par exemple un modèle approprié pourrait être un bruit blanc si l'ACF est près de 0 pour tout $h > 0$.

Fonction d'autocorrélation échantionnale

Soient $\{x_1, x_2, \dots, x_n\}$ des données observées.

La fonction d'autocorrélation (ACF) échantillonnale permet d'estimer l'autocorrélation si la série est stationnaire et d'identifier un modèle.

➔ Par exemple un modèle approprié pourrait être un bruit blanc si l'ACF est près de 0 pour tout $h > 0$.

La **moyenne échantillonnale** est :

$$\bar{x} = \frac{1}{n} = \sum_{t=1}^n x_t.$$

La **fonction d'autocovariance échantillonnale** est :

$$\hat{\gamma}(h) = \frac{1}{n} \sum_{t=1}^{n-|h|} (x_{t+|h|} - \bar{x})(x_t - \bar{x}), \quad -n < h < n.$$

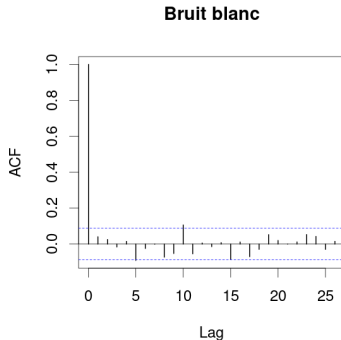
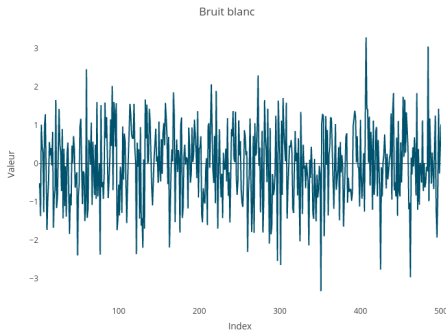
La **fonction d'autocorrélation échantillonnale** est :

$$\hat{\rho}(h) = \frac{\hat{\gamma}(h)}{\hat{\gamma}(0)}.$$

Exemple : bruit blanc

Séries de v.a. non-corrélées avec $\mu = 0$ et $\sigma = 1$ et

$$\gamma_X(h) = \begin{cases} \sigma^2 & \text{si } h = 0 \\ 0 & \text{sinon.} \end{cases}$$



→ Approximativement 95% des autocorrélations échantionnales doivent tomber entre les bornes $\pm 1.96/\sqrt{n}$

Exemple : marche aléatoire

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

$$\gamma_S(t+h, t) = t\sigma^2$$

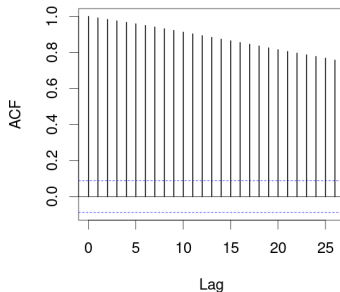
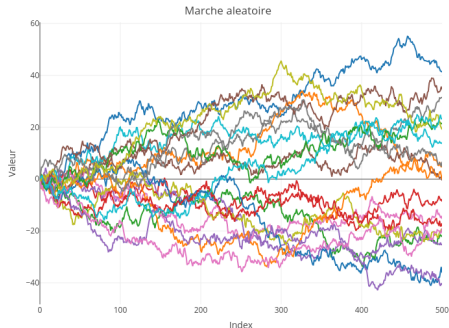
Exemple : marche aléatoire

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

$$\gamma_S(t+h, t) = t\sigma^2$$

Marche aleatoire



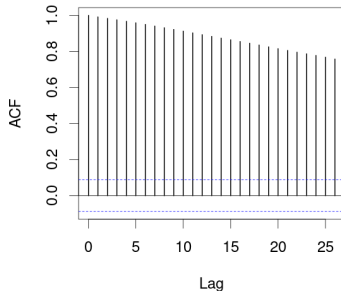
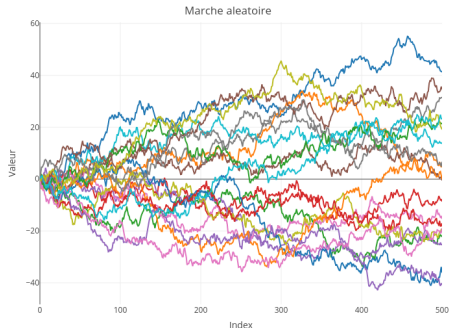
Exemple : marche aléatoire

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

$$\gamma_S(t+h, t) = t\sigma^2$$

Marche aleatoire

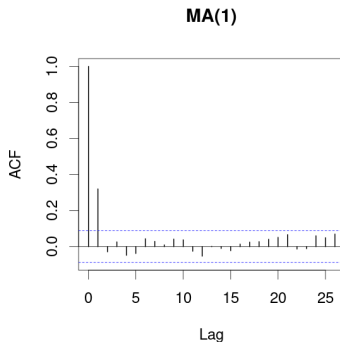
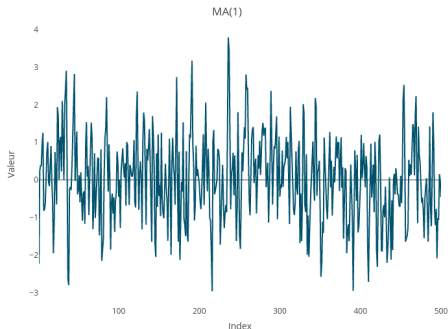


- ➔ Longues périodes de tendances apparentes à la hausse ou à la baisse
- ➔ Changements abruptes de direction
- ➔ La prévision d'une marche aléatoire : $\hat{S}_{t+h|t} = S_t$

Exemple : processus MA(1)

$$X_t = Z_t + \theta Z_{t-1}, \quad t = 0, \pm 1, \dots \quad \rho_X(h) = \begin{cases} 1 & \text{si } h = 0 \\ \frac{\theta}{1+\theta^2} & \text{si } h = \pm 1 \\ 0 & \text{sinon.} \end{cases}$$

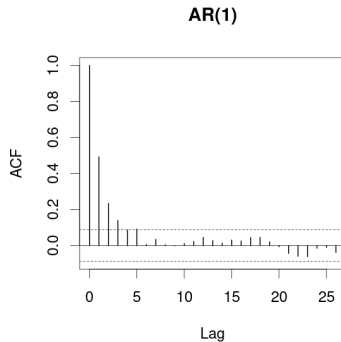
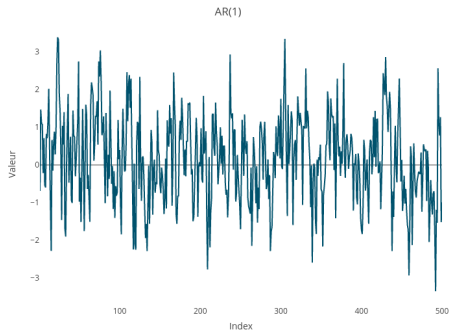
$$\theta = 0.5$$



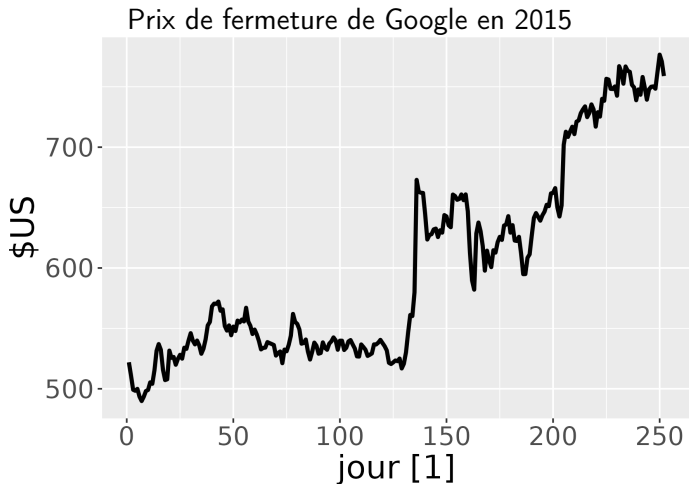
Exemple : processus AR(1)

$$X_t = \phi X_{t-1} + Z_t \quad t = 0, \pm 1, \dots \quad \rho_X(h) = \phi^{|h|}$$

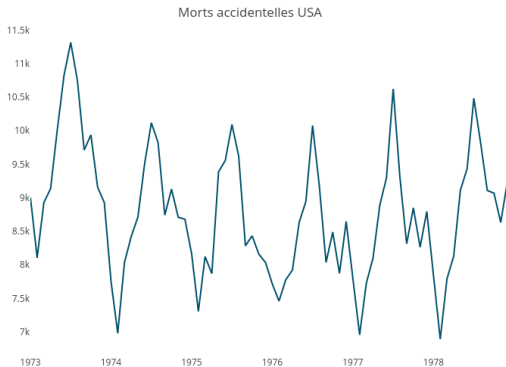
$$\phi = 0.5$$



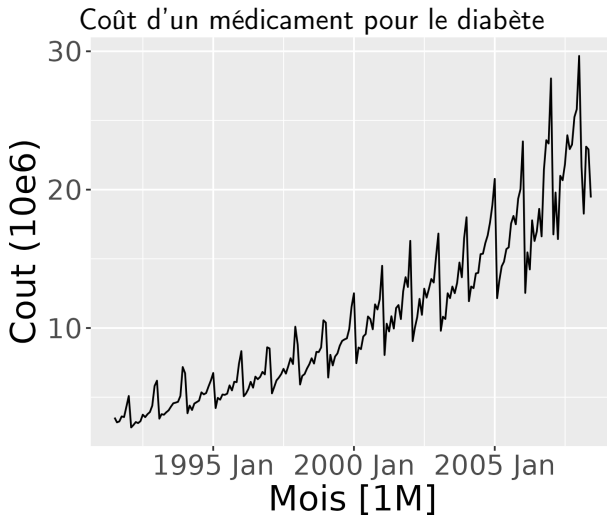
Séries stationnaires ?



Séries stationnaires ?



Séries stationnaires ?



Opération de différenciation : rendre une série stationnaire

Premier cas : enlever la composante de tendance

$$X_t = T_t + I_t$$

où T_t est la composante tendance et I_t est la composante irrégulière.

Opération de différenciation : rendre une série stationnaire

Premier cas : enlever la composante de tendance

$$X_t = T_t + I_t$$

où T_t est la composante tendance et I_t est la composante irrégulière.

Soit une différenciation d'ordre 1 :

$$\nabla X_t = X_t - X_{t-1}$$

Supposons que la tendance est linéaire : $T_t = c_0 + c_1 t$.

→ Appliquons ∇X_t

Opération de différenciation : rendre une série stationnaire

Premier cas : enlever la composante de tendance

$$X_t = T_t + I_t$$

où T_t est la composante tendance et I_t est la composante irrégulière.

Soit une différenciation d'ordre 1 :

$$\nabla X_t = X_t - X_{t-1}$$

Supposons que la tendance est linéaire : $T_t = c_0 + c_1 t$.

→ Appliquons ∇X_t

→ Effectuons une différenciation d'ordre 2 $\nabla^2 X_t = \nabla(\nabla X_t)$

Opération de différenciation : rendre une série stationnaire

Premier cas : enlever la composante de tendance

$$X_t = T_t + I_t$$

où T_t est la composante tendance et I_t est la composante irrégulière.

Soit une différenciation d'ordre 1 :

$$\nabla X_t = X_t - X_{t-1}$$

Supposons que la tendance est linéaire : $T_t = c_0 + c_1 t$.

→ Appliquons ∇X_t

→ Effectuons une différenciation d'ordre 2 $\nabla^2 X_t = \nabla(\nabla X_t)$

→ Si la tendance suit un polynôme de degré k , on peut le réduire à une constante en appliquant k fois l'opérateur de différenciation : $\nabla^k X_t$.

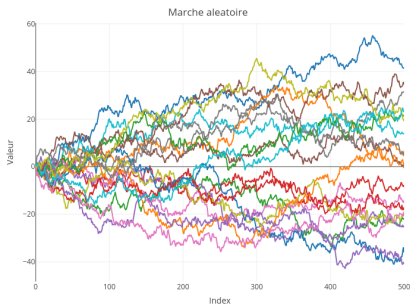
En pratique, on veut éviter d'inclure une tendance polynômiale et $k \leq 2$

Exemple : marches aléatoires

Soit $\{X_t\}$ un bruit i.i.d.

$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

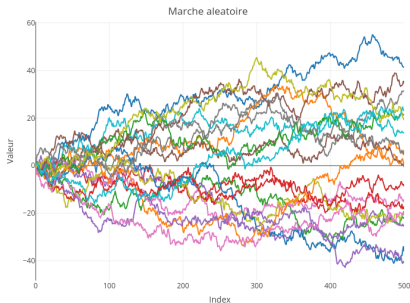


Exemple : marches aléatoires

Soit $\{X_t\}$ un bruit i.i.d.

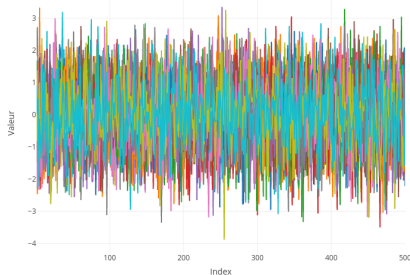
$$S_0 = 0$$

$$S_t = X_1 + X_2 + \dots + X_t \quad t = 1, 2, \dots$$

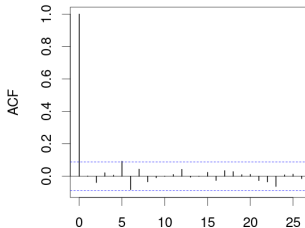


Après une différenciation d'ordre 1

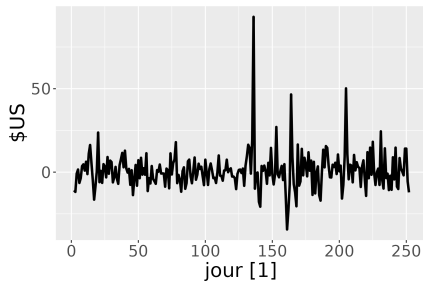
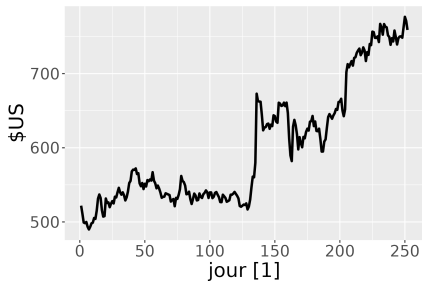
Marche aléatoire avec différenciation d'ordre 1



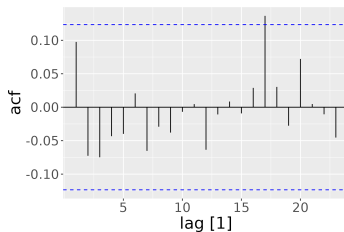
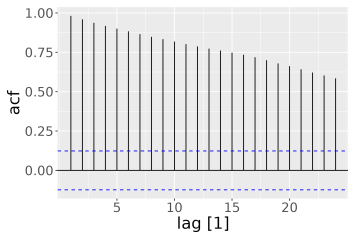
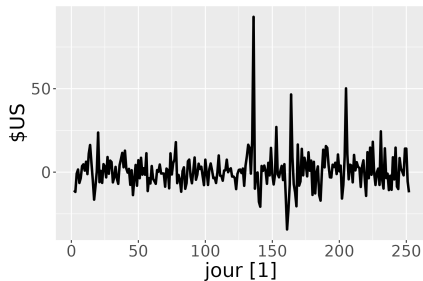
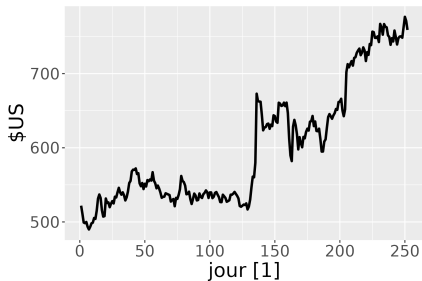
Marche aléatoire



Exemple : prix fermeture Google 2015



Exemple : prix fermeture Google 2015



Opération de différenciation : rendre une série stationnaire

Deuxième cas : enlever la composante saisonnière

$$X_t = T_t + S_t + I_t$$

où T_t est la composante tendance, S_t est la composante irrégulière et I_t est la composante irrégulière.

Opération de différenciation : rendre une série stationnaire

Deuxième cas : enlever la composante saisonnière

$$X_t = T_t + S_t + I_t$$

où T_t est la composante tendance, S_t est la composante irrégulière et I_t est la composante irrégulière.

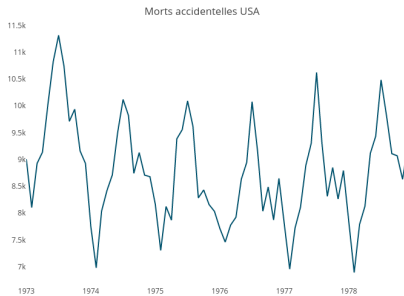
Soit une différenciation d'ordre 1 de lag d :

$$\nabla_d X_t = X_t - X_{t-d}.$$

Si d est la période de S_t ($S_t = S_{t+dk}, \forall k \in \mathbb{N}$), alors :

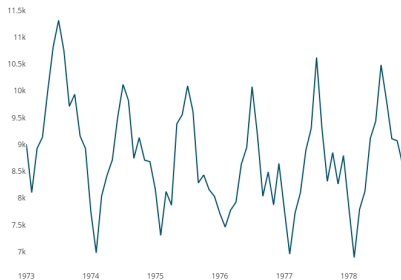
$$\nabla_d X_t = \underbrace{T_t - T_{t-d}}_{\text{tendance}} + \underbrace{I_t - I_{t-d}}_{\text{bruit}}.$$

Exemple : morts accidentelles

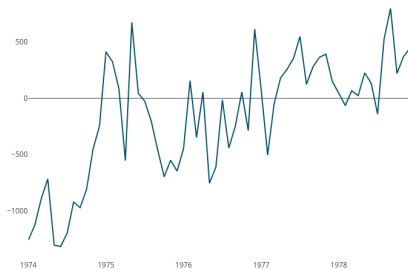


Exemple : morts accidentelles

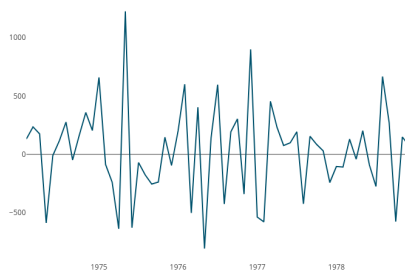
Morts accidentelles USA



$\nabla_{12}X_t$
Morts accidentelles USA



$\nabla(\nabla_{12}X_t)$
Morts accidentelles USA



- 1 Stationnarité et différenciation
- 2 Processus ARMA et ARIMA non-saisonniers
- 3 Modélisation et prévision
- 4 Autres modèles

Processus linéaires stationnaires : modèles pour des séries stationnaires

Processus linéaires stationnaires : modèles pour des séries stationnaires

Deux outils importants pour identifier la structure d'un processus ARMA(p,q) sont :

- la fonction d'autocorrélation (ACF)

$$\rho(h) = \frac{\gamma(h)}{\gamma(0)}$$

- la fonction d'autocorrélation partielle (PACF) qui enlève l'effet des lags 1 à $k - 1$ dans le calcul de la corrélation de lag k .

Processus linéaires stationnaires : modèles pour des séries stationnaires

Deux outils importants pour identifier la structure d'un processus ARMA(p,q) sont :

- la fonction d'autocorrélation (ACF)

$$\rho(h) = \frac{\gamma(h)}{\gamma(0)}$$

- la fonction d'autocorrélation partielle (PACF) qui enlève l'effet des lags 1 à $k - 1$ dans le calcul de la corrélation de lag k .

→ Processus auto-régressif ARMA(p,0) et moyenne mobile ARMA(0,q)

Processus auto-régressif AR(p)

Généralisation du processus AR(1) à p de lags :

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + Z_t \quad t = 0, \pm 1, \dots$$

c : dérive du processus

ϕ_i : coefficient du lag i de X_t avec une condition de stationnarité

$\{Z_t\}$ un bruit blanc de variance σ^2

Processus auto-régressif AR(p)

Généralisation du processus AR(1) à p de lags :

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + Z_t \quad t = 0, \pm 1, \dots$$

c : dérive du processus

ϕ_i : coefficient du lag i de X_t avec une condition de stationnarité

$\{Z_t\}$ un bruit blanc de variance σ^2

Pour un processus AR(1) :

- avec $-1 < \phi_1 < 1$, X_t est stationnaire
dans le cas $p > 1$, des restrictions sur ϕ_i , $1 \leq i \leq p$ assurent la stationnarité
- avec $\phi_1 = 0$ et $c = 0$, X_t est un bruit blanc
- avec $\phi_1 = 1$ et $c = 0$, X_t est une marche aléatoire

On se restreint aux cas où les processus $AR(p)$ sont stationnaires

→ Identification de la structure d'un processus $AR(p)$:

- Fonction d'autocorrélation (ACF) : diminution exponentielle ou sinusoïdale ;
- Fonction d'autocorrélation partielle (PACF) : disparaît après le lag p .

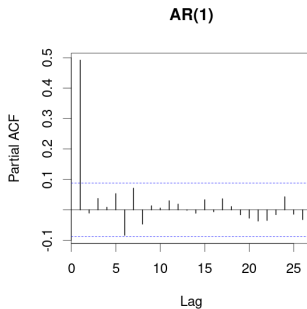
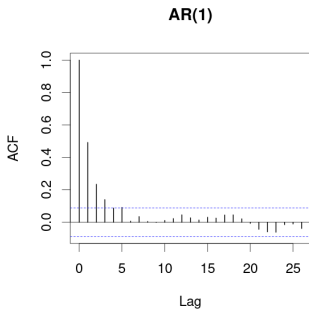
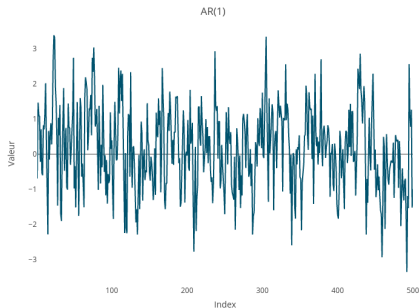
Processus AR(p) : exemple avec $p = 1$

$$X_t = \phi_1 X_{t-1} + Z_t$$

$$-1 < \phi_1 < 1$$

$$\phi_1 = 0.5$$

Stationnarité : $-1 < \phi_1 < 1$

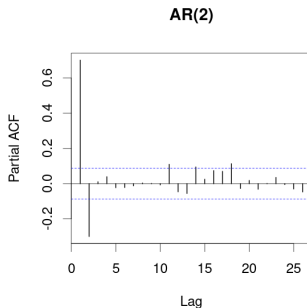
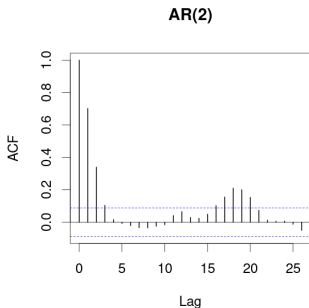
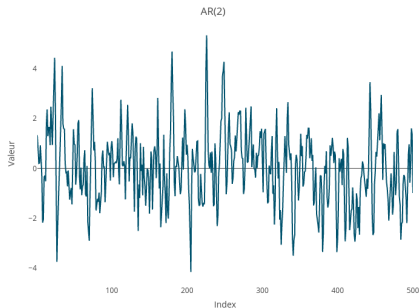


Processus AR(p) : exemple avec $p = 2$

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + Z_t$$

$$\phi_1 = 0.9 \quad \phi_2 = -0.3$$

Stationnarité : $-1 < \phi_2 < 1$,
 $\phi_1 + \phi_2 < 1$, $\phi_2 - \phi_1 < 1$



Processus MA(q)

Généralisation du processus MA(1) à q de lags :

$$X_t = c + Z_t + \sum_{i=1}^q \theta_i Z_{t-i} \quad t = 0, \pm 1, \dots$$

c : dérive du processus

$\{Z_t\}$ un bruit blanc de variance σ^2

θ_i : coefficient du lag i de Z_t

Processus MA(q)

Généralisation du processus MA(1) à q de lags :

$$X_t = c + Z_t + \sum_{i=1}^q \theta_i Z_{t-i} \quad t = 0, \pm 1, \dots$$

c : dérive du processus

$\{Z_t\}$ un bruit blanc de variance σ^2

θ_i : coefficient du lag i de Z_t

➔ Identification de la structure d'un processus MA(q) :

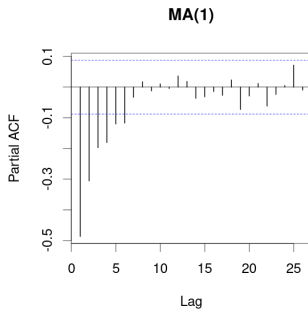
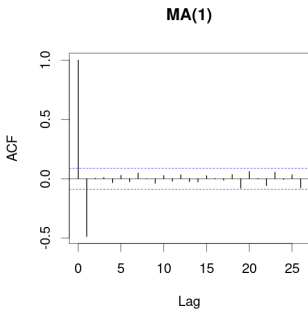
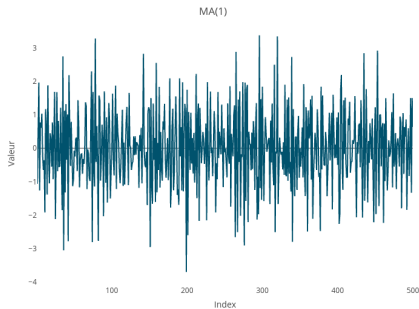
- Fonction d'autocorrélation (ACF) : disparaît après le lag q ;
- Fonction d'autocorrélation partielle (PACF) : diminution exponentielle ou sinusoïdale.

Processus MA(q) : exemple avec $q = 1$

$$X_t = Z_t + \theta_1 Z_{t-1}$$

$$\theta_1 = -0.7$$

Invertibilité : $-1 < \theta_1 < 1$



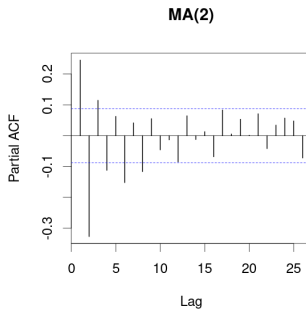
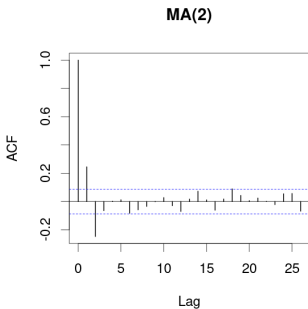
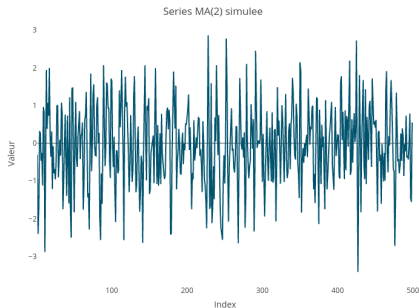
Processus MA(q) : exemple avec $q = 2$

$$X_t = Z_t + \theta_1 Z_{t-1} + \theta_2 Z_{t-2}$$

$$\theta_1 = 0.5 \quad \theta_2 = -0.3$$

Invertibilité : $-1 < \theta_2 < 1$

$$\theta_2 + \theta_1 > -1, \theta_1 - \theta_2 < 1$$



Processus ARMA(p,q)

Combinaison d'un processus AR(p) et MA(q) :

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + Z_t + \sum_{i=1}^q \theta_i Z_{t-i} \quad t = 0, \pm 1, \dots$$

c : dérive du processus ϕ_i, θ_i : coefficient du lag i

$\{Z_t\}$ un bruit blanc de variance σ^2

Processus ARMA(p,q)

Combinaison d'un processus AR(p) et MA(q) :

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + Z_t + \sum_{i=1}^q \theta_i Z_{t-i} \quad t = 0, \pm 1, \dots$$

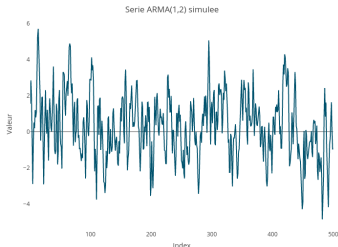
c : dérive du processus ϕ_i, θ_i : coefficient du lag i

$\{Z_t\}$ un bruit blanc de variance σ^2

Exemple : processus ARMA(1,2)

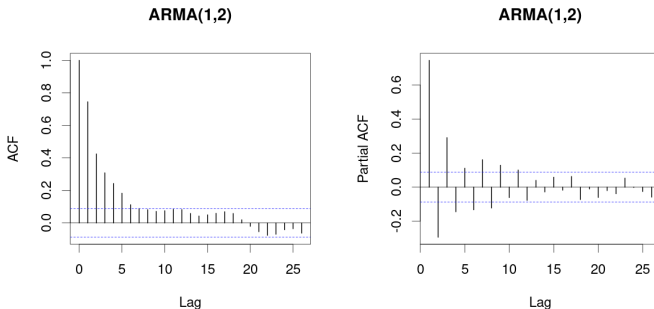
$$X_t = \phi_1 X_{t-1} + Z_t + \theta_1 Z_{t-1} + \theta_2 Z_{t-2} \quad t = 0, \pm 1, \dots$$

Avec $\phi_1 = 0.7$ et $\theta_1 = 0.5, \theta_2 = -0.3$



Exemple : processus ARMA(1,2)

Graphiques ACF et PACF



- ➔ Les deux graphiques décroissent lentement.
- ➔ Pas de rupture qui puisse nous indiquer l'ordre approprié.

Processus $ARIMA(p,d,q)$

Généralisation des processus $ARMA(p, q)$:

→ Un processus est appelé $ARIMA(p, d, q)$ s'il se réduit à un processus $ARMA(p, q)$ après un nombre fini d de différentiation.

Processus $ARIMA(p,d,q)$

Généralisation des processus $ARMA(p, q)$:

→ Un processus est appelé $ARIMA(p, d, q)$ s'il se réduit à un processus $ARMA(p, q)$ après un nombre fini d de différentiation.

Cas particuliers

- Bruit blanc : $ARIMA(0, 0, 0)$ sans dérive
- Marche aléatoire : $ARIMA(0, 1, 0)$ sans dérive
- Processus $ARMA(p, q)$: $ARIMA(p, 0, q)$

Processus ARIMA(p, d, q)

Généralisation des processus $ARMA(p, q)$:

→ Un processus est appelé $ARIMA(p, d, q)$ s'il se réduit à un processus $ARMA(p, q)$ après un nombre fini d de différenciation.

Cas particuliers

- Bruit blanc : $ARIMA(0, 0, 0)$ sans dérive
 - Marche aléatoire : $ARIMA(0, 1, 0)$ sans dérive
 - Processus $ARMA(p, q)$: $ARIMA(p, 0, q)$
- Estimation des paramètres par maximum de vraisemblance
- Sélection de p , d et q peut suivre plusieurs stratégies
(on verra plus loin)

Processus ARIMA(p,d,q)

Influence de la dérive c sur le comportement du processus ARIMA

Lorsque $c = 0$

- $d = 0$: les prévisions sur le long-terme vont tendre vers zéro
- $d = 1$: les prévisions sur le long-terme vont tendre vers une constante non-nulle
- $d = 2$: les prévisions sur le long-terme vont suivre une ligne droite

Processus ARIMA(p,d,q)

Influence de la dérive c sur le comportement du processus ARIMA

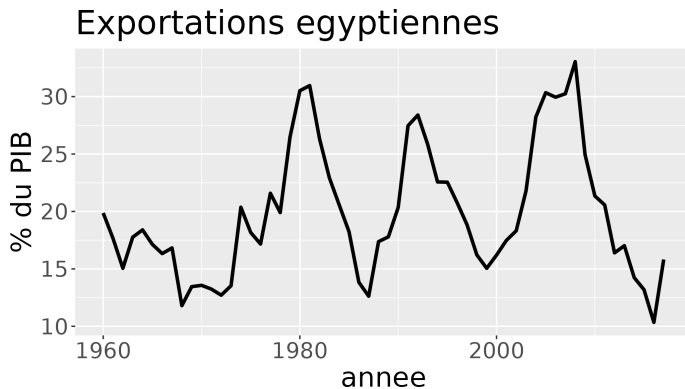
Lorsque $c = 0$

- $d = 0$: les prévisions sur le long-terme vont tendre vers zéro
- $d = 1$: les prévisions sur le long-terme vont tendre vers une constante non-nulle
- $d = 2$: les prévisions sur le long-terme vont suivre une ligne droite

Lorsque $c \neq 0$

- $d = 0$: les prévisions sur le long-terme vont tendre vers la moyenne des données
 - $d = 1$: les prévisions sur le long-terme vont suivre une ligne droite
 - $d = 2$: les prévisions sur le long-terme vont suivre une tendance quadratique
- pas permis dans la librairie

Exemple : exportations égyptiennes



Exemple : exportations égyptiennes

Ajustement automatique $ARIMA(p, d, q)$

Series: Exports

Model: ARIMA(2,0,1) w/ mean

Coefficients:

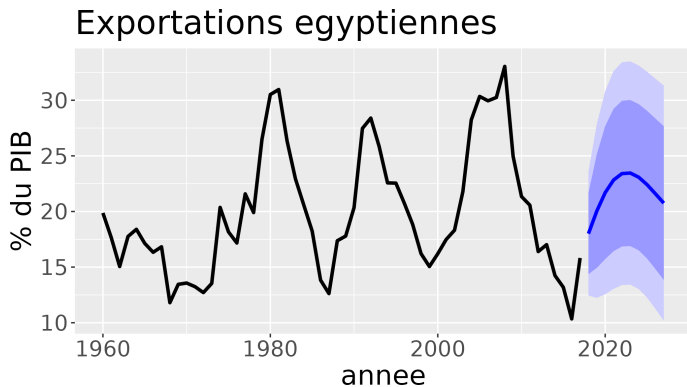
	ar1	ar2	ma1	constant
	1.6764	-0.8034	-0.6896	2.5623
s.e.	0.1111	0.0928	0.1492	0.1161

sigma² estimated as 8.046: log likelihood=-141.57

AIC=293.13 AICc=294.29 BIC=303.43

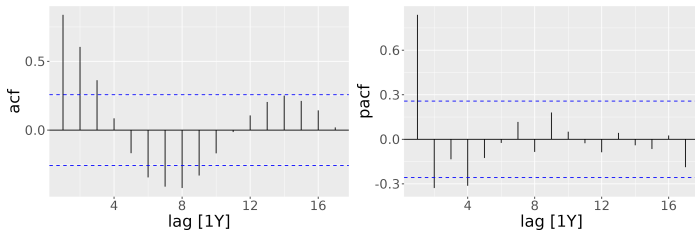
$$X_t = 2.56 + 1.68X_{t-1} - 0.80X_{t-2} - 0.69Z_{t-1} + Z_t$$

Exemple : exportations égyptiennes

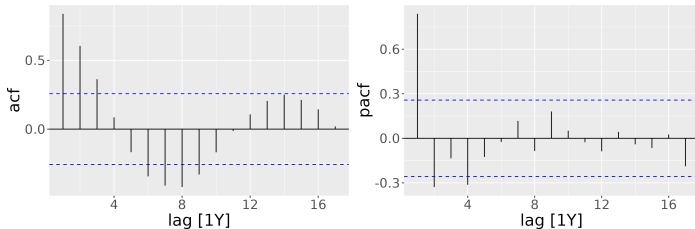


$c \neq 0$ $d = 0$: prévisions tendent vers la moyenne des données

Exemple : exportations égyptiennes



Exemple : exportations égyptiennes



Ajustement manuel $ARIMA(4, 0, 0)$

```
model(ARIMA(Exports ~ pdq(4,0,0)))
```

```
sigma^2 estimated as 7.885:  log likelihood=-140.53  
AIC=293.05   AICc=294.7   BIC=305.41
```

- 1 Stationnarité et différenciation
- 2 Processus ARMA et ARIMA non-saisonniers
- 3 Modélisation et prévision**
- 4 Autres modèles

Sélection du niveau de différences d

Graphique temporel

→ l'opération de différences sert à stabiliser la moyenne en enlevant des changements dans le niveau de la série

d est bien choisi s'il n'y a plus de tendance ni de saisonnalité apparente

Sélection du niveau de différences d

Graphique temporel

→ l'opération de différences sert à stabiliser la moyenne en enlevant des changements dans le niveau de la série

d est bien choisi s'il n'y a plus de tendance ni de saisonnalité apparente

Graphique d'auto-corrélation

→ Pour un série stationnaire, les r_k décroissent rapidement

Sélection du niveau de différences d

Graphique temporel

→ l'opération de différences sert à stabiliser la moyenne en enlevant des changements dans le niveau de la série

d est bien choisi s'il n'y a plus de tendance ni de saisonnalité apparente

Graphique d'auto-corrélation

→ Pour une série stationnaire, les r_k décroissent rapidement

Test de racine unitaire

H_0 : les données sont stationnaires

Prix de fermeture de Google :

statistique de test KPSS 3.56 et $\text{valeur} - p = 0.01$

Après une différence de lag 1 :

statistique de test KPSS 0.0989 et $\text{valeur} - p = 0.1$

→ Permet d'automatiser la sélection de d

Critère d'information de Akaike (AIC)

$$-2 \ln(\text{vraisemblance}) + 2 \underbrace{(p + q + k + 1)}_{\# \text{ paramètres}}$$

$k = 1$ ($k = 0$) si $c \neq 0$ ($c = 0$) et la variance des résidus σ^2

Critère d'information de Akaike (AIC)

$$-2 \ln(\text{vraisemblance}) + 2 \underbrace{(p + q + k + 1)}_{\# \text{ paramètres}}$$

$k = 1$ ($k = 0$) si $c \neq 0$ ($c = 0$) et la variance des résidus σ^2

Critère d'information de Akaike corrigé (AICc)

$$AIC + \frac{2(p + q + k + 1)(p + q + k + 2)}{T - p - q - k - 2}$$

Critère d'information de Akaike (AIC)

$$-2 \ln(\text{vraisemblance}) + 2 \underbrace{(p + q + k + 1)}_{\# \text{ paramètres}}$$

$k = 1$ ($k = 0$) si $c \neq 0$ ($c = 0$) et la variance des résidus σ^2

Critère d'information de Akaike corrigé (AICc)

$$AIC + \frac{2(p + q + k + 1)(p + q + k + 2)}{T - p - q - k - 2}$$

Critère d'information de Bayes (BIC)

$$AIC + (\ln(T) - 2)(p + q + k + 1)$$

Critère d'information de Akaike (AIC)

$$-2 \ln(\text{vraisemblance}) + 2 \underbrace{(p + q + k + 1)}_{\# \text{ paramètres}}$$

$k = 1$ ($k = 0$) si $c \neq 0$ ($c = 0$) et la variance des résidus σ^2

Critère d'information de Akaike corrigé (AICc)

$$AIC + \frac{2(p + q + k + 1)(p + q + k + 2)}{T - p - q - k - 2}$$

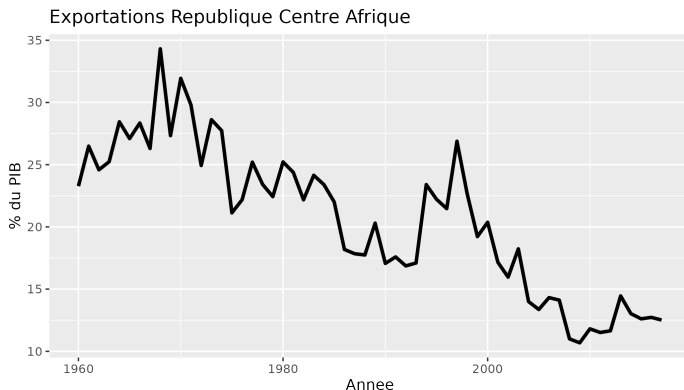
Critère d'information de Bayes (BIC)

$$AIC + (\ln(T) - 2)(p + q + k + 1)$$

- Des critères faibles indiquent de meilleurs modèles
- Dans le livre, AICc est privilégié
- Critères valables pour sélectionner p et q avec d fixé
les différences changent la vraisemblance

Étapes de modélisation : exportations Centre-Afrique

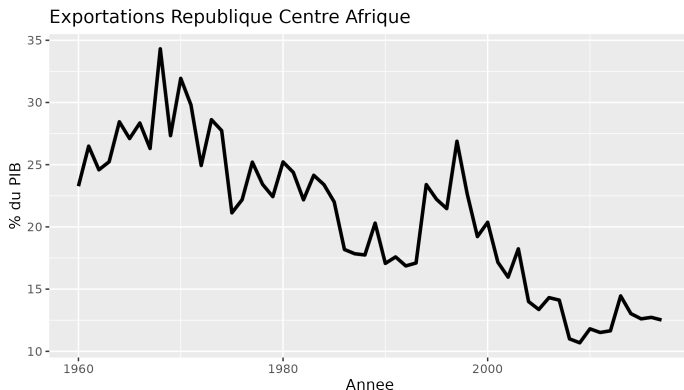
Étape 1 : Graphique temporel des données



→ Non-stationnarité : tendance à la baisse

Étapes de modélisation : exportations Centre-Afrique

Étape 1 : Graphique temporel des données



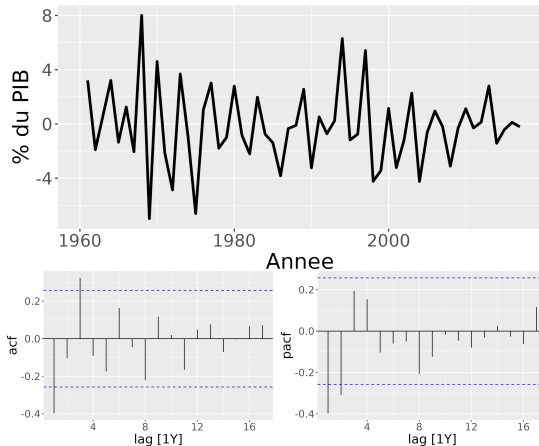
→ Non-stationnarité : tendance à la baisse

Étape 2 : Transformation de Box-Cox

→ Stabilisation de la variance : pas nécessaire dans ce cas

Étapes de modélisation : exportations Centre-Afrique

Étape 3 : Application de l'opérateur de différences



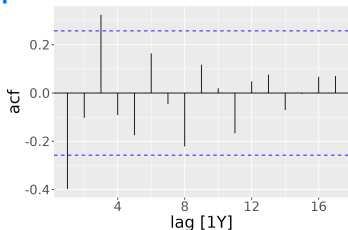
Test de racine unitaire (stationnarité) :

Série originale : statistique = 1.28 et valeur-p = 0.01

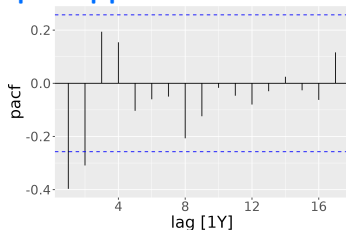
Différence de lag 1 : statistique = 0.0922 et valeur-p = 0.1

Étapes de modélisation : exportations Centre-Afrique

Étape 4 : Identification des ordres p et q potentiels

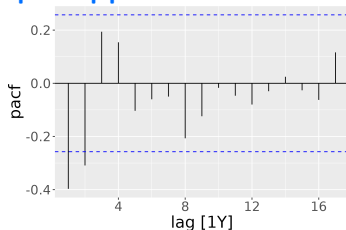
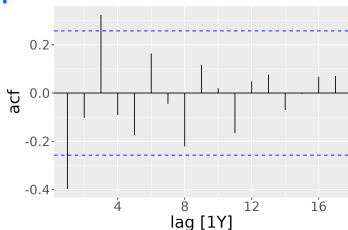


ACF suggère un modèle MA(3)



PACF suggère un modèle AR(2)

Étape 4 : Identification des ordres p et q potentiels



ACF suggère un modèle MA(3)

PACF suggère un modèle AR(2)

Étape 5 : Ajustement de modèles

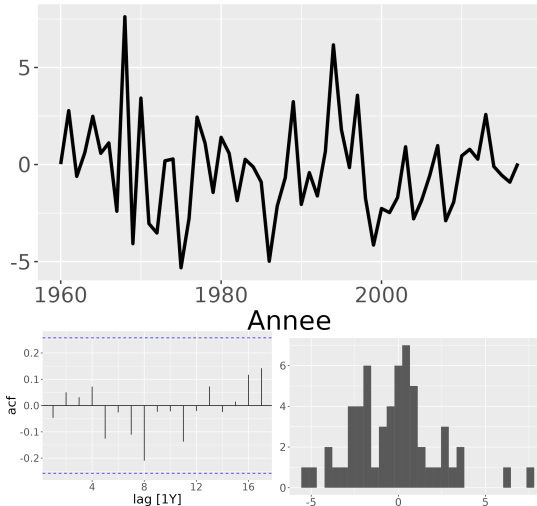
Modèles ARIMA(0,1,3) et ARIMA(2,1,0) d'après ACF et PACF

Modèles ARIMA(2,1,2) et ARIMA(3,1,0) par recherche automatique

AICc presque identique pour ces quatre modèles

Étapes de modélisation : exportations Centre-Afrique

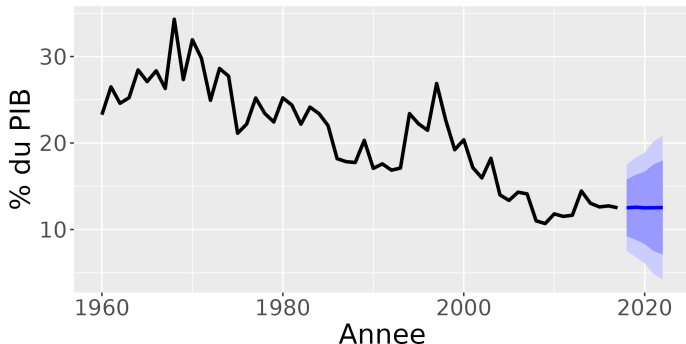
Étape 6 : Analyse des résidus : modèle ARIMA(3,1,0)



Test de portmanteau (auto-corrélation) : Statistique 5.75 Valeur-p 0.569

Étapes de modélisation : exportations Centre-Afrique

Étape 7 : Prédiction



➔ Semblable à la méthode naïve (marche aléatoire) mais intervalles de prévision plus étroits

Étape 7 : Prévision - calculs

Modèle ARIMA(3,1,0) ajusté :

$$\nabla X_t = -0.44\nabla X_{t-1} - 0.19\nabla X_{t-2} + 0.21\nabla X_{t-3} + Z_t$$

En utilisant $\nabla X_t = X_t - X_{t-1}$ et en ré-arrangeant :

$$X_t = 0.56X_{t-1} + 0.25X_{t-2} + 0.4X_{t-3} - 0.21X_{t-4} + Z_t$$

➔ Montrer que $\hat{x}_{T+1} = 12.50$ et $\hat{x}_{T+2} = 12.61$ sachant que $x_T = 12.52$, $x_{T-1} = 12.73$, $x_{T-2} = 12.61$, $x_{T-3} = 13.03$

Étape 7 : Prévision - calculs

Modèle ARIMA(3,1,0) ajusté :

$$\nabla X_t = -0.44\nabla X_{t-1} - 0.19\nabla X_{t-2} + 0.21\nabla X_{t-3} + Z_t$$

En utilisant $\nabla X_t = X_t - X_{t-1}$ et en ré-arrangeant :

$$X_t = 0.56X_{t-1} + 0.25X_{t-2} + 0.4X_{t-3} - 0.21X_{t-4} + Z_t$$

→ Montrer que $\hat{x}_{T+1} = 12.50$ et $\hat{x}_{T+2} = 12.61$ sachant que $x_T = 12.52$, $x_{T-1} = 12.73$, $x_{T-2} = 12.61$, $x_{T-3} = 13.03$

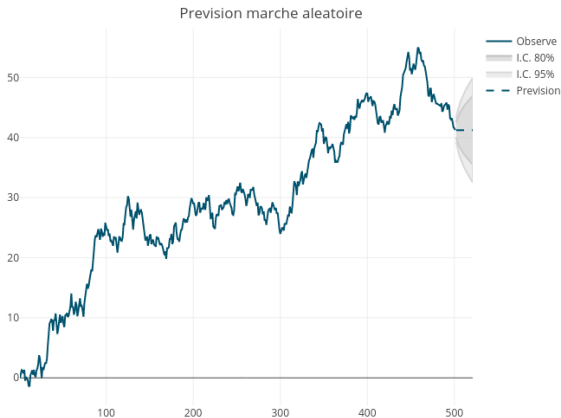
Intervalles de prévisions théoriques : valables si les résidus suivent une loi normale et sont non-corrélés

→ expressions pour le cas général assez complexes

Si les résidus ne suivent pas une loi normale mais sont non-corrélés :

→ méthode du bootstrap

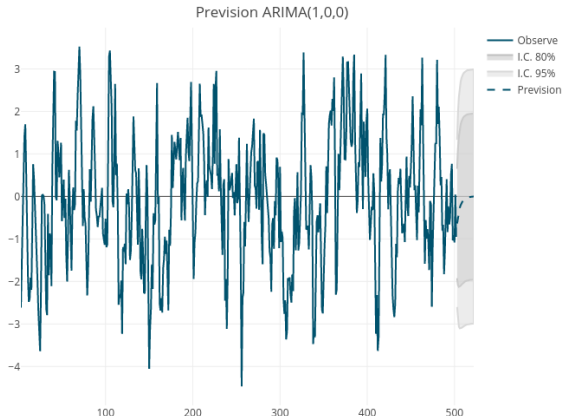
Prévision : marche aléatoire ou ARIMA(0,1,0)



→ Équivalent à la méthode naïve

Prévision : ARIMA(1,1,0)

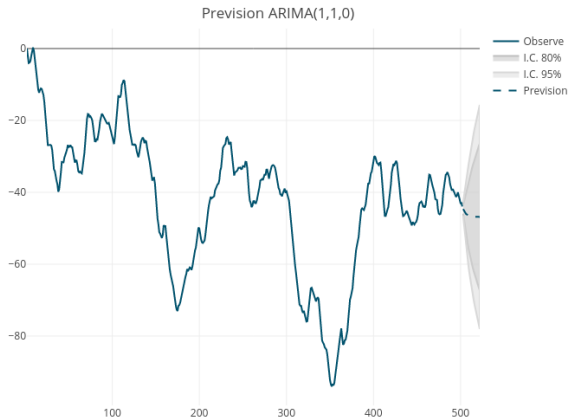
Prévision pour une série différenciée d'ordre 1 qui suit un processus ARMA(1,0) :



→ La prévision se comporte comme une puissance de ϕ : $\hat{x}_{T+h} = \phi^h x_T$.

Prévision : ARIMA(1,1,0)

Prévision pour la série non-différenciée qui suit un processus ARIMA(1,1,0) :



→ La prévision se base sur la dernière observation à laquelle on additionne la prévision pour le processus AR(1) sous-jacent.

- 1 Stationnarité et différenciation
- 2 Processus ARMA et ARIMA non-saisonniers
- 3 Modélisation et prévision
- 4 Autres modèles

Ajout de termes saisonniers dans les processus ARIMA :

$$ARIMA \quad \underbrace{(p, d, q)}_{\text{non-saisonnier}} + \underbrace{(P, D, Q)_m}_{\text{saisonnier}}$$

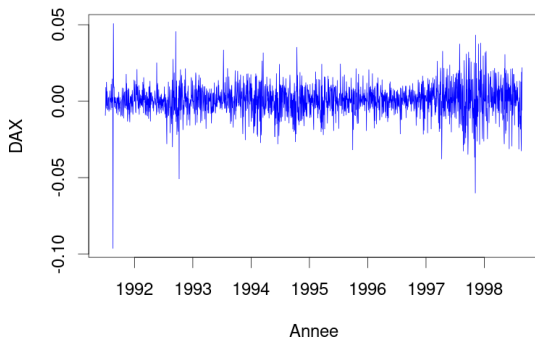
avec m la période saisonnière ($m = 12$ pour des données mensuelles)

- Les opérations de différences, d'auto-régression et de moyennes mobiles sont appliquées aux lags saisonniers (des multiples de m)
- Pour identifier D , on vérifie la stationnarité (on applique la différence saisonnière avant la différence de lag 1)
- Pour identifier P et Q , dans les graphiques ACF et PACF, on examine le comportement aux lags saisonniers

Processus ARCH-GARCH : séries financières

Série des log-rendements de l'indice DAX :

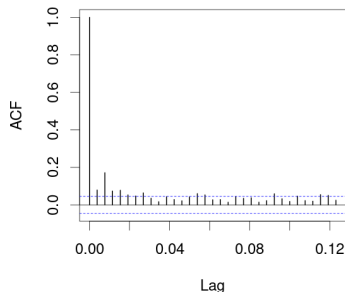
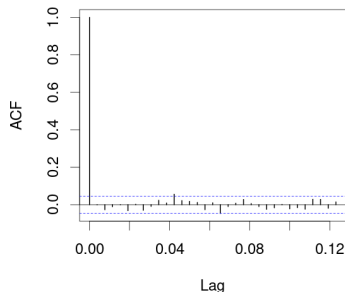
$$Z_t = \nabla \ln(P_t) = \ln(P_t) - \ln(P_{t-1})$$



→ Semblable à du bruit blanc mais présence de **volatilité persistente ou hétéroscédasticité** et de **dépendance sans corrélation**

Processus ARCH-GARCH : séries financières

Auto-corrélation de $Z_t = \nabla \ln(P_t) = \ln(P_t) - \ln(P_{t-1})$ et de Z_t^2



- Pas de corrélation présente mais série qui n'est pas **indépendante**
- Développement de modèles ARCH-GARCH pour représenter et exploiter cette dépendance